Modern AI Foundations

A Comprehensive Introduction to Modern AI Foundations:
From Classical Machine Learning to Large Language Models



An code-first guide to the ideas behind modern artificial intelligence: the mathematics that powers it, the classical machine-learning engine at its heart, and the deep-learning revolution that brought us transformers and large language models , with curated sources for every step of the journey.

Prerequisites: Intermediate Python • Basic NumPy & Pandas • Calculus & Linear Algebra Basics

Course Design Team

Version 1.1 • September 2025 • License: Text under CC BY 4.0 • Code under MIT

Repository: https://github.com/DanielBerns/AI-course

Contents

About this course 4

Prerequisites and required background 4

1 Introduction and the AI Landscape 6
- 1.1 Defining intelligence: weak vs. strong AI . . . 6
- 1.2 From expert systems to data-driven AI . . . 6
- 1.3 The modern stack: compute, cloud, and data pipelines . . . 7
- 1.4 Ethics, safety, and security: the non-optional module . . . 8

2 Mathematics and Data Foundations 11
- 2.1 Linear algebra: the language of data . . . 11
- 2.2 Probability and statistics: the language of uncertainty . . . 19
- 2.3 Optimisation: how models learn . . . 20

3 Classical Machine Learning: The Core Engine 23
- 3.1 Supervised learning . . . 23
- 3.2 Unsupervised learning . . . 30
- 3.3 Self-supervised learning: data self-annotation . . . 31
- 3.4 Reinforcement learning . . . 32
- 3.5 Modern evaluation methodology & diagnostics . . . 33

4 Transition to Deep Learning and Modern AI 35
- 4.1 Neural networks: perceptrons, activations, backpropagation . . . 35

5 Network Architecture and Forward Pass 36

6 The Loss Function and the Goal 36

7 The Error Term (δ) 37

8 Deriving Gradients using the Chain Rule 37
- 8.1 Gradient with respect to Weights . . . 37
- 8.2 Gradient with respect to Biases . . . 37

9 Backpropagating the Error 38
- 9.1 Step 1: Error at the Output Layer ($l = L$) . . . 38
- 9.2 Step 2: Error at Hidden Layers ($l < L$) . . . 38

10 Summary of the Backpropagation Algorithm **39**
10.1 Representation learning: from manual engineering to latent spaces 42
10.2 Architectures: CNNs, RNNs, and transformers 45
10.3 Transformer architecture variants: encoder, decoder, and encoder–decoder 47

11 Foundation Models and Generative AI **50**
11.1 Self-supervised learning and subword tokenization 50
11.2 Scaling laws and generative architectures 54
11.3 Fine-tuning, parameter efficiency, and preference alignment 55
11.4 Foundation model inference, KV-caching, and RAG 56
11.5 Ethics, alignment, and safety revisited: the generative AI frontier 56

12 Modern LLM Architectures **59**
12.1 Core modifications in modern decoder-only LLMs 59
12.2 Grouped-Query Attention (GQA) 59
12.3 Mixture-of-Experts (MoE) 60

13 Efficient Attention Mechanisms **62**
13.1 The memory bottleneck: IO complexity of self-attention 62
13.2 FlashAttention: IO-aware tiled attention 62
13.3 Sliding-Window Attention (SWA) 62
13.4 Positional encoding and length extrapolation 63

14 Model Quantization **64**
14.1 Quantization fundamentals 64
14.2 GPTQ: second-order post-training quantization 64
14.3 AWQ: activation-aware weight quantization 65
14.4 bitsandbytes and NF4 65

15 Distributed Training **67**
15.1 Data parallelism and the All-Reduce bottleneck 67
15.2 ZeRO: sharding optimizer state, gradients, and parameters 67
15.3 Tensor and pipeline parallelism 68
15.4 Gradient checkpointing and mixed precision 68

16 Evaluation of Language Models **70**
16.1 Perplexity and bits-per-character 70
16.2 Standard benchmarks 70
16.3 HELM and holistic evaluation 71
16.4 Benchmark contamination and the evaluation stack 71

17 LLM Deployment and Serving **73**
17.1 The KV-cache management problem . . . 73
17.2 PagedAttention and vLLM . . . 73
17.3 Serving metrics . . . 74
17.4 Speculative decoding . . . 74
17.5 TGI and TensorRT-LLM . . . 74

Where to go next **76**

Practical Coding Projects Series **77**

Appendix: Answer Key for Check Your Understanding **78**

Glossary **82**

References **84**

About this course

This document develops an eleven-module comprehensive introduction to modern artificial intelligence for technically capable students. It assumes basic background summarised in the prerequisite matrix on the next page: comfortable **Python**, basic **NumPy & Pandas**, and **elementary calculus and linear algebra**. Everything else, from eigenvectors to multi-head attention, model quantization, and distributed training, is built from scratch in context with citations to the primary literature and standard textbooks.

The eleven modules span core foundations to advanced LLM systems engineering, as the roadmap below shows: the AI landscape and ethics (*Module 1*), mathematical foundations (*Module 2*), classical machine learning (*Module 3*), neural networks and representation learning (*Module 4*), foundation models and generative AI (*Module 5*), modern LLM architectures (*Module 6*), efficient attention mechanisms (*Module 7*), model quantization (*Module 8*), distributed training (*Module 9*), LLM evaluation (*Module 10*), and deployment and serving (*Module 11*). All code snippets assume Python 3.9+ with `numpy`, `pandas`, `scikit-learn`, and (for Modules 4–11) `torch`, `transformers`, and `vLLM` installed.

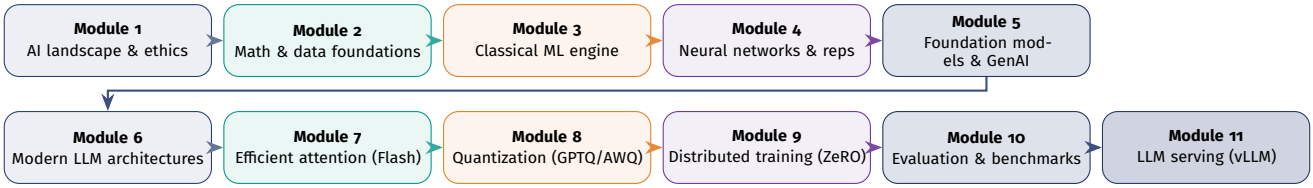


Figure 1. The learning path: spanning core mathematical/algorithmic foundations (Tier 1) and modern large-scale LLM engineering (Tier 2).

Throughout the guide you will meet the following recurring callout boxes:

Box	What it gives you
Module # at a glance	Focus, key concepts, and learning goals for each module.
Key concept	Definitions and intuitions worth memorising.
Hands-on with Python	Short, runnable code tied to the prerequisite skills.
Ethics & safety spotlight	Responsible-AI notes and open questions.
Check your understanding	A quick self-test before moving on.
Sources & further reading	Curated references backing each module.

Table 1. The callout boxes used throughout this guide.

Prerequisites and required background

The table below is the assumed starting point. None of the rows requires expert mastery; the goal of each is narrow and practical.

Python (intermediate). You should be comfortable writing functions, loops, list comprehensions, and simple classes, and know your way around a virtual environment. The course is deliberately code-first: every mathematical idea is paired with a few lines of Python. A good refresher, free to read online, is VanderPlas (2016).

NumPy & Pandas (basic). You should be able to create arrays, slice them, and compute with them (`A @ B` for matrix products), and load a CSV into a DataFrame to clean and summarise it (Harris et al. 2020; McKinney 2010). The practice boxes assume exactly this level and no more.

Skill	Required level	Primary use case
Python programming	Intermediate	Implementing algorithms, data manipulation
NumPy & Pandas	Basic	Matrix operations and data cleaning
Calculus & Linear Algebra	Basic	Derivatives, partials, vectors, and matrices

Table 2. Prerequisite matrix: the background assumed by this guide.

Table 3. Notation reference table.

Symbol / Notation	Description and Typical Context
$\mathbf{x}, \mathbf{y}, \mathbf{z}$	Bold lowercase letters denote column vectors (e.g., sample features $\mathbf{x} \in \mathbb{R}^n$).
X, W, A	Uppercase letters denote matrices (e.g., dataset matrix $X \in \mathbb{R}^{m \times n}$, weights W).
$\boldsymbol{\theta}, \mathbf{w}, \boldsymbol{\beta}$	Model parameter vectors (e.g., regression weights β, neural net parameters $\boldsymbol{\theta}$).
$\mathcal{L}(\hat{y}, y),\ J(\boldsymbol{\theta})$	Loss function for a single instance ($\mathcal{L}$) vs. cost function averaged over data (J).
$\nabla_{\boldsymbol{\theta}} J$	Gradient vector of partial derivatives with respect to parameters $\boldsymbol{\theta}$.
Σ vs. $\sum$	Uppercase sigma Σ denotes a Covariance Matrix; summation operator $\sum_{i=1}^{m}$ denotes summing a series.
$\odot$	Hadamard (element-wise) vector or matrix product.
$\phi(z), \phi'(z)$	Activation function and its scalar derivative.

Calculus (basic). You need the derivative as a slope, the power rule, and the chain rule. That is genuinely all : gradients, the engine of all learning in this course, are just derivatives collected into a vector.

The two calculus ideas you actually need

- **A derivative is a local slope:** $f'(x) \approx \dfrac{f(x+h) - f(x)}{h}$ it tells you which direction is *downhill* and how steeply.
- **The chain rule composes slopes:** $\dfrac{d}{dx} f(g(x)) = f'(g(x))\, g'(x)$.

Backpropagation (Module 4), which serves as the core gradient-computation algorithm for training modern neural networks, is nothing more than this second rule applied systematically, layer by layer.

Self-check before starting.

- Can you write a Python function that returns the mean and standard deviation of a list of numbers?
- Given two NumPy arrays, can you compute their dot product and the shape of `A @ B`?
- Can you differentiate $f(w) = w^2$ and say what $f'(3)$ means geometrically?

If any answer is “not yet”, spend a week with VanderPlas (2016) and a standard calculus refresher before returning, as everything else is built for you from here.

1 Introduction and the AI Landscape

Module 1, at a glance (Estimated Study: 4 hours • Level: Beginner)

Focus. What artificial intelligence is (and is not), how the field evolved from hand-written rules to learning from data, and the infrastructure and ethical questions that shape modern practice.

Key concepts. Defining intelligence; weak vs. strong AI; expert systems vs. data-driven AI; GPUs, TPUs, cloud ecosystems and data pipelines; bias, alignment, copyright and environmental impact.

After this module you will be able to explain the difference between narrow and general AI, describe the paradigm shift from rules to data, sketch an end-to-end machine-learning pipeline, and name the main ethical questions attached to deploying an AI system.

1.1 Defining intelligence: weak vs. strong AI

Before we can build artificial intelligence we have to say what we mean by *intelligence*. Russell and Norvig (2021) organise the many definitions along two axes: are we concerned with *thought processes* or with *behaviour*, and should the benchmark be *human-like* performance or *rationality*, acting so as to achieve the best expected outcome? Alan Turing sidestepped the philosophy in 1950 with an operational test: if a machine's written answers are indistinguishable from a human's, is it not, for practical purposes, intelligent? (Turing 1950). Modern AI research mostly aims at rational behaviour on well-defined tasks, exactly what the learning algorithms of Modules 2 to 5 deliver.

Weak (narrow) AI vs. strong AI

Weak / narrow AI (today)	Strong AI / AGI (hypothetical)
Excels at one well-defined task: spam filters, image classifiers, next-token prediction	Reasons, plans and learns across arbitrary domains, as a human can
Learns from task-specific data; capability does not transfer	Would transfer knowledge flexibly to novel problems
Every deployed system, including today's LLMs, is narrow in this sense	Unknown whether or when it is achievable; an active research and policy debate

A second classical debate is worth knowing. Philosopher John Searle's *Chinese room* argument holds that symbol manipulation alone is not understanding: a person following rules could answer questions in Chinese without knowing any Chinese (Searle 1980). The argument has returned with force in the era of large language models, because fluent output is cheap while grounded understanding is hard to verify, a theme we revisit in the ethics spotlight below.

1.2 From expert systems to data-driven AI

The term *artificial intelligence* was coined by John McCarthy in the 1955 proposal for the 1956 Dartmouth workshop. For roughly the next three decades the dominant paradigm was *symbolic*: human knowledge was hand-encoded as if–then rules. Expert systems such as MYCIN encoded

several hundred rules elicited from physicians and impressed clinicians in blind evaluations, yet only inside their narrow domain. Rules are brittle, and extracting them from experts (the *knowledge-acquisition bottleneck*) is slow and expensive. Deep Blue's 1997 victory over Garry Kasparov represented the high-water mark of this symbolic, search-based paradigm, demonstrating immense raw evaluation power, yet remaining strictly bound by human-engineered evaluation heuristics.

Machine learning inverts the pipeline: instead of programming rules, we collect examples of the desired behaviour and let an algorithm discover the rules. In Mitchell (1997)'s classic formulation, a program *learns* from experience E with respect to task T and performance measure P if its performance at T, measured by P, improves with E. Deep learning pushes the inversion one step further by learning not only the rules but the *features* they operate on (LeCun et al. 2015). Three results announced the new era: AlexNet cut the ImageNet top-5 error rate from roughly 26% to 16% by training a deep network on two consumer GPUs (Krizhevsky et al. 2012); AlphaGo defeated the world Go champion using deep networks combined with search (Silver et al. 2016); and transformer language models (Module 4) turned plain text prediction into a general-purpose technology.

The paradigm shift: from rules to data

Classical programming: rules + data $\longrightarrow$ answers
Machine learning: data + answers $\longrightarrow$ rules (the model)

Instead of hand-crafting the logic, we specify an objective and let an algorithm search for parameters that meet it, which is precisely why the mathematics of Modules 2 and 3 matters so much.

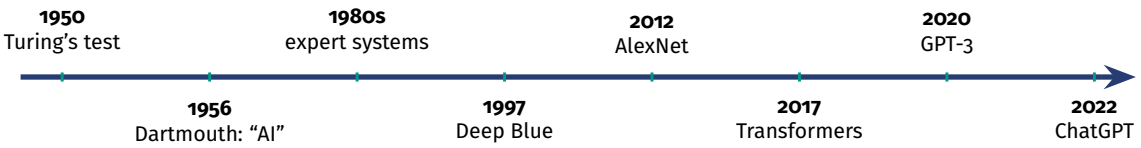


Figure 2. A selective history: from the founding of the field to the generative-AI era.

1.3 The modern stack: compute, cloud, and data pipelines

Compute: GPUs and TPUs. Training a modern model is, computationally, one thing: enormous numbers of matrix–vector products. A CPU optimises a handful of fast, sequential cores; a GPU packs thousands of simpler cores that perform these products in parallel. That is why Krizhevsky et al. (2012) could train a vision network that CPU-era attempts could not, and why hardware co-design now shapes chip roadmaps. Google's Tensor Processing Units, for instance, execute the tensor algebra of neural networks directly (Jouppi et al. 2017). Memory bandwidth is the other half of the story: modern accelerators are as much about feeding data as about computing with it.

Cloud ecosystems. The same hardware is available to anyone, rented by the hour: a laptop, a browser, and a credit card are enough to requisition a cluster of GPUs with a reproducible software image. This democratised experimentation, and concentrated an enormous amount of AI capability in a handful of providers.

Data engineering, dataset construction, and contamination. Models live inside a loop, not a vacuum (Figure 3). Practitioners spend most project time collecting, cleaning, deduplicating, and validating data (McKinney 2010). Modern AI development requires rigorous data hygiene:

- **Dataset Construction & Labeling Noise:** Human annotation introduces errors, ambiguities, and labeler bias. Clean labels are often more valuable than model architecture tweaks.
- **Train/Test Leakage & Contamination:** Pretraining datasets scraped at web scale frequently suffer from *benchmark contamination*: test set questions leaking into training corpora, inflating evaluation scores artificially. Temporal leakage (using future data to predict past events) and group leakage (splitting samples from the same patient/user across train and test sets) yield invalid accuracy claims.
- **Data Augmentation & Synthetic Data:** Generating synthetic training examples or expanding datasets via domain-specific transforms mitigates data scarcity but risks amplifying generator biases.

Hands-on with Python

```python
import pandas as pd
import numpy as np

# Ingest / synthetic creation for standalone demo
rng = np.random.default_rng(42)
df = pd.DataFrame({
    "created": pd.date_range("2026-01-01", periods=100, freq="h"),
    "amount": rng.uniform(10, 200, 100),
    "quantity": rng.integers(1, 5, 100),
    "region": rng.choice(["North", "South", "East", "West"], 100)
})
df = df.dropna(subset=["amount"])           # basic cleaning
df["revenue"] = df["amount"] * df["quantity"]
summary = df.groupby("region")["revenue"].agg(["count", "mean", "sum"])
print(summary)
```

Pandas covers the ingest, clean, transform, aggregate loop that occupies most of a data scientist's day.

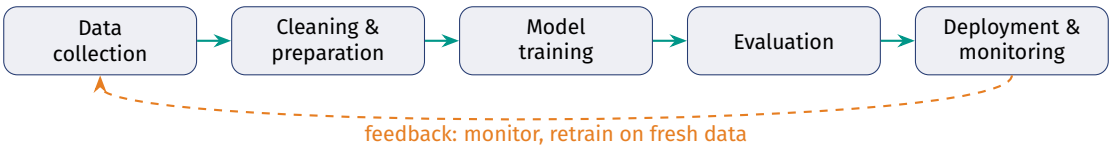


Figure 3. A modern data pipeline: the machine-learning lifecycle as a loop, not a line.

1.4 Ethics, safety, and security: the non-optional module

Technical capability has outpaced governance. Six essential dimensions should accompany every system you build:

Core dimensions of responsible AI and safety

Bias & Fairness. Models inherit patterns from training data. Commercial classifiers have shown severe demographic accuracy gaps (Buolamwini and Gebru 2018). Crucially, *simply stripping demographic features (e.g., race, gender) does NOT guarantee fairness*, because correlated proxy variables retain discriminatory signal. Furthermore, mathematical fairness criteria (such as Equalized Odds vs. Predictive Parity) frequently conflict under unequal base rates: **there is no single universally correct fairness metric**.

Privacy & Security. Training on sensitive records risks membership inference and model inversion attacks. Safeguards include Differential Privacy (ϵ, δ-guarantees) and federated aggregation. Additionally, deployment surfaces new threat vectors: *prompt injection*, data poisoning, and model extraction.

Alignment & Harm Mitigation. Objective misalignment causes reward hacking (Amodei et al. 2016; Christian 2020). Red teaming, safety guardrails, and preference optimization (Module 4) help steer foundation models toward helpful and safe execution.

Copyright & Provenance. Data scraping raises unresolved legal questions surrounding fair use, licensing, watermarking, and artist attribution.

Environment. Large-scale model training consumes significant energy and water (Strubell et al. 2019; Patterson et al. 2021). Mitigations include right-sizing architectures and using energy-efficient accelerators.

Automation Bias & Grounding. Bender et al. (2021) warned that LLMs are *stochastic parrots* reproducing surface-level statistics without grounded comprehension. Over-reliance (automation bias) leads users to accept unverified model claims.

A practitioner's checklist.

- Define the task, population served, and failure modes *before* modeling.
- Audit training data for representation gaps, proxy correlations, and leakage.
- Evaluate metrics across demographic sub-groups; balance cost trade-offs.
- Document limitations via Model Cards, Datasheets, and clear system prompts.
- Establish red teaming, active monitoring, and human-in-the-loop oversight.

Check your understanding

1. A chess engine, a spam filter, and a claimed "fully autonomous reasoning" startup demo: classify each as weak or strong AI, and justify the label.
2. Why did GPUs unlock deep learning at scale, whereas CPUs could not supply the necessary arithmetic throughput? Answer in terms of parallel hardware architecture.
3. Your model screens job applications. Name two concrete harms and one mitigation for each.

Sources & further reading

- S. J. Russell and P. Norvig (2021). *Artificial Intelligence: A Modern Approach*. 4th ed. Hoboken, NJ: Pearson
- A. M. Turing (1950). "Computing Machinery and Intelligence". In: *Mind* 59.236, pp. 433–460
- J. R. Searle (1980). "Minds, Brains, and Programs". In: *Behavioral and Brain Sciences* 3.3, pp. 417–424
- Y. LeCun et al. (2015). "Deep Learning". In: *Nature* 521.7553, pp. 436–444
- A. Krizhevsky et al. (2012). "ImageNet Classification with Deep Convolutional Neural Networks". In: *Advances in Neural Information Processing Systems 25*, pp. 1097–1105
- D. Silver et al. (2016). "Mastering the Game of Go with Deep Neural Networks and Tree Search". In: *Nature* 529.7587, pp. 484–489
- N. P. Jouppi et al. (2017). "In-Datacenter Performance Analysis of a Tensor Processing Unit". In: *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA)*, pp. 1–12
- J. Buolamwini and T. Gebru (2018). "Gender Shades: Intersectional Accuracy Disparities in Commercial Gender Classification". In: *Proceedings of the 1st Conference on Fairness, Accountability*

and Transparency, pp. 77–91
- D. Amodei et al. (2016). "Concrete Problems in AI Safety". In: *arXiv preprint arXiv:1606.06565*
- B. Christian (2020). *The Alignment Problem: Machine Learning and Human Values*. New York: W. W. Norton & Company
- E. Strubell et al. (2019). "Energy and Policy Considerations for Deep Learning in NLP". in: *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 3645–3650
- D. Patterson et al. (2021). "Carbon Emissions and Large Neural Network Training". In: *Advances in Neural Information Processing Systems 34*, pp. 13470–13481
- E. M. Bender et al. (2021). "On the Dangers of Stochastic Parrots: Can Language Models Be Too Big?" In: *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency (FAccT)*, pp. 610–623

2 Mathematics and Data Foundations

Module 2, at a glance (Estimated Study: 8 hours • Level: Intermediate)

Focus. The small mathematical toolkit that quietly powers everything else in the course: linear algebra to *represent* data, probability to *quantify* uncertainty, and optimisation to *learn* from mistakes.

Key concepts. Vectors, matrices, eigenvalues and dimensionality reduction; Bayes' theorem, distributions and hypothesis testing; loss and cost functions, gradients, and gradient descent.

After this module you will be able to describe any dataset as linear-algebra objects, reason correctly about conditional probabilities, explain and implement gradient descent from scratch.

Mathematics here is a working tool, not an exam subject. Each concept is introduced the moment it earns its keep, and each is paired with NumPy code that uses it.

2.1 Linear algebra: the language of data

Machine-learning data lives in vectors, matrices, and tensors (Strang 2012; Strang 2019; Deisenroth et al. 2020). A sample instance becomes a feature vector $\mathbf{x} = [x_1, x_2, \ldots, x_n]^\top \in \mathbb{R}^n$; a dataset of m instances becomes a design matrix $X \in \mathbb{R}^{m \times n}$; a multi-channel image or sequence batch becomes a tensor $\mathcal{T} \in \mathbb{R}^{B \times C \times H \times W}$. Linear algebra provides both the representation language for high-dimensional data and the geometric framework for modern AI architectures (Goodfellow et al. 2016).

Vectors, inner products, and matrix multiplication perspectives. The inner product (dot product) of two vectors $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$, defined as $\mathbf{u}^\top \mathbf{v} = \sum_{i=1}^{n} u_i v_i = \|\mathbf{u}\|_2 \|\mathbf{v}\|_2 \cos\theta$, measures directional alignment. Cosine similarity $\frac{\mathbf{u}^\top \mathbf{v}}{\|\mathbf{u}\|_2 \|\mathbf{v}\|_2}$ directly underpins modern semantic retrieval, embedding search, and vector databases (Strang 2019).

Matrix multiplication $C = AB$, where $A \in \mathbb{R}^{m \times k}$ and $B \in \mathbb{R}^{k \times n}$, can be understood through five distinct mathematical perspectives highlighted by Strang (2012, Chapter 1):

1. **Entrywise (dot product):** $c_{ij} = \mathbf{a}_{i,:}^\top \mathbf{b}_{:,j} = \sum_{p=1}^{k} a_{ip} b_{pj}$.
2. **Column perspective:** Column j of C is a linear combination of columns of A: $\mathbf{c}_{:,j} = A\mathbf{b}_{:,j} = \sum_{p=1}^{k} b_{pj} \mathbf{a}_{:,p}$.
3. **Row perspective:** Row i of C is a linear combination of rows of B: $\mathbf{c}_{i,:} = \mathbf{a}_{i,:}^\top B = \sum_{p=1}^{k} a_{ip} \mathbf{b}_{p,:}$.
4. **Outer product expansion (column-by-row):** C is the sum of k rank-1 matrices:

$$C = \sum_{p=1}^{k} \mathbf{a}_{:,p} \mathbf{b}_{p,:}^\top. \tag{1}$$

5. **Block matrix multiplication:** Partitioning A and B into sub-blocks for distributed/parallel GEMM execution (Golub and Van Loan 2013).

The outer-product expansion (1) is foundational to low-rank weight updates (such as LoRA (Hu et al. 2022)) and low-rank matrix approximations (SVD compression).

Matrix operations: rank, determinant, inverse, condition number, SVD outer product

```python
import numpy as np

# --- Basic matrix construction and properties ---
A = np.array([[4, 3], [6, 3]], dtype=float)
print("A =\n", A)
print("Rank:", np.linalg.matrix_rank(A))          # 2  (full rank)
print("Det:", np.linalg.det(A))                   # -6
print("Trace:", np.trace(A))                      # 7  (sum of diagonals)

# --- Transpose, inverse, and matrix product ---
B = np.array([[1, 2], [3, 4]], dtype=float)
print("A @ B =\n", A @ B)                         # standard product
print("A.T =\n", A.T)                             # transpose
print("A^{-1} =\n", np.linalg.inv(A))            # inverse

# --- Solving Ax = b ---
b = np.array([10, 12], dtype=float)
x = np.linalg.solve(A, b)
print("Solution x:", x)                           # [1. 2.]

# --- Singular Value Decomposition (SVD) and Outer-Product Sum ---
U, s, Vt = np.linalg.svd(A, full_matrices=False)
print("Singular values:", s)                      # [8.60, 0.70]
# Reconstruct via outer-product expansion: A = sum_i s_i u_i v_i^T
A_reconstructed = sum(s[i] * np.outer(U[:, i], Vt[i, :]) for i in range(len(s)))
print("Reconstruction error:", np.max(np.abs(A - A_reconstructed)))

# --- Condition Number and Pseudoinverse ---
cond_num = np.linalg.cond(A)
print(f"Condition number kappa(A): {cond_num:.2f}")

A_pinv = np.linalg.pinv(A)
print("Moore-Penrose Pseudoinverse A^+ =\n", A_pinv)
print("Verify A @ A^+ @ A = A:", np.allclose(A @ A_pinv @ A, A))
```

Vector spaces and Strang's Four Fundamental Subspaces. A matrix $A \in \mathbb{R}^{m \times n}$ defines a linear transformation $T : \mathbb{R}^n \to \mathbb{R}^m$ via $T(\mathbf{x}) = A\mathbf{x}$. Strang (2012, Chapter 2) and Strang (2019, Chapter 1) establish that every matrix connects four fundamental orthogonal vector spaces:

- **Column Space** $\mathcal{C}(A) \subseteq \mathbb{R}^m$: The span of the columns of A, representing all attainable outputs $\mathbf{b} = A\mathbf{x}$. $\dim(\mathcal{C}(A)) = r$ (the matrix rank).
- **Nullspace** $\mathcal{N}(A) \subseteq \mathbb{R}^n$: The subspace of all inputs mapping to zero, $\{\mathbf{x} \in \mathbb{R}^n : A\mathbf{x} = \mathbf{0}\}$. $\dim(\mathcal{N}(A)) = n - r$.
- **Row Space** $\mathcal{C}(A^\top) \subseteq \mathbb{R}^n$: The span of the rows of A (or column space of $A^\top$). $\dim(\mathcal{C}(A^\top)) = r$.
- **Left Nullspace** $\mathcal{N}(A^\top) \subseteq \mathbb{R}^m$: The subspace of vectors mapping to zero under $A^\top$, $\{\mathbf{y} \in \mathbb{R}^m : A^\top\mathbf{y} = \mathbf{0}\}$. $\dim(\mathcal{N}(A^\top)) = m - r$.

The Fundamental Theorem of Linear Algebra (Strang 2012)

The four subspaces are pairwise orthogonal complements:

$$\mathcal{C}(A^\top) \perp \mathcal{N}(A) \quad \text{in } \mathbb{R}^n, \qquad \mathcal{C}(A) \perp \mathcal{N}(A^\top) \quad \text{in } \mathbb{R}^m. \tag{2}$$

Furthermore, the **Rank-Nullity Theorem** dictates:

$$\operatorname{rank}(A) + \operatorname{nullity}(A) = r + (n - r) = n. \tag{3}$$

In overparameterized deep networks ($n \gg m$), the nullspace $\mathcal{N}(W)$ has high dimension ($n - m$), creating large invariant directions where parameter shifts produce zero change in output.

Strang's Four Fundamental Subspaces and Orthogonality in Python

```python
import numpy as np
from scipy.linalg import null_space

# --- Define a rectangular matrix A (3x4) of rank 2 ---
A = np.array([
    [1, 2, 3, 4],
    [2, 4, 7, 10],
    [3, 6, 10, 14]
], dtype=float)

m, n = A.shape
r = np.linalg.matrix_rank(A)
print(f"Matrix A ({m}x{n}), Rank r = {r}")

# 1. Column Space C(A) in R^m and Left Nullspace N(A^T) in R^m
U, s, Vt = np.linalg.svd(A, full_matrices=True)
col_space = U[:, :r]             # Basis for C(A)
left_null_space = U[:, r:]       # Basis for N(A^T)

# 2. Row Space C(A^T) in R^n and Nullspace N(A) in R^n
row_space = Vt[:r, :].T          # Basis for C(A^T)
null_sp = Vt[r:, :].T            # Basis for N(A)

print(f"1. Column space C(A) dim = {col_space.shape[1]}")
print(f"2. Nullspace N(A) dim = {null_sp.shape[1]}")
print(f"3. Row space C(A^T) dim = {row_space.shape[1]}")
print(f"4. Left Nullspace N(A^T) dim = {left_null_space.shape[1]}")

# --- Verify Rank-Nullity Theorem: dim(C(A^T)) + dim(N(A)) = n ---
assert col_space.shape[1] + null_sp.shape[1] == n
print(f"Rank-Nullity Theorem verified: r ({r}) + (n-r) ({n - r}) = n ({n})")

# --- Verify Orthogonality: C(A^T) perp N(A) in R^n ---
ortho_check_1 = np.max(np.abs(row_space.T @ null_sp))
print(f"Orthogonality check C(A^T) perp N(A) max inner product: {ortho_check_1:.2e}")
assert ortho_check_1 < 1e-12

# --- Verify Orthogonality: C(A) perp N(A^T) in R^m ---
ortho_check_2 = np.max(np.abs(col_space.T @ left_null_space))
print(f"Orthogonality check C(A) perp N(A^T) max inner product: {ortho_check_2:.2e}")
assert ortho_check_2 < 1e-12

print("Strang's Four Fundamental Subspaces successfully verified!")
```

Linear systems, projections, and least squares. A linear system $A\mathbf{x} = \mathbf{b}$ has an exact solution if and only if $\mathbf{b} \in \mathcal{C}(A)$. When $m > n$ (overdetermined system), data is noisy and no exact solution exists. The **Ordinary Least Squares (OLS)** objective minimizes the residual Euclidean norm:

$$\hat{\mathbf{x}} = \arg\min_{\mathbf{x} \in \mathbb{R}^n} \|\mathbf{b} - A\mathbf{x}\|_2^2. \tag{4}$$

Geometrically, the error vector $\mathbf{e} = \mathbf{b} - A\hat{\mathbf{x}}$ must be orthogonal to every column of A, meaning $\mathbf{e} \in \mathcal{N}(A^\top) = \mathcal{C}(A)^\perp$ (Strang 2012, Chapter 4). Thus $A^\top(\mathbf{b} - A\hat{\mathbf{x}}) = \mathbf{0}$, yielding the famous **Normal**

Equations:

$$A^\top A\hat{\mathbf{x}} = A^\top \mathbf{b} \quad \Longrightarrow \quad \hat{\mathbf{x}} = (A^\top A)^{-1}A^\top \mathbf{b}. \tag{5}$$

The **Projection Matrix** $P = A(A^\top A)^{-1}A^\top \in \mathbb{R}^{m\times m}$ projects any vector $\mathbf{b} \in \mathbb{R}^m$ orthogonally onto $\mathcal{C}(A)$, satisfying $P^2 = P$ (idempotent) and $P^\top = P$ (symmetric). When A is singular or non-square, the general solution uses the **Moore-Penrose Pseudoinverse** $A^+ = V\Sigma^+U^\top$ (Golub and Van Loan 2013; Strang 2019).

Vector and matrix norms, condition numbers, and optimization stability. Norms quantify vector magnitude and matrix scaling operator strength:

- **Vector ℓ_p norms:** $\|\mathbf{x}\|_p = (\sum_{i=1}^n |x_i|^p)^{1/p}$. ℓ_1 encourages sparsity (Lasso), ℓ_2 measures Euclidean length (Ridge), $\ell_\infty = \max_i |x_i|$.
- **Frobenius matrix norm:** $\|A\|_F = \sqrt{\sum_{i,j} a_{ij}^2} = \sqrt{\mathrm{tr}(A^\top A)} = \sqrt{\sum_{i=1}^r \sigma_i^2}$.
- **Spectral matrix norm:** $\|A\|_2 = \sigma_{\max}(A) = \max_{\|\mathbf{x}\|_2=1} \|A\mathbf{x}\|_2$.

The **condition number** of a matrix A under the spectral norm is defined as:

$$\kappa(A) = \|A\|_2\|A^{-1}\|_2 = \frac{\sigma_{\max}(A)}{\sigma_{\min}(A)}. \tag{6}$$

When $\kappa(A) \gg 1$, the matrix is *ill-conditioned*. In optimization, ill-conditioned Hessian matrices create elongated "ravines" in the loss surface, forcing standard gradient descent to oscillate inefficiently. This directly motivates momentum, preconditioning, and adaptive optimizers such as Adam (Boyd and Vandenberghe 2004; Strang 2019; Trefethen and Bau 1997).

Quadratic forms, positive definiteness, and loss landscapes. For a symmetric matrix $A = A^\top \in \mathbb{R}^{n\times n}$, the function $f(\mathbf{x}) = \mathbf{x}^\top A\mathbf{x}$ is a **quadratic form**. A is **positive definite** ($A \succ 0$) if $\mathbf{x}^\top A\mathbf{x} > 0$ for all non-zero $\mathbf{x} \in \mathbb{R}^n$, and **positive semi-definite** ($A \succeq 0$) if $\mathbf{x}^\top A\mathbf{x} \geq 0$.

Equivalently, $A \succ 0$ if and only if all eigenvalues $\lambda_i > 0$. Covariance matrices $\hat{\Sigma} = \frac{1}{m-1}X_c^\top X_c$ and Gram matrices $K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$ are guaranteed to be positive semi-definite (Bishop 2006). In deep learning optimization, the **Hessian matrix** $H = \nabla^2 J(\boldsymbol{\theta})$ governs local curvature:

- $H \succ 0$ (all $\lambda_i > 0$): Strict local minimum.
- $H \prec 0$ (all $\lambda_i < 0$): Strict local maximum.
- Indefinite H (mixed positive and negative λ_i): **Saddle point**. High-dimensional neural network loss landscapes are dominated by saddle points (Goodfellow et al. 2016).

Eigenvalues, eigenvectors, and the Rayleigh quotient. For a square matrix $A \in \mathbb{R}^{n\times n}$, a non-zero vector $\mathbf{v}$ is an *eigenvector* with eigenvalue λ if $A\mathbf{v} = \lambda\mathbf{v}$. Geometrically, A acts on $\mathbf{v}$ purely by scaling (Strang 2012).

Spectral Theorem for Symmetric Matrices (Strang 2012, Chapter 6)

If $A = A^\top \in \mathbb{R}^{n\times n}$, then all n eigenvalues λ_i are real, and there exists an orthonormal matrix of eigenvectors $V = [\mathbf{v}_1 \mid \cdots \mid \mathbf{v}_n]$ such that:

$$A = V\Lambda V^\top = \sum_{i=1}^n \lambda_i \mathbf{v}_i\mathbf{v}_i^\top, \qquad V^\top V = I. \tag{7}$$

For a symmetric matrix A, the **Rayleigh quotient** $R(\mathbf{x}) = \frac{\mathbf{x}^\top A\mathbf{x}}{\mathbf{x}^\top\mathbf{x}}$ measures variance along direction $\mathbf{x}$. By the Courant–Fischer minimax theorem (Golub and Van Loan 2013; Strang 2019):

$$\max_{\|\mathbf{x}\|_2=1} \mathbf{x}^\top A\mathbf{x} = \lambda_{\max}(A), \quad \text{achieved at } \mathbf{x} = \mathbf{v}_1; \qquad \min_{\|\mathbf{x}\|_2=1} \mathbf{x}^\top A\mathbf{x} = \lambda_{\min}(A), \quad \text{achieved at } \mathbf{x} = \mathbf{v}_n. \tag{8}$$

Eigendecomposition: verifying $Av = \lambda v$ and Rayleigh quotient

```python
import numpy as np

# --- Eigen-decomposition of a symmetric matrix ---
A = np.array([[3, 1], [1, 3]], dtype=float)
eigenvalues, eigenvectors = np.linalg.eigh(A)   # eigh: symmetric solver
# eigenvalues are returned in ASCENDING order by eigh
print("Eigenvalues:", eigenvalues)               # [2., 4.]
print("Eigenvectors (columns):\n", eigenvectors) # orthonormal columns

# --- Verify the defining equation A v = lambda v ---
for i, lam in enumerate(eigenvalues):
    v = eigenvectors[:, i]
    lhs = A @ v
    rhs = lam * v
    print(f"lambda={lam:.1f}: A@v={lhs}, lambda*v={rhs}",
          "  OK" if np.allclose(lhs, rhs) else "  MISMATCH")

# --- Spectral decomposition: A = V diag(lambda) V^T ---
V = eigenvectors
A_reconstructed = V @ np.diag(eigenvalues) @ V.T
print("Reconstruction error:", np.max(np.abs(A - A_reconstructed)))

# --- Rayleigh quotient: measures variance along direction v ---
v_test = np.array([1, 0], dtype=float)          # x-axis direction
rayleigh = (v_test @ A @ v_test) / (v_test @ v_test)
print("Rayleigh quotient along x-axis:", rayleigh)  # = 3
# Maximum Rayleigh quotient equals the largest eigenvalue (4)
```

Singular Value Decomposition (SVD) and low-rank adaptation. Every real matrix $A \in \mathbb{R}^{m\times n}$ admits the factorisation:

$$A = U\Sigma V^\top = \sum_{i=1}^{r} \sigma_i \mathbf{u}_i \mathbf{v}_i^\top, \tag{9}$$

where $U \in \mathbb{R}^{m\times m}$ and $V \in \mathbb{R}^{n\times n}$ are orthogonal matrices, and $\Sigma \in \mathbb{R}^{m\times n}$ is diagonal with non-negative singular values $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0$ (Trefethen and Bau 1997; Strang 2019). The singular values are square roots of eigenvalues of $A^\top A$ (and $AA^\top$).

Eckart–Young–Mirsky Theorem (Golub and Van Loan 2013; Strang 2012)

The optimal rank-k matrix approximation $A_k = \sum_{i=1}^{k} \sigma_i \mathbf{u}_i \mathbf{v}_i^\top$ (where $k < r$) minimizes the reconstruction error under both Frobenius and Spectral norms:

$$\min_{\text{rank}(B)\leq k} \|A-B\|_F = \|A-A_k\|_F = \sqrt{\sum_{i=k+1}^{r} \sigma_i^2}, \qquad \min_{\text{rank}(B)\leq k} \|A-B\|_2 = \|A-A_k\|_2 = \sigma_{k+1}. \tag{10}$$

In modern LLM fine-tuning, **Low-Rank Adaptation (LoRA)** parameterizes weight updates as low-rank decompositions $\Delta W = BA$ (where $B \in \mathbb{R}^{d\times r}, A \in \mathbb{R}^{r\times k}$ with $r \ll \min(d,k)$), freezing base weights W_0 while training only $r(d+k)$ parameters (Hu et al. 2022).

Matrix factorizations: Strang's "Great Factorizations". Strang (2019, Chapter 7) classifies five fundamental matrix factorizations that serve as the computational backbone of scientific computing and machine learning:

Summary of the 5 Great Matrix Factorizations

Factorization	Formula	Matrix Requirements	Primary ML Role
LU (Elimination)	$A = LU$ (or $PA = LU$)	Square, non-singular	Solving linear systems $A\mathbf{x} = \mathbf{b}$ efficiently.
QR (Orthogonal)	$A = QR$	Rectangular $m \geq n$	Stable OLS, Gram-Schmidt, Householder solver.
Cholesky	$A = LL^\top$	Symmetric $A \succ 0$	Gaussian processes, MVN sampling (Golub and Van Loan 2013).
Spectral	$A = V\Lambda V^\top$	Symmetric $A = A^\top$	PCA, Graph Laplacians, Rayleigh optimization.
SVD	$A = U\Sigma V^\top$	Any rectangular matrix	Low-rank compression, LoRA, pseudoinverse.

Executing the 5 Matrix Factorizations in Python

```python
import numpy as np
from scipy.linalg import lu, qr, cholesky

# --- 1. LU Decomposition (A = P L U) ---
A_square = np.array([[2, 1, 1], [4, -6, 0], [-2, 7, 2]], dtype=float)
P, L, U = lu(A_square)
print("1. LU Factorization:")
print("Reconstruction error:", np.max(np.abs(P @ L @ U - A_square)))
assert np.allclose(P @ L @ U, A_square)

# --- 2. QR Decomposition (A = Q R) ---
A_rect = np.array([[1, 2], [3, 4], [5, 6]], dtype=float)
Q, R = qr(A_rect, mode='economic')
print("\n2. QR Factorization:")
print("Q shape:", Q.shape, "R shape:", R.shape)
print("Orthogonality Q^T Q:\n", np.round(Q.T @ Q, 6))
print("Reconstruction error:", np.max(np.abs(Q @ R - A_rect)))
assert np.allclose(Q @ R, A_rect)

# --- 3. Cholesky Decomposition (A = L L^T) for Positive Definite Matrix ---
A_spd = np.array([[4, 12, -16], [12, 37, -43], [-16, -43, 98]], dtype=float)
L_chol = cholesky(A_spd, lower=True)
print("\n3. Cholesky Factorization:")
print("Reconstruction error:", np.max(np.abs(L_chol @ L_chol.T - A_spd)))
assert np.allclose(L_chol @ L_chol.T, A_spd)

# --- 4. Eigendecomposition (A = V Lambda V^T) for Symmetric Matrix ---
eigenvalues, V = np.linalg.eigh(A_spd)
A_eig_rec = V @ np.diag(eigenvalues) @ V.T
print("\n4. Eigendecomposition:")
print("Reconstruction error:", np.max(np.abs(A_eig_rec - A_spd)))
assert np.allclose(A_eig_rec, A_spd)

# --- 5. Singular Value Decomposition (A = U Sigma V^T) ---
U_svd, s, Vt_svd = np.linalg.svd(A_rect, full_matrices=False)
A_svd_rec = U_svd @ np.diag(s) @ Vt_svd
print("\n5. Singular Value Decomposition (SVD):")
print("Singular values:", s)
print("Reconstruction error:", np.max(np.abs(A_svd_rec - A_rect)))
assert np.allclose(A_svd_rec, A_rect)
```

Principal Component Analysis (PCA) revisited. PCA, introduced by Pearson (1901), finds orthogonal directions that maximize variance. Given centered dataset $X_c \in \mathbb{R}^{m\times d}$ ($X_c = X - \mathbf{1}\bar{\mathbf{x}}^\top$) with sample covariance matrix $\hat{\Sigma} = \frac{1}{m-1}X_c^\top X_c$:

$$\mathbf{v}_1 = \arg\max_{\|\mathbf{v}\|_2=1} \mathbf{v}^\top \hat{\Sigma}\mathbf{v}. \tag{11}$$

By the Rayleigh quotient theorem (8), $\mathbf{v}_1$ is the principal eigenvector of $\hat{\Sigma}$ corresponding to $\lambda_{\max}$ (Strang 2012; Bishop 2006). The complete PCA algorithm follows five steps:

1. **Center data:** $X_c = X - \mathbf{1}\bar{\mathbf{x}}^\top \in \mathbb{R}^{m\times d}$.
2. **Compute covariance:** $\hat{\Sigma} = \frac{1}{m-1}X_c^\top X_c \in \mathbb{R}^{d\times d}$.
3. **Eigen-decompose:** $\hat{\Sigma} = V\Lambda V^\top$; sort eigenvalues $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_d \geq 0$.
4. **Project onto k PCs:** $Z = X_c W_k \in \mathbb{R}^{m\times k}$, where $W_k = [\mathbf{v}_1, \ldots, \mathbf{v}_k] \in \mathbb{R}^{d\times k}$.
5. **Reconstruct:** $\hat{X}_c = ZW_k^\top$; reconstruction MSE equals the sum of discarded eigenvalues $\sum_{i=k+1}^{d} \lambda_i$.

PCA via SVD. Applying SVD directly to centered data $X_c = U\Sigma_s V^\top$ yields right singular vectors V identical to covariance eigenvectors, with $\sigma_i^2 = (m-1)\lambda_i$ (Trefethen and Bau 1997; Strang 2019). For high-dimensional regimes ($d \gg m$), SVD on X_c computes PCs in $O(m^2 d)$ operations, avoiding the formation of the huge $d \times d$ covariance matrix.

PCA from scratch: centring, covariance, eigen-decomposition, scree table

```python
import numpy as np

# ---------------------------------------------------------------------
# Worked example: PCA on a 2-D dataset with a known correlation structure
# ---------------------------------------------------------------------
rng = np.random.default_rng(42)

# Generate 200 points: strong positive correlation between x1 and x2
mean = np.array([2.0, 3.0])
cov  = np.array([[4.0, 3.2],   # Var(x1)=4, Cov(x1,x2)=3.2
                 [3.2, 4.0]])  # Cov(x1,x2)=3.2, Var(x2)=4
X = rng.multivariate_normal(mean, cov, size=200)

# Step 1 - Centre the data (subtract column means)
X_c = X - X.mean(axis=0)
print("Centred mean (should be ~0):", X_c.mean(axis=0).round(6))

# Step 2 - Compute the (unbiased) sample covariance matrix
n, d = X_c.shape                          # 200 samples, 2 features
Sigma = (X_c.T @ X_c) / (n - 1)          # d x d symmetric matrix
print("Sample covariance:\n", Sigma.round(3))

# Step 3 - Eigen-decompose Sigma (eigh returns ascending order)
eigenvalues, eigenvectors = np.linalg.eigh(Sigma)
# Sort descending: largest variance first
idx = np.argsort(eigenvalues)[::-1]
eigenvalues  = eigenvalues[idx]
eigenvectors = eigenvectors[:, idx]

print("Eigenvalues:", eigenvalues.round(3))
print("Explained variance ratio:", (eigenvalues / eigenvalues.sum()).round(3))
print("PC1 direction:", eigenvectors[:, 0].round(3))
print("PC2 direction:", eigenvectors[:, 1].round(3))

# Step 4 - Project onto top-k principal components
k = 1                                     # retain 1 component
W = eigenvectors[:, :k]                   # projection matrix (d x k)
X_proj = X_c @ W                          # (n x k) scores
print(f"\nProjected data shape (n x k): {X_proj.shape}")

# Step 5 - Reconstruct and measure compression error

X_recon = X_proj @ W.T                    # back to d dimensions
recon_error = np.mean((X_c - X_recon)**2)
print(f"Mean squared reconstruction error (k={k}): {recon_error:.4f}")
print(f"  (equals discarded eigenvalue: {eigenvalues[k]:.4f})")

# ---------------------------------------------------------------------
# Scree plot values (for manual inspection)
# ---------------------------------------------------------------------
print("\nScree table:")
for i, ev in enumerate(eigenvalues):
    print(f"  PC{i+1}: eigenvalue={ev:.3f},"
          f" var explained={ev/eigenvalues.sum()*100:.1f}%")
```

2.2 Probability and statistics: the language of uncertainty

Real data is noisy and future observations are stochastic, so predictions in machine learning are fundamentally probabilistic statements.

Expectation, variance, and covariance. For a random variable X, the **expected value** $\mathbb{E}[X] = \int xp(x)\,dx$ (or $\sum x_i P(x_i)$) measures central tendency. The **variance** quantifies dispersion:

$$\mathrm{Var}(X) = \mathbb{E}\big[(X - \mathbb{E}[X])^2\big] = \mathbb{E}[X^2] - (\mathbb{E}[X])^2. \tag{12}$$

For two variables X and Y, **covariance** measures joint linear variation $\mathrm{Cov}(X, Y) = \mathbb{E}\big[(X - \mathbb{E}[X])(Y - \mathbb{E}[Y])\big]$. When normalized by standard deviations $\sigma_X \sigma_Y$, it yields the Pearson correlation coefficient $\rho_{X,Y} \in [-1, 1]$.

Common distributions. A small family covers foundational ML tasks: the *Bernoulli* for binary outcomes $y \in \{0, 1\}$; the *Categorical* for K-class labels and LLM tokens; and the *Gaussian (Normal)* distribution for measurement noise:

$$p(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right). \tag{13}$$

Likelihood, MLE, and MAP estimation. Parameter fitting estimates unknown distribution parameters $\boldsymbol{\theta}$ from data $\mathcal{D} = \{x_1, \ldots, x_m\}$. The **Likelihood function** $L(\boldsymbol{\theta}) = p(\mathcal{D} \mid \boldsymbol{\theta}) = \prod_{i=1}^{m} p(x_i \mid \boldsymbol{\theta})$ evaluates data probability under candidate $\boldsymbol{\theta}$. Maximizing likelihood (or minimizing negative log-likelihood $-\ln L(\boldsymbol{\theta})$) gives the **Maximum Likelihood Estimator (MLE)**:

$$\hat{\boldsymbol{\theta}}_{\mathrm{MLE}} = \arg\max_{\boldsymbol{\theta}} \sum_{i=1}^{m} \ln p(x_i \mid \boldsymbol{\theta}). \tag{14}$$

When prior knowledge $p(\boldsymbol{\theta})$ is available, **Maximum A Posteriori (MAP)** estimation incorporates the prior via Bayes' theorem:

$$\hat{\boldsymbol{\theta}}_{\mathrm{MAP}} = \arg\max_{\boldsymbol{\theta}} \left[\sum_{i=1}^{m} \ln p(x_i \mid \boldsymbol{\theta}) + \ln p(\boldsymbol{\theta})\right], \tag{15}$$

which directly derives ℓ_2 (Ridge) and ℓ_1 (Lasso) regularisation terms from Gaussian and Laplace priors, respectively.

Bayes' theorem. The foundation of probabilistic updating connects prior beliefs to observed data:

$$P(A \mid B) \;=\; \frac{P(B \mid A)\,P(A)}{P(B)}. \tag{16}$$

> **Bayes' theorem, in words**
>
> Posterior $\propto$ likelihood $\times$ prior: what you believe *after* seeing evidence is proportional to how well each hypothesis would have produced that evidence, weighted by what you believed *before*.

A medical screening example demonstrates why base rates dominate intuition. A diagnostic test with 99% sensitivity and a 5% false-positive rate, applied to a rare disease with 1% prevalence,

yields

$$P(\text{disease} \mid +) = \frac{P(+ \mid \text{disease})\,P(\text{disease})}{P(+)} \tag{17}$$

$$= \frac{\overbrace{0.99}^{\text{sensitivity}} \times \overbrace{0.01}^{P(\text{disease})}}{\underbrace{0.99}_{\text{sensitivity}} \times \underbrace{0.01}_{P(\text{disease})} + \underbrace{0.05}_{\text{false pos rate}} \times \underbrace{0.99}_{P(\text{no disease})}} \approx 0.17 : \tag{18}$$

only one positive in six is a true case, because the disease is rare to begin with. Note that the two 0.99 terms in the denominator represent entirely different quantities: test sensitivity $P(+ \mid \text{disease}) = 0.99$ versus healthy prior $P(\text{no disease}) = 1 - 0.01 = 0.99$. The same base-rate logic governs fraud alerts, security anomaly detection, and imbalanced classification.

Hypothesis testing, confidence intervals, and the CLT. To discipline empirical findings:

- **Central Limit Theorem (CLT):** For independent observations with finite variance σ^2, the standard error of the sample mean is exactly $\sigma/\sqrt{n}$. The CLT additionally states that as $n \to \infty$, the standardized sample mean distribution converges to a standard normal distribution $\mathcal{N}(0, 1)$.
- **Confidence Intervals & Bootstrap Resampling:** Rather than relying solely on single-point estimates or p-values (which can be vulnerable to p-hacking under multiple testing), practitioners construct 95% confidence intervals via asymptotic standard errors or non-parametric *bootstrap resampling* (repeatedly sampling data with replacement to estimate metric variance empirical distributions).

Hands-on with Python

```python
import numpy as np

rng = np.random.default_rng(42)
# 10,000 experiments: average of 40 Exponential(1) draws each
means = rng.exponential(scale=1.0, size=(10_000, 40)).mean(axis=1)
print(f"Sample Mean: {means.mean():.3f} | SE: {means.std():.3f}") # ~1.0 & 1/sqrt(40)
```

The standard deviation of sample means matches standard error $\sigma/\sqrt{n}$, illustrating CLT convergence.

2.3 Optimisation: how models learn

A **loss function** $\mathcal{L}(\hat{y}, y)$ scores error on a single instance; the **cost function** $J(\theta)$ (empirical risk) averages loss over m samples:

$$J(\boldsymbol{\theta}) = \frac{1}{2m} \sum_{i=1}^{m} \big(h_{\boldsymbol{\theta}}(\mathbf{x}_i) - y_i\big)^2. \tag{19}$$

The **gradient** $\nabla_{\boldsymbol{\theta}} J$ points in the direction of steepest *ascent*. Parameter updates step in the opposite direction:

$$\boldsymbol{\theta}_{t+1} \;=\; \boldsymbol{\theta}_t \;-\; \eta\, \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}_t), \tag{20}$$

where $\eta > 0$ is the learning rate. For strictly convex objectives (e.g., OLS linear regression with full-rank X or ℓ_2 regularisation), gradient descent converges to the unique global minimum. For non-convex neural network loss surfaces, stochastic mini-batch gradient descent (SGD) and adaptive optimizers reliably locate high-quality local minima.

Adam and AdamW optimisers. Standard SGD uses a fixed step size. The **Adam** optimiser (Kingma and Ba 2015) maintains exponential moving averages of gradients g_t (first moment m_t) and squared gradients g_t^2 (second moment v_t):

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \qquad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2, \tag{21}$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t}, \qquad \boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \frac{\eta\, \hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}. \tag{22}$$

In modern deep learning and Transformer training, **AdamW** (Loshchilov and Hutter 2019) is strongly preferred over standard Adam because it decouples ℓ_2 weight decay regularization from adaptive gradient moment updates, preventing gradient scaling from inadvertently suppressing weight decay penalty terms.

Adam optimiser step from scratch

```python
import numpy as np

def adam_step(theta, grad, m, v, t, lr=0.01,
              beta1=0.9, beta2=0.999, eps=1e-8):
    m = beta1 * m + (1 - beta1) * grad
    v = beta2 * v + (1 - beta2) * (grad ** 2)
    m_hat = m / (1 - beta1 ** t)
    v_hat = v / (1 - beta2 ** t)
    theta_next = theta - (lr * m_hat) / (np.sqrt(v_hat) + eps)
    return theta_next, m, v
```

Adam updates each parameter using bias-corrected first and second moment estimates.

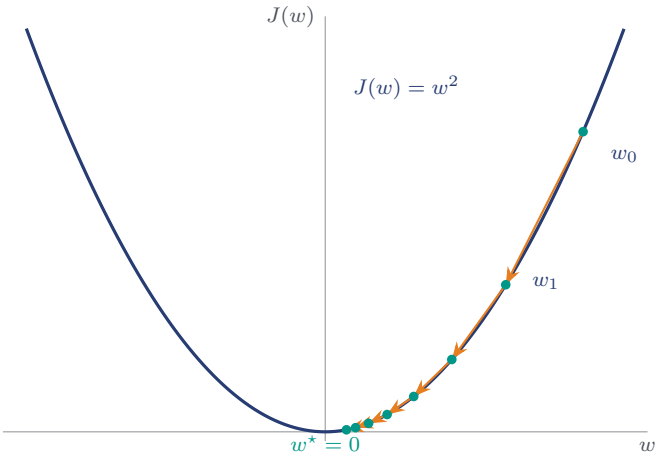


Figure 4. Gradient descent on $J(w) = w^2$ with learning rate $\eta = 0.15$: each step $w \leftarrow w - \eta\, 2w$ shrinks the distance to the minimum by 30%.

Hands-on with Python

```python
w, lr = 5.0, 0.15
for step in range(8):
    grad = 2 * w            # derivative of f(w) = w**2
    w = w - lr * grad       # the update rule, eq. (11)
    print(f"step {step}:  w = {w:.4f}")
    # w -> 0.2882: convergence toward the minimum
```

Check your understanding

1. In plain words, what does an eigenvector of the covariance matrix tell you about a dataset?
2. In the medical-test example, what happens to $P(\text{disease} \mid +)$ as prevalence rises to 10%?
3. What goes wrong, mechanically, when the learning rate in equation (20) is set to 1.1 for $J(w) = w^2$?

Sources & further reading

- G. Strang (2019). *Linear Algebra and Learning from Data*. Wellesley, MA: Wellesley–Cambridge Press
- G. Strang (2012). *Linear Algebra and Its Applications*. 4th ed. Cengage Learning
- G. H. Golub and C. F. Van Loan (2013). *Matrix Computations*. 4th ed. Baltimore, MD: Johns Hopkins University Press
- L. N. Trefethen and D. Bau (1997). *Numerical Linear Algebra*. Philadelphia, PA: SIAM
- K. Pearson (1901). “On Lines and Planes of Closest Fit to Systems of Points in Space”. In: *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science* 2.11, pp. 559–572
- L. Wasserman (2004). *All of Statistics: A Concise Course in Statistical Inference*. New York: Springer
- S. Boyd and L. Vandenberghe (2004). *Convex Optimization*. Cambridge: Cambridge University Press
- I. Goodfellow et al. (2016). *Deep Learning*. Cambridge, MA: MIT Press
- C. M. Bishop (2006). *Pattern Recognition and Machine Learning*. New York: Springer
- C. R. Harris et al. (2020). “Array Programming with NumPy”. In: *Nature* 585, pp. 357–362
- W. McKinney (2010). “Data Structures for Statistical Computing in Python”. In: *Proceedings of the 9th Python in Science Conference (SciPy)*, pp. 56–61
- J. VanderPlas (2016). *Python Data Science Handbook: Essential Tools for Working with Data*. Sebastopol, CA: O’Reilly Media

3 Classical Machine Learning: The Core Engine

Module 3, at a glance (Estimated Study: 10 hours • Level: Intermediate)

Focus. The algorithms that still run the majority of real-world, structured-data workloads, and the evaluation discipline that separates real learning from memorisation.

Key concepts. Supervised learning: regression, decision trees, SVMs, ensembles. Unsupervised learning: K-Means clustering and anomaly detection. Self-supervised learning: data self-annotation and pretext tasks. Reinforcement learning: MDPs, multi-armed bandits, exploration vs. exploitation. Evaluation: accuracy, precision, recall, F1, ROC-AUC.

After this module you will be able to choose an appropriate classical algorithm for a task, cluster unlabelled data, construct self-annotation pretext tasks, solve multi-armed bandit problems, and (most importantly) evaluate any model honestly.

On the tabular, structured data of business (customers, transactions, sensors), classical machine learning frequently still beats glamorous deep models. It is also the best place to internalise the habits (splits, metrics, baselines) that govern everything later (Hastie et al. 2009; Pedregosa et al. 2011).

3.1 Supervised learning

Supervised learning starts from labelled pairs $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^m$ and a split into *training*, *validation*, and *test* sets. If y is a number the task is *regression*; if y is a category it is *classification*.

Constructing the design matrix X from raw data. Consider a dataset of m observations $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^m$, where each feature vector $\mathbf{x}_i = (x_{i1}, x_{i2}, \ldots, x_{in})^\top \in \mathbb{R}^n$ represents n input variables. A linear regression model expresses the prediction as an affine combination:

$$\hat{y}_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_n x_{in} = \beta_0 + \boldsymbol{\beta}^\top \mathbf{x}_i, \tag{23}$$

where $\beta_0 \in \mathbb{R}$ is the intercept (bias) and $\boldsymbol{\beta} = (\beta_1, \ldots, \beta_n)^\top \in \mathbb{R}^n$ is the weight vector. To express predictions across all m samples compactly as a single matrix–vector product $\hat{\mathbf{y}} = X\boldsymbol{\beta}_{\text{full}}$, we prepend a dummy constant feature $x_{i0} = 1$ to each sample vector $\tilde{\mathbf{x}}_i = (1, x_{i1}, \ldots, x_{in})^\top \in \mathbb{R}^{n+1}$ and define the parameter vector $\boldsymbol{\beta}_{\text{full}} = (\beta_0, \beta_1, \ldots, \beta_n)^\top \in \mathbb{R}^{n+1}$. Stacking these augmented row vectors $\tilde{\mathbf{x}}_i^\top$ forms the **design matrix** $X \in \mathbb{R}^{m \times (n+1)}$:

$$X = \begin{bmatrix} 1 & x_{11} & x_{12} & \ldots & x_{1n} \\ 1 & x_{21} & x_{22} & \ldots & x_{2n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{m1} & x_{m2} & \ldots & x_{mn} \end{bmatrix} \in \mathbb{R}^{m \times (n+1)}. \tag{24}$$

Under this representation, the Ordinary Least Squares (OLS) objective minimises the residual sum of squares $\|\mathbf{y} - X\boldsymbol{\beta}\|_2^2$, yielding the closed-form normal equations solution:

$$\hat{\boldsymbol{\beta}}_{\text{OLS}} = (X^\top X)^{-1} X^\top \mathbf{y}, \tag{25}$$

computed via QR factorisation or SVD for numerical stability (Strang 2012; Strang 2019).

Feature maps and basis expansion. The matrix X is not restricted to raw measurements. By applying a non-linear feature mapping $\boldsymbol{\phi}(\mathbf{x}) = [\phi_1(\mathbf{x}), \ldots, \phi_d(\mathbf{x})]^\top$, linear models can capture complex non-linear curves. For example, a 1D input x expanded into a degree-d polynomial basis

yields row vectors $\tilde{\mathbf{x}}_i^\top = [1, x_i, x_i^2, \ldots, x_i^d]$, transforming non-linear fitting into a linear problem in feature space $\mathbb{R}^{d+1}$.

Building Design Matrix X & Basis Expansion

```python
import numpy as np

x_raw = np.array([1.5, 2.0, 3.5, 4.0, 5.5])
y = np.array([2.1, 2.9, 6.1, 8.2, 14.5])

# 1. Prepend bias column of 1s -> X in R^(m x 2)
X_linear = np.column_stack([np.ones(len(x_raw)), x_raw])

# 2. Basis expansion: Polynomial features [1, x, x^2, x^3]
X_poly = np.column_stack([x_raw**p for p in range(4)])

# Closed-form OLS via QR decomposition (R beta = Q^T y) & SVD (avoiding inv(X^T X)):
Q, R = np.linalg.qr(X_poly)
beta_poly_qr = np.linalg.solve(R, Q.T @ y)
beta_poly_svd = np.linalg.lstsq(X_poly, y, rcond=None)[0]

print(f"Linear X shape:   {X_linear.shape} | beta: {np.linalg.lstsq(X_linear, y,
    rcond=None)[0].round(2)}")
print(f"Poly X (QR):      {X_poly.shape} | beta: {beta_poly_qr.round(2)}")
print(f"Poly X (SVD):     {X_poly.shape} | beta: {beta_poly_svd.round(2)}")
```

Regularisation: Ridge (ℓ_2) and Lasso (ℓ_1). When features are highly correlated or $n > m$, OLS exhibits high variance. Regularisation fights overfitting by penalising model complexity. Ridge regression adds an ℓ_2 penalty:

$$\hat{\boldsymbol{\beta}}_{\text{Ridge}} = \arg\min_{\boldsymbol{\beta}} \frac{1}{2m}\|\mathbf{y} - X\boldsymbol{\beta}\|_2^2 + \alpha\|\boldsymbol{\beta}\|_2^2, \tag{26}$$

which shrinks all coefficients smoothly toward zero but keeps them non-zero. By contrast, the **Lasso** (ℓ_1 regularisation) penalises the sum of absolute values:

$$\hat{\boldsymbol{\beta}}_{\text{Lasso}} = \arg\min_{\boldsymbol{\beta}} \frac{1}{2m}\|\mathbf{y} - X\boldsymbol{\beta}\|_2^2 + \alpha\|\boldsymbol{\beta}\|_1, \qquad \|\boldsymbol{\beta}\|_1 = \sum_{j=1}^{n} |\beta_j|. \tag{27}$$

Why Lasso induces sparsity. Unlike ℓ_2 sphere contours, the ℓ_1 constraint set $\{\boldsymbol{\beta} : \|\boldsymbol{\beta}\|_1 \leq t\}$ is a geometric polytope (a diamond in 2D) with sharp corners located directly on the coordinate axes. The elliptical level sets of the unconstrained OLS loss expand until they touch the constraint boundary; because the diamond's vertices lie on the axes where one or more coefficients equal zero, the solution frequently lands directly on a corner, driving uninformative feature weights strictly to zero (Hastie et al. 2009). Analytically, the absolute value function $|\beta_j|$ is non-differentiable at zero, yielding a subdifferential interval $[-1, 1]$. Coordinate descent solves for β_j via the *soft-thresholding operator*:

$$\mathcal{S}_\gamma(z) = \operatorname{sign}(z)\max\big(0,\ |z| - \gamma\big), \tag{28}$$

which clamps small feature correlations below $\gamma = \alpha$ to exactly zero, performing automated feature selection.

Lasso (ℓ_1) Regularisation & Numerical Solvers

```python
import numpy as np

# Synthetic dataset with collinearity & sparse ground truth
rng = np.random.default_rng(42)
m, n = 100, 5
X = rng.normal(size=(m, n))
X[:, 4] = X[:, 0] + X[:, 1] + 1e-4 * rng.normal(size=m) # collinear feature
true_w = np.array([3.0, -2.5, 1.5, 0.0, 0.0])
y = X @ true_w + 0.2 * rng.normal(size=m)

# 1. Numerically stable OLS Normal Equations via QR & SVD
Q, R = np.linalg.qr(X)
beta_qr = np.linalg.solve(R, Q.T @ y)             # QR decomposition solver
beta_svd = np.linalg.lstsq(X, y, rcond=None)[0]   # SVD solver

# 2. Lasso via Coordinate Descent with Soft-Thresholding
def soft_threshold(z, gamma):
    return np.sign(z) * np.maximum(0, np.abs(z) - gamma)

def lasso_cd(X, y, alpha, max_iter=500):
    m, n = X.shape
    w = np.zeros(n)
    norm_x2 = np.sum(X**2, axis=0)
    for _ in range(max_iter):
        for j in range(n):
            r_j = y - (X @ w - X[:, j] * w[j])     # partial residual
            rho_j = X[:, j] @ r_j
            w[j] = soft_threshold(rho_j, m * alpha) / norm_x2[j]
    return w

w_lasso_np = lasso_cd(X, y, alpha=0.1)

print("True weights: ", true_w)
print("OLS QR:       ", beta_qr.round(2))          # unstable on collinear feature
print("NumPy Lasso:  ", w_lasso_np.round(2))       # zeroes noise & collinear weights
```

Classification: Sigmoid, Logistic Regression, and Softmax. When target labels y are categorical, linear output scores $z = \boldsymbol{\beta}^\top \mathbf{x} + \beta_0$ are unbounded and unsuitable for probabilities.

Binary Logistic Regression. For two classes $y \in \{0, 1\}$, the **sigmoid (logistic) function** maps any real score $z \in \mathbb{R}$ into a valid probability $p \in (0, 1)$:

$$\sigma(z) = \frac{1}{1 + e^{-z}} = \frac{e^z}{e^z + 1}. \tag{29}$$

The model interprets $p = P(y = 1 \mid \mathbf{x})$, implying that linear log-odds are given by $\ln\left(\frac{p}{1-p}\right) = \boldsymbol{\beta}^\top \mathbf{x} + \beta_0$. Fitting parameters via Maximum Likelihood Estimation yields the **Binary Cross-Entropy (BCE)** cost function:

$$\mathcal{L}(\boldsymbol{\beta}) = -\frac{1}{m} \sum_{i=1}^{m} \left[y_i \ln \hat{y}_i + (1 - y_i) \ln(1 - \hat{y}_i) \right], \qquad \hat{y}_i = \sigma(\boldsymbol{\beta}^\top \mathbf{x}_i + \beta_0). \tag{30}$$

Multinomial Logistic Regression and Softmax. For $K > 2$ discrete classes $y \in \{1, \ldots, K\}$, the model outputs a vector of unnormalised scores (logits) $\mathbf{z} = W\mathbf{x} + \mathbf{b} \in \mathbb{R}^K$, where $W \in \mathbb{R}^{K \times n}$. The **Softmax function** exponentiates and normalises logits into a categorical probability distribution

$\hat{\mathbf{p}} = [\hat{p}_1, \ldots, \hat{p}_K]^\top$:

$$\text{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \qquad k = 1, \ldots, K. \tag{31}$$

Key properties of Softmax:

1. **Positivity and Normalisation:** Exponentiation enforces $e^{z_k} > 0$, and dividing by $\sum_j e^{z_j}$ guarantees $\sum_{k=1}^{K} \hat{p}_k = 1$ on the probability simplex.
2. **Temperature Scaling:** Incorporating a temperature parameter $T > 0$ as $\text{softmax}(\mathbf{z}/T)$ controls confidence: as $T \to 0^+$ predictions sharpen toward a one-hot argmax, whereas high $T \to \infty$ produces a uniform distribution (a mechanism central to LLM sampling in Module 4).
3. **Generalisation of Sigmoid:** For $K = 2$, Softmax on $[z_1, z_2]$ simplifies identically to Sigmoid on $(z_1 - z_2)$.

Training multinomial logistic regression minimises the **Categorical Cross-Entropy** loss:

$$\mathcal{L}(W, \mathbf{b}) = -\frac{1}{m} \sum_{i=1}^{m} \sum_{k=1}^{K} y_{i,k} \ln \hat{p}_{i,k}, \tag{32}$$

where $y_{i,k} \in \{0, 1\}$ is the one-hot target indicator for class k.

Sigmoid, Softmax & Cross-Entropy Loss from Scratch

```python
import numpy as np

def sigmoid(z):
    return 1 / (1 + np.exp(-np.clip(z, -500, 500)))

def softmax(Z, temp=1.0):
    # Subtract max logit per row for numerical stability
    Z_scaled = Z / temp
    exp_Z = np.exp(Z_scaled - np.max(Z_scaled, axis=-1, keepdims=True))
    return exp_Z / np.sum(exp_Z, axis=-1, keepdims=True)

# 3 samples, 4 classes (logits Z in R^(3 x 4))
logits = np.array([[2.0, 1.0, 0.1, 3.0],
                   [0.5, 4.0, 1.2, 0.0],
                   [-1.0, 0.0, 5.0, 0.2]])

probs = softmax(logits, temp=1.0)
y_true = np.array([3, 1, 2]) # True targets for each sample

# Categorical Cross-Entropy Loss: -log(p_true)
nll = -np.log(probs[np.arange(len(y_true)), y_true])
print("Probabilities:\n", probs.round(3))
print(f"Per-sample loss: {nll.round(3)} | Mean: {nll.mean():.3f}")
```

Naive Bayes and k-NN baselines. Bridging directly from Bayes' theorem (equation 16), the **Naive Bayes classifier** computes the posterior probability of class $y \in \mathcal{Y}$ given features $\mathbf{x} = (x_1, \ldots, x_n)$ by assuming features are conditionally independent given y:

$$\hat{y} = \arg\max_{y \in \mathcal{Y}} \; P(y \mid \mathbf{x}) = \arg\max_{y \in \mathcal{Y}} \; P(y) \prod_{j=1}^{n} P(x_j \mid y). \tag{33}$$

Depending on data types, feature likelihoods $P(x_j \mid y)$ assume specific parametric distributions:

- **Gaussian Naive Bayes** for continuous features: $P(x_j \mid y) = \frac{1}{\sqrt{2\pi\sigma_{y,j}^2}} \exp\left(-\frac{(x_j-\mu_{y,j})^2}{2\sigma_{y,j}^2}\right)$, where $\mu_{y,j}$ and $\sigma_{y,j}^2$ are class-conditional empirical means and variances.
- **Multinomial & Bernoulli Naive Bayes** for discrete token counts or binary indicators in NLP.

To prevent zero-frequency probability collapse on unseen features during inference, **Laplace smoothing** adds a prior pseudo-count $\alpha \geq 1$: $\hat{P}(x_j \mid y) = \frac{N_{y,x_j}+\alpha}{N_y+\alpha \cdot d}$. In practice, products of small probabilities cause numerical underflow; computations sum log-likelihoods instead: $\ln P(y) + \sum_{j=1}^{n} \ln P(x_j \mid y)$.

The **k-Nearest Neighbors (k-NN)** algorithm is a non-parametric instance-based baseline that stores all training observations $\mathcal{D}$. For a query point $\mathbf{x}$, it identifies the set $\mathcal{N}_k(\mathbf{x})$ of k closest samples under a distance metric (e.g., Euclidean distance $d_2(\mathbf{x}, \mathbf{z}) = \|\mathbf{x} - \mathbf{z}\|_2$ or Manhattan distance $d_1(\mathbf{x}, \mathbf{z}) = \|\mathbf{x} - \mathbf{z}\|_1$) and assigns the majority class vote:

$$\hat{y} = \operatorname{mode}\big(\{y_i : \mathbf{x}_i \in \mathcal{N}_k(\mathbf{x})\}\big). \tag{34}$$

Hyperparameter k controls the bias–variance trade-off: $k = 1$ creates complex, low-bias decision boundaries prone to overfitting noise, whereas large k over-smooths class margins (high bias). While k-NN requires zero training compute, inference cost scales linearly as $O(m \cdot n)$ per query; spatial indexing trees (KD-Trees, Ball Trees) accelerate spatial queries in low-to-moderate dimensions ($n \ll 50$).

Gaussian Naive Bayes from Scratch & k-NN Baselines

```python
import numpy as np
from sklearn.neighbors import KNeighborsClassifier

class GaussianNBScratch:
    def fit(self, X, y):
        self.classes = np.unique(y)
        self.priors = np.array([np.mean(y == c) for c in self.classes])
        self.means = np.array([X[y == c].mean(axis=0) for c in self.classes])
        self.vars = np.array([X[y == c].var(axis=0) + 1e-9 for c in self.classes])
        return self

    def predict(self, X):
        # Log likelihood for numerical stability: log P(y) + sum log P(x_j|y)
        log_probs = []
        for i, c in enumerate(self.classes):
            log_prior = np.log(self.priors[i])
            log_pdf = -0.5 * np.sum(np.log(2 * np.pi * self.vars[i]) +
                                    ((X - self.means[i])**2) / self.vars[i], axis=1)
            log_probs.append(log_prior + log_pdf)
        return self.classes[np.argmax(np.array(log_probs), axis=0)]

# Synthetic binary dataset
rng = np.random.default_rng(42)
X = rng.normal(size=(200, 4))
y = (X[:, 0] + X[:, 1] > 0).astype(int)

gnb = GaussianNBScratch().fit(X[:150], y[:150])
knn = KNeighborsClassifier(n_neighbors=5).fit(X[:150], y[:150])

print(f"Gaussian NB (Scratch) Acc: {(gnb.predict(X[150:]) == y[150:]).mean():.3f}")
print(f"k-NN (k=5) Test Acc:       {knn.score(X[150:], y[150:]):.3f}")
```

Naive Bayes computes class posterior probabilities in log-space via Gaussian feature likelihoods, matching k-NN baseline performance on smooth distributions.

Decision trees and ensembles. A decision tree partitions feature space recursively into axis-aligned rectangular regions using a sequence of greedy binary splits. At node m, the algorithm searches over all features j and split thresholds t to maximize impurity reduction $\Delta I = I(m) - \frac{|S_L|}{|S|} I(S_L) - \frac{|S_R|}{|S|} I(S_R)$, where $|S|$ is node sample count. Common classification impurity metrics for class proportions $\mathbf{p} = (p_1, \ldots, p_K)$ include:

$$\text{Gini Impurity:} \quad I_G(\mathbf{p}) = 1 - \sum_{k=1}^{K} p_k^2, \qquad \text{Entropy:} \quad H(\mathbf{p}) = -\sum_{k=1}^{K} p_k \log_2 p_k. \tag{35}$$

For regression trees, splits minimize variance (MSE) $I_{\text{MSE}}(S) = \frac{1}{|S|} \sum_{i \in S} (y_i - \bar{y}_S)^2$. Unconstrained trees grow until leaves are pure, causing severe overfitting (high variance). Overfitting is controlled via pre-pruning constraints (`max_depth`, `min_samples_split`) or cost-complexity post-pruning $R_\alpha(T) = R(T) + \alpha |T|$.

To overcome single-tree overfitting, **ensemble methods** combine multiple weak tree learners (Hastie et al. 2009):

- **Random Forests (Bagging):** Grows B independent de-correlated trees on bootstrap samples of training rows, evaluating only a random subset of $\sqrt{n}$ features per split (Breiman 2001). Averaging tree outputs reduces variance by factor $\text{Var}(\bar{X}) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$, where ρ is pairwise tree correlation.
- **Gradient Boosting (XGBoost / LightGBM):** Builds trees sequentially: $\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + \eta f_t(\mathbf{x}_i)$, where step t fits tree f_t to pseudo-residuals $r_{i,t} = -\left[\frac{\partial L(y_i, \hat{y})}{\partial \hat{y}}\right]_{\hat{y}^{(t-1)}}$ (Chen and Guestrin 2016). Gradient boosting achieves state-of-the-art accuracy on tabular data.

Decision Trees, Gini Impurity & Ensembles

```python
import numpy as np
from sklearn.ensemble import RandomForestClassifier, HistGradientBoostingClassifier

def gini_impurity(y):
    """Compute Gini Impurity I_G(p) = 1 - sum(p_k^2)"""
    if len(y) == 0: return 0.0
    p = np.bincount(y) / len(y)
    return 1.0 - np.sum(p**2)

# Synthetic classification dataset with non-linear interaction
rng = np.random.default_rng(42)
X = rng.normal(size=(500, 10))
y = (X[:, 0]*2 - X[:, 1] + X[:, 2]**2 > 1.0).astype(int)

rf = RandomForestClassifier(n_estimators=100, max_depth=5, random_state=42).fit(X[:400],
    y[:400])
gb = HistGradientBoostingClassifier(max_iter=100, max_depth=5,
    random_state=42).fit(X[:400], y[:400])

print(f"Root Split Gini Impurity:   {gini_impurity(y[:400]):.3f}")
print(f"Random Forest Test Acc:     {rf.score(X[400:], y[400:]):.3f}")
print(f"Gradient Boosting Test Acc: {gb.score(X[400:], y[400:]):.3f}")
```

Ensembles aggregate decision tree splits; gradient boosting iteratively corrects prior tree errors to model complex non-linear feature interactions.

Support Vector Machines (SVMs). The SVM finds the decision boundary $f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b = 0$ maximizing the margin $\frac{2}{\|\mathbf{w}\|_2}$ to the nearest training points (Cortes and Vapnik 1995). For non-

linearly separable data, the **soft-margin formulation** introduces slack variables $\xi_i \geq 0$:

$$\min_{\mathbf{w},b,\boldsymbol{\xi}} \frac{1}{2}\|\mathbf{w}\|_2^2 + C\sum_{i=1}^{m}\xi_i \qquad \text{s.t.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0, \tag{36}$$

where target labels $y_i \in \{-1, +1\}$ and hyperparameter $C > 0$ trades off margin maximization against training errors. Mathematically, soft-margin SVM minimizes unconstrained **Hinge Loss** plus ℓ_2 regularization: $\mathcal{L} = \sum_{i=1}^{m} \max\big(0, 1 - y_i f(\mathbf{x}_i)\big) + \frac{1}{2C}\|\mathbf{w}\|_2^2$.

Formulating the dual problem using Lagrange multipliers $\alpha_i \geq 0$ yields:

$$\max_{\boldsymbol{\alpha}} \sum_{i=1}^{m}\alpha_i - \frac{1}{2}\sum_{i=1}^{m}\sum_{j=1}^{m}\alpha_i\alpha_j y_i y_j \big(\mathbf{x}_i^\top \mathbf{x}_j\big) \qquad \text{s.t.} \quad 0 \leq \alpha_i \leq C, \quad \sum_{i=1}^{m}\alpha_i y_i = 0. \tag{37}$$

Observations with $\alpha_i > 0$ form the **support vectors** lying on or inside the margin. The dual formulation reveals that predictions depend exclusively on dot products $\mathbf{x}_i^\top \mathbf{x}_j$. Replacing this dot product with a non-linear kernel function $K(\mathbf{x}_i, \mathbf{x}_j) = \boldsymbol{\phi}(\mathbf{x}_i)^\top \boldsymbol{\phi}(\mathbf{x}_j)$, known as the **Kernel Trick**, allows SVMs to construct non-linear decision boundaries in high-dimensional Hilbert spaces without explicitly computing expensive feature maps $\boldsymbol{\phi}(\cdot)$:

- **Linear Kernel:** $K(\mathbf{x}, \mathbf{z}) = \mathbf{x}^\top \mathbf{z}$.
- **Polynomial Kernel:** $K(\mathbf{x}, \mathbf{z}) = (\mathbf{x}^\top \mathbf{z} + c)^d$.
- **Radial Basis Function (RBF / Gaussian Kernel):** $K(\mathbf{x}, \mathbf{z}) = \exp\big(-\gamma\|\mathbf{x} - \mathbf{z}\|_2^2\big)$, where $\gamma > 0$ controls the inverse length scale (higher γ yields a smaller effective radius and tighter decision boundaries; lower γ yields broader spatial influence).

Support Vector Machines: Linear vs. RBF Kernel

```python
import numpy as np
from sklearn.svm import SVC

# Generate non-linearly separable concentric circles dataset
rng = np.random.default_rng(42)
X = rng.uniform(-2, 2, size=(300, 2))
y = ((X[:, 0]**2 + X[:, 1]**2) < 1.5).astype(int)

# Compare Linear vs RBF Kernel SVM
svm_lin = SVC(kernel='linear', C=1.0).fit(X[:240], y[:240])
svm_rbf = SVC(kernel='rbf', C=1.0, gamma='scale').fit(X[:240], y[:240])

print(f"Linear Kernel Test Acc: {svm_lin.score(X[240:], y[240:]):.3f}")
print(f"RBF Kernel Test Acc:    {svm_rbf.score(X[240:], y[240:]):.3f}")
print(f"RBF Support Vectors:    {len(svm_rbf.support_vectors_)} / 240 samples")
```

The RBF kernel projects non-linearly separable points into implicit infinite-dimensional space, creating complex circular decision boundaries governed by support vectors.

Feature scaling requirement. Algorithms that calculate spatial distances (K-Means, k-NN, SVM) or compute gradient steps (Logistic/Linear Regression, Neural Networks) are highly sensitive to feature scales: features with large magnitudes (e.g., house area in sq ft) dominate Euclidean metrics over features with small ranges (e.g., room count). Scaling is generally critical for distance-based algorithms (k-NN, SVM, K-Means) and often beneficial for gradient-based optimization (Logistic/Linear Regression, Neural Networks), while tree-based models are largely scale-invariant.

Table 4. Choosing a first algorithm for tabular data.

Algorithm	Reach for it when	Strengths	Watch out
Linear / logistic regression	Baselines; small, clean data	Fast, interpretable, statistically principled	Underfits complex nonlinear structure
Naive Bayes	Text classification, fast baseline	Very fast, handles high dimensions	Strong feature independence assumption
k-NN	Small datasets, distance baselines	No training phase; flexible boundaries	Slow inference $O(mn)$; scale-sensitive
Decision tree	Interpretability is the product	Human-readable rules; no feature scaling	Single trees overfit badly
Random forest	Default robust choice	Variance reduction via bagging; few knobs	Less interpretable; larger memory
Gradient boosting (XGBoost)	Maximum accuracy on tables	State of the art on structured data	Needs tuning; can overfit small data
SVM	Small–medium data, many features	Max-margin generalisation; kernels	Scales poorly to very large datasets

⚖ CRITICAL: Avoid Data Leakage

Fit scalers on training data ONLY. A ubiquitous beginner error is fitting a scaler on the entire dataset prior to splitting. This causes *data leakage*: information from the test set (mean μ and standard deviation σ) leaks into the training process, yielding artificially optimistic evaluation metrics. Always compute scaling parameters using `fit_transform()` on the training split, and apply `transform()` on validation/test splits. Use `sklearn.pipeline.Pipeline` to automate this safely.

3.2 Unsupervised learning

Unsupervised learning discovers underlying structure without target labels.

K-Means & Silhouette Score. The canonical partitioning algorithm **K-Means** (MacQueen 1967) minimizes within-cluster inertia $J = \sum_{k=1}^{K} \sum_{\mathbf{x}_i \in C_k} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|_2^2$ via Lloyd's algorithm (hard-assignment EM updates).[1] Cluster separation quality is quantified by the **Silhouette Score** $s(i) = \frac{b(i)-a(i)}{\max(a(i),b(i))}$ (Rousseeuw 1987).

Density-based & Mixture Models: DBSCAN & GMMs. K-Means assumes spherical, equal-sized clusters. To identify arbitrary non-convex clusters, **DBSCAN** (Ester et al. 1996) groups points based on density (ϵ-neighborhoods and minimum core samples), naturally isolating noise outliers. For probabilistic soft-clustering, **Gaussian Mixture Models (GMMs)** represent data distributions as a linear combination of K Gaussians $\sum \pi_k \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k, \Sigma_k)$, fitted via the Expectation-Maximization (EM) algorithm (Bishop 2006).

Non-linear Dimensionality Reduction: t-SNE & UMAP. While PCA computes linear orthogonal projections, non-linear manifolds require non-linear embedding algorithms:

- **t-SNE** (Maaten and Hinton 2008) converts pairwise Euclidean distances into conditional probabilities under Student-t distributions, preserving local neighborhood structures in 2D/3D visualizations.

[1] K-Means uses hard assignment (each point belongs to exactly one cluster), whereas Gaussian Mixture Models (GMMs) employ soft responsibilities via the full Expectation-Maximization algorithm as detailed below.

- **UMAP** (McInnes et al. 2018) constructs fuzzy simplicial sets to optimize low-dimensional layouts, preserving both local and global structure with superior scalability.

Anomaly Detection. Methods include Isolation Forest (Liu et al. 2008) (random feature splits isolating anomalies near root nodes), Elliptic Envelope, and Autoencoders (Module 4).

DBSCAN, K-Means & Silhouette Score

```python
from sklearn.cluster import KMeans, DBSCAN
from sklearn.metrics import silhouette_score
from sklearn.datasets import make_blobs
import numpy as np

# Synthetic data generation for standalone execution
X_tr, _ = make_blobs(n_samples=150, centers=3, cluster_std=0.6, random_state=42)

# Fit K-Means and DBSCAN on synthetic data
kmeans = KMeans(n_clusters=3, n_init=10, random_state=42).fit(X_tr)
dbscan = DBSCAN(eps=0.5, min_samples=5).fit(X_tr)

score = silhouette_score(X_tr, kmeans.labels_)
print(f"K-Means Silhouette Score: {score:.3f}")
print(f"DBSCAN Clusters Found:   {len(set(dbscan.labels_)) - (1 if -1 in dbscan.labels_
    else 0)}")
```

3.3 Self-supervised learning: data self-annotation

While supervised learning requires manual target labels $(\mathbf{x}_i, y_i)$ and unsupervised learning discovers unlabelled structure without targets, **Self-Supervised Learning (SSL)** bridges this divide by turning unlabelled data into its own supervisor through **data self-annotation**.

The Paradigm Shift: From Manual to Self-Annotation. Manual human annotation is expensive, slow, bottlenecked by labor, and subject to annotator bias. Self-supervised learning eliminates manual labeling by partitioning or transforming unlabelled raw inputs $\mathbf{x}$ so that the data automatically generates both context features $\mathbf{x}_{\text{context}}$ and target labels y_{target} without human intervention.

Mechanisms of Data Self-Annotation (Pretext Tasks). The self-annotation process relies on formulating *pretext tasks* directly on unlabelled corpora:

- **Masked / Predictive Self-Annotation:** The system hides or masks a portion of the input vector or sequence, defining the hidden portion as the ground-truth target $y_{\text{target}} = \mathbf{x}_{\text{masked}}$. For text, masking tokens in a sentence (e.g., BERT (Devlin et al. 2019)) forces the network to annotate and predict missing words from surrounding context. In computer vision, Masked Autoencoders (MAE (He et al. 2022)) mask image patches and use raw pixel values as targets.
- **Autoregressive Self-Annotation:** In sequence modeling, predicting the next token x_{t+1} given preceding context $x_{1:t}$ self-annotates every token position. A text corpus of N tokens automatically yields $N-1$ labeled training pairs $(x_{1:t}, x_{t+1})$ with zero manual annotation cost.
- **Contrastive / Transformation Self-Annotation:** Applying spatial or chromatic augmentations (cropping, rotation, color jittering) to an image $\mathbf{x}_i$ creates two views $(\mathbf{x}_i^{(1)}, \mathbf{x}_i^{(2)})$. The framework automatically self-annotates this pair as a positive instance ($y = 1$), while pairing $\mathbf{x}_i^{(1)}$ with a different image sample $\mathbf{x}_j$ creates a negative instance ($y = 0$) (Chen et al. 2020).
- **Context Restoration & Spatial Permutations:** Shuffling image patches or introducing synthetic noise self-annotates the original spatial layout or uncorrupted signal as the target label.

Self-Supervised Learning: Data Self-Annotation

Self-Supervised Learning converts unlabelled raw data into supervised training signals by having the data annotate itself. By masking, splitting, or transforming inputs, SSL algorithms create pretext target labels y_{target} automatically, unlocking web-scale pre-training for foundation models.

Data Self-Annotation in Python: Masked & Autoregressive Pretext Tasks

```python
import numpy as np

# Raw unlabelled sequence (e.g., token IDs or sensor measurements)
raw_sequence = np.array([12, 45, 78, 23, 56, 89, 34, 67, 90, 11])

# 1. Autoregressive Self-Annotation: x_{1:t} -> target y = x_{t+1}
def autoregressive_self_annotation(seq):
    contexts = [seq[:t] for t in range(1, len(seq))]
    targets  = seq[1:]
    return contexts, targets

# 2. Masked Self-Annotation: mask 20% of tokens; target = original values
def masked_self_annotation(seq, mask_prob=0.2, mask_val=0, seed=42):
    rng = np.random.default_rng(seed)
    mask = rng.random(len(seq)) < mask_prob
    masked_seq = seq.copy()
    masked_seq[mask] = mask_val
    targets = np.where(mask, seq, -100) # -100 ignored in loss evaluation
    return masked_seq, targets

contexts_ar, targets_ar = autoregressive_self_annotation(raw_sequence)
seq_m, targets_m = masked_self_annotation(raw_sequence)

print("Raw Sequence:         ", raw_sequence)
print("Autoregressive Step: ", f"Context {contexts_ar[2]} -> Target {targets_ar[2]}")
print("Masked Input:         ", seq_m)
print("Self-Annotated Labels:", targets_m)
```

Data self-annotation generates supervised training pairs directly from raw unlabelled input arrays.

3.4 Reinforcement learning

While supervised learning requires target pairs $(\mathbf{x}_i, y_i)$ and unsupervised learning discovers unlabelled structure, **Reinforcement Learning (RL)** optimizes sequential decision-making through trial-and-error environment interaction (Sutton and Barto 2018). An agent observes state $s_t \in \mathcal{S}$, executes action $a_t \in \mathcal{A}$ under policy $\pi(a \mid s)$, transitions to s_{t+1}, and receives reward r_{t+1}.

MDPs & Bellman Equations. Environments obey the Markov property: $P(S_{t+1} = s', R_{t+1} = r \mid S_t, A_t, \dots) = P(S_{t+1} = s', R_{t+1} = r \mid S_t, A_t)$. An MDP $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$ defines state/action spaces, transition dynamics $\mathcal{P}(s' \mid s, a)$, reward function $\mathcal{R}$, and discount factor $\gamma \in [0, 1)$. Value functions $V^\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$ and $Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]$ satisfy the **Bellman Optimality Equation**:

$$Q^\star(s, a) = \sum_{s' \in \mathcal{S}} \mathcal{P}(s' \mid s, a) \left[\mathcal{R}(s, a, s') + \gamma \max_{a' \in \mathcal{A}} Q^\star(s', a') \right]. \tag{38}$$

Temporal Difference (TD) Learning, SARSA, & Q-Learning. Model-free RL learns directly from interaction:

- **Monte Carlo vs. TD Learning:** Monte Carlo updates values after full episode completion, whereas TD learning updates predictions incrementally from immediate reward plus estimated next-state value (bootstrapping): $V(s_t) \leftarrow V(s_t) + \alpha[r_{t+1} + \gamma V(s_{t+1}) - V(s_t)]$.
- **SARSA (On-Policy) vs. Q-Learning (Off-Policy):** SARSA updates Q-values using the actual next action executed by the policy: $Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)]$. Off-policy Q-learning (Watkins and Dayan 1992) updates using the maximum possible next-state action value $\max_a Q(s_{t+1}, a)$.

Actor-Critic Methods & Advantage Functions. To reduce policy gradient variance, **Actor-Critic** architectures maintain an *Actor* (policy $\pi_{\boldsymbol{\theta}}(a \mid s)$) and a *Critic* (value function $V_{\boldsymbol{\phi}}(s)$). The update scales gradients by the **Advantage function**:

$$A(s, a) = Q(s, a) - V(s) \;\approx\; r_{t+1} + \gamma V(s_{t+1}) - V(s_t), \tag{39}$$

quantifying how much better action a is compared to the average expected value of state s. Modern algorithms like Proximal Policy Optimization (PPO) (Schulman et al. 2017) clip policy updates to prevent destabilizing gradient steps, serving as the core engine for RLHF (Module 4).

Clarification on AlphaGo & MCTS. Note that Monte Carlo Tree Search (MCTS) in AlphaGo (Silver et al. 2016) performs lookahead search over a perfect game simulator, combining heuristic rollouts with neural evaluation. This lookahead search over a simulator is distinct from model-based RL algorithms that must explicitly learn an empirical transition probability model $\hat{\mathcal{P}}(s' \mid s, a)$ from raw environment data.

3.5 Modern evaluation methodology & diagnostics

Evaluating a model requires moving beyond simple train/test accuracy.

Classification Metrics: Confusion Matrix, PR-AUC, and ROC-AUC. Precision $\frac{TP}{TP+FP}$, Recall $\frac{TP}{TP+FN}$, $F_1 = 2\frac{P \cdot R}{P+R}$, and ROC-AUC measure classification performance. For severely imbalanced datasets (e.g., fraud, rare diseases), ROC-AUC can present an overly optimistic score because false positives are divided by a vast number of true negatives (TN). The **Precision-Recall Curve (PR-AUC)** is strongly preferred for imbalanced evaluation because it focuses exclusively on the positive class.

Probabilistic Calibration & Proper Scoring Rules. Model probabilities $\hat{p} \in [0, 1]$ should match real-world frequencies: among events assigned $\hat{p} = 0.8$, exactly 80% should occur. **Calibration curves** (reliability diagrams) plot predicted probability bins against empirical frequencies. Quantitative calibration error is measured by the **Brier Score** $\frac{1}{m}\sum(\hat{y}_i - y_i)^2$ and Categorical Cross-Entropy (Log-Loss), both of which are *proper scoring rules* minimized strictly when predicted probabilities equal true posterior distributions.

Advanced Resampling & Data Drift. Standard k-fold cross-validation fails on structured data dependencies:

- **Nested Cross-Validation:** Outer folds evaluate generalization error while inner folds perform hyperparameter optimization, preventing optimistic hyperparameter selection bias.
- **Time-Series & Grouped CV:** Time-dependent data requires rolling-window splits ($t_{\text{train}} < t_{\text{test}}$) to prevent temporal leakage; grouped data (multiple samples per user) requires `GroupKFold` to prevent user leakage across splits.
- **Data Drift & Concept Drift:** In production, input feature distributions change over time (*covariate shift* $P(X)$ changes) or target relationships alter (*concept drift* $P(Y \mid X)$ changes), requiring monitoring statistics (Kolmogorov-Smirnov tests, Population Stability Index) and scheduled model retraining.

Check your understanding

1. For a cancer-screening tool, which metric would you prioritise, and what is the cost of that choice?
2. Why does K-Means struggle with two concentric rings of points, and what does that reveal about its assumptions?
3. In a K-armed bandit problem, how does increasing ϵ in an ϵ-greedy strategy impact short-term vs. long-term return?
4. Your cross-validation scores vary from 0.62 to 0.91 across folds. What does that variance tell you, and what would you do next?
5. Explain how Self-Supervised Learning (SSL) turns unlabelled data into supervised targets. Give one concrete example of a self-annotation pretext task.

Sources & further reading

- T. M. Mitchell (1997). *Machine Learning*. New York: McGraw-Hill
- T. Hastie et al. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2nd ed. New York: Springer
- C. M. Bishop (2006). *Pattern Recognition and Machine Learning*. New York: Springer
- C. Cortes and V. Vapnik (1995). "Support-Vector Networks". In: *Machine Learning* 20.3, pp. 273–297
- L. Breiman (2001). "Random Forests". In: *Machine Learning* 45.1, pp. 5–32
- T. Chen and C. Guestrin (2016). "XGBoost: A Scalable Tree Boosting System". In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794
- J. MacQueen (1967). "Some Methods for Classification and Analysis of Multivariate Observations". In: *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*. Vol. 1, pp. 281–297
- F. T. Liu et al. (2008). "Isolation Forest". In: *Proceedings of the 8th IEEE International Conference on Data Mining (ICDM)*, pp. 413–422
- R. S. Sutton and A. G. Barto (2018). *Reinforcement Learning: An Introduction*. 2nd ed. Cambridge, MA: MIT Press
- C. J. C. H. Watkins and P. Dayan (1992). "Q-Learning". In: *Machine Learning* 8.3-4, pp. 279–292
- V. Mnih et al. (2015). "Human-level Control Through Deep Reinforcement Learning". In: *Nature* 518.7540, pp. 529–533
- J. Schulman et al. (2017). "Proximal Policy Optimization Algorithms". In
- D. Silver et al. (2016). "Mastering the Game of Go with Deep Neural Networks and Tree Search". In: *Nature* 529.7587, pp. 484–489
- D. Amodei et al. (2016). "Concrete Problems in AI Safety". In: *arXiv preprint arXiv:1606.06565*
- T. Fawcett (2006). "An Introduction to ROC Analysis". In: *Pattern Recognition Letters* 27.8, pp. 861–874
- F. Pedregosa et al. (2011). "Scikit-learn: Machine Learning in Python". In: *Journal of Machine Learning Research* 12, pp. 2825–2830

4 Transition to Deep Learning and Modern AI

Module 4, at a glance (Estimated Study: 8 hours • Level: Intermediate–Advanced)

Focus. The leap from engineered features to learned representation hierarchies: perceptrons, activation functions, backpropagation, regularization & normalization, autoencoders, latent spaces, CNNs, RNNs/LSTMs, and self-attention in Transformers.

Key concepts. Perceptrons, backpropagation, activation functions, vanishing gradients, Dropout, Batch/Layer Normalization, autoencoders, latent spaces, Word2Vec, CNNs, translation equivariance, RNNs, LSTMs, Scaled Dot-Product Attention, and Multi-Head Attention.

After this module you will be able to explain how neural networks learn continuous latent representations, derive backpropagation matrix calculus, contrast normalization techniques, and build Multi-Head Attention mechanisms from scratch.

The jump from classical ML to deep learning is a jump in *who designs the features*. A classical pipeline is given hand-crafted inputs; a deep network learns a hierarchy of them: edges, then textures, then parts, then objects (LeCun et al. 2015). What engineering achieved by hand, depth achieves with data and compute.

4.1 Neural networks: perceptrons, activations, backpropagation

The **perceptron** (Rosenblatt 1958) is a single neuron: a weighted sum passed through a threshold,

$$\hat{y} = \mathbb{1}\{\mathbf{w}^\top \mathbf{x} + b > 0\} \in \{0, 1\} \quad (\text{or } \operatorname{sgn}(\mathbf{w}^\top \mathbf{x} + b) \in \{-1, +1\}). \tag{40}$$

It can learn any function that a straight line can separate, and provably cannot learn XOR, a limitation that chilled the field's first winter (Minsky and Papert 1969). The repair is to *stack* units and interleave **nonlinear activation functions** between layers:

$$\mathbf{a}^{(\ell)} = \phi\left(W^{(\ell)}\,\mathbf{a}^{(\ell-1)} + \mathbf{b}^{(\ell)}\right), \qquad \ell = 1, \dots, L. \tag{41}$$

Without ϕ, the stack would collapse into a single linear map (Strang 2012). The rectified linear unit, $\phi(z) = \max(0, z)$, became the default for being cheap, non-saturating, and sparse in its activations (Nair and Hinton 2010). Such networks are universal approximators (Hornik et al. 1989), a statement about *capacity*, with no guarantee that training finds good parameters.

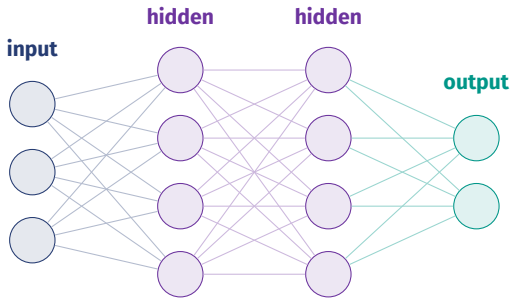


Figure 5. A fully connected 3–4–4–2 network. Every edge carries a learnable weight; each layer applies its matrix, then a nonlinearity.

Backpropagation (Rumelhart et al. 1986) computes the gradient of the loss $\mathcal{L}$ with respect to every weight (the chain rule from Module 1) organised backwards through the layers. With

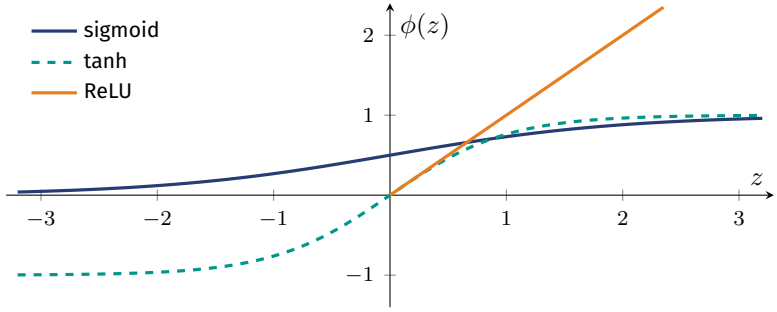


Figure 6. Common activation functions. ReLU's lack of saturation for $z > 0$ helped make deep networks trainable at scale (Nair and Hinton 2010).

$\delta^{(\ell)} \equiv \partial \mathcal{L} / \partial \mathbf{z}^{(\ell)}$:

$$\boldsymbol{\delta}^{(L)} = \nabla_{\mathbf{a}^{(L)}} \mathcal{L} \odot \phi'\big(\mathbf{z}^{(L)}\big), \tag{42}$$

$$\boldsymbol{\delta}^{(\ell)} = \big(W^{(\ell+1)\top} \boldsymbol{\delta}^{(\ell+1)}\big) \odot \phi'\big(\mathbf{z}^{(\ell)}\big), \qquad \ell = L-1, \dots, 1, \tag{43}$$

$$\frac{\partial \mathcal{L}}{\partial W^{(\ell)}} = \boldsymbol{\delta}^{(\ell)} \, \mathbf{a}^{(\ell-1)\top}, \tag{44}$$

where $\odot$ is element-wise multiplication. Backprop is a gradient-*computation* algorithm; the learning itself is still gradient descent (equation (20)), now run on mini-batches through frameworks such as PyTorch or JAX.

5 Network Architecture and Forward Pass

Consider a standard feedforward artificial neural network with L layers. We denote the input layer as layer 0 and the output layer as layer L. Let N_l be the number of neurons in layer l.

For a specific layer l (where $1 \leq l \leq L$), the forward propagation equations are:

$$z_j^{(l)} = \sum_{k=1}^{N_{l-1}} W_{jk}^{(l)} a_k^{(l-1)} + b_j^{(l)} \tag{45}$$

$$a_j^{(l)} = \sigma^{(l)}\left(z_j^{(l)}\right) \tag{46}$$

where:

- $W_{jk}^{(l)}$ is the weight connecting neuron k in layer $l-1$ to neuron j in layer l.
- $b_j^{(l)}$ is the bias of neuron j in layer l.
- $z_j^{(l)}$ is the weighted input (pre-activation) to neuron j in layer l.
- $a_j^{(l)}$ is the activation output of neuron j in layer l. $a_k^{(0)}$ represents the raw input features.
- $\sigma^{(l)}$ is the non-linear activation function for layer l (e.g., ReLU, Sigmoid, Tanh).

6 The Loss Function and the Goal

Let $\mathcal{L}$ be the scalar loss function that compares the network's final output $a^{(L)}$ to the true target values y. Common choices include Mean Squared Error (MSE) or Cross-Entropy.

Our goal is to minimize $\mathcal{L}$ by adjusting the parameters $W_{jk}^{(l)}$ and $b_j^{(l)}$. To do this using gradient descent, we must compute the partial derivatives:

$$\frac{\partial \mathcal{L}}{\partial W_{jk}^{(l)}} \quad \text{and} \quad \frac{\partial \mathcal{L}}{\partial b_j^{(l)}} \tag{47}$$

7 The Error Term (δ)

To simplify the application of the chain rule, we define an intermediate “error term" for each neuron, denoted as $\delta_j^{(l)}$. This term represents the sensitivity of the overall loss to changes in the pre-activation $z_j^{(l)}$:

$$\delta_j^{(l)} = \frac{\partial \mathcal{L}}{\partial z_j^{(l)}} \tag{48}$$

By computing this error term for every neuron in the network, we can easily find the gradients for the weights and biases.

8 Deriving Gradients using the Chain Rule

The chain rule states that if a variable y depends on u, and u depends on x, then $\frac{\partial y}{\partial x} = \frac{\partial y}{\partial u}\frac{\partial u}{\partial x}$. We apply this to our network variables.

8.1 Gradient with respect to Weights

We want to find how the loss changes as we vary a specific weight $W_{jk}^{(l)}$. Notice that $W_{jk}^{(l)}$ only affects the network through $z_j^{(l)}$. Applying the chain rule:

$$\frac{\partial \mathcal{L}}{\partial W_{jk}^{(l)}} = \frac{\partial \mathcal{L}}{\partial z_j^{(l)}} \frac{\partial z_j^{(l)}}{\partial W_{jk}^{(l)}} \tag{49}$$

From our definition, the first term is exactly $\delta_j^{(l)}$. For the second term, differentiating equation (1) with respect to $W_{jk}^{(l)}$ yields:

$$\frac{\partial z_j^{(l)}}{\partial W_{jk}^{(l)}} = a_k^{(l-1)} \tag{50}$$

Therefore, the weight gradient is:

$$\frac{\partial \mathcal{L}}{\partial W_{jk}^{(l)}} = \delta_j^{(l)} a_k^{(l-1)} \tag{51}$$

8.2 Gradient with respect to Biases

Similarly, $b_j^{(l)}$ only influences the loss through $z_j^{(l)}$:

$$\frac{\partial \mathcal{L}}{\partial b_j^{(l)}} = \frac{\partial \mathcal{L}}{\partial z_j^{(l)}} \frac{\partial z_j^{(l)}}{\partial b_j^{(l)}} \tag{52}$$

Differentiating equation (1) with respect to $b_j^{(l)}$ gives 1. Thus, the bias gradient is simply:

$$\frac{\partial \mathcal{L}}{\partial b_j^{(l)}} = \delta_j^{(l)} \tag{53}$$

9 Backpropagating the Error

Equations (6) and (8) show that if we know $\delta_j^{(l)}$, computing the parameter gradients is trivial. We now derive how to compute $\delta_j^{(l)}$ recursively, starting from the output layer and moving backwards to the first layer.

9.1 Step 1: Error at the Output Layer ($l = L$)

By definition and the chain rule:

$$\delta_j^{(L)} = \frac{\partial \mathcal{L}}{\partial z_j^{(L)}} = \frac{\partial \mathcal{L}}{\partial a_j^{(L)}} \frac{\partial a_j^{(L)}}{\partial z_j^{(L)}} \tag{54}$$

Using equation (2), the derivative of the activation function is $\sigma^{(L)\prime}(z_j^{(L)})$. This yields the output error equation:

$$\delta_j^{(L)} = \frac{\partial \mathcal{L}}{\partial a_j^{(L)}} \odot \sigma^{(L)\prime}(z_j^{(L)}) \tag{55}$$

9.2 Step 2: Error at Hidden Layers ($l < L$)

The propagation of errors backward through neural network layers relies on matrix multiplication by the transposed weight matrix $(W^{(l)})^\top$, expressing linear transformations between vector spaces (Strang 2012). To find $\delta_k^{(l-1)}$ for a neuron k in layer $l-1$, we must look at how its pre-activation $z_k^{(l-1)}$ affects the loss. The output of this neuron, $a_k^{(l-1)}$, acts as an input to *every* neuron j in the subsequent layer l.

By the multivariable chain rule, we must sum the contributions over all neurons j in layer l:

$$\delta_k^{(l-1)} = \frac{\partial \mathcal{L}}{\partial z_k^{(l-1)}} = \sum_{j=1}^{N_l} \frac{\partial \mathcal{L}}{\partial z_j^{(l)}} \frac{\partial z_j^{(l)}}{\partial z_k^{(l-1)}} \tag{56}$$

Substitute $\delta_j^{(l)} = \frac{\partial \mathcal{L}}{\partial z_j^{(l)}}$. Now we need the derivative of $z_j^{(l)}$ with respect to $z_k^{(l-1)}$.

$$z_j^{(l)} = \sum_{k=1}^{N_{l-1}} W_{jk}^{(l)} \sigma^{(l-1)}(z_k^{(l-1)}) + b_j^{(l)} \tag{57}$$

Differentiating this gives:

$$\frac{\partial z_j^{(l)}}{\partial z_k^{(l-1)}} = W_{jk}^{(l)} \sigma^{(l-1)\prime}(z_k^{(l-1)}) \tag{58}$$

Substituting this back into our multivariable chain rule expression:

$$\delta_k^{(l-1)} = \sum_{j=1}^{N_l} \delta_j^{(l)} W_{jk}^{(l)} \sigma^{(l-1)\prime}(z_k^{(l-1)}) \tag{59}$$

This is the recursive step of backpropagation: it allows us to calculate the error at layer $l-1$ using the weights W and the error δ from layer l.

10 Summary of the Backpropagation Algorithm

In modern deep learning frameworks, these operations are heavily vectorized. Let $\mathbf{W}^{(l)}$ be the weight matrix, $\mathbf{b}^{(l)}$ be the bias vector, and $\odot$ denote element-wise multiplication. The four fundamental equations of backpropagation are:

1. Output Layer Error:

$$\delta^{(L)} = \nabla_{\mathbf{a}^{(L)}} \mathcal{L} \odot \sigma^{(L)'}(\mathbf{z}^{(L)}) \tag{60}$$

2. Hidden Layer Error (Backpropagation Step):

$$\delta^{(l)} = \left((\mathbf{W}^{(l+1)})^T \delta^{(l+1)}\right) \odot \sigma^{(l)'}(\mathbf{z}^{(l)}) \tag{61}$$

3. Gradient with respect to Biases:

$$\nabla_{\mathbf{b}^{(l)}} \mathcal{L} = \delta^{(l)} \tag{62}$$

4. Gradient with respect to Weights:

$$\nabla_{\mathbf{W}^{(l)}} \mathcal{L} = \delta^{(l)} (\mathbf{a}^{(l-1)})^T \tag{63}$$

Once these gradients are computed, the parameters are updated using gradient descent (or variants like Adam) with a learning rate η:

$$\mathbf{W}^{(l)} \leftarrow \mathbf{W}^{(l)} - \eta \nabla_{\mathbf{W}^{(l)}} \mathcal{L} \tag{64}$$

$$\mathbf{b}^{(l)} \leftarrow \mathbf{b}^{(l)} - \eta \nabla_{\mathbf{b}^{(l)}} \mathcal{L} \tag{65}$$

Vanishing and exploding gradients. In deep multi-layer architectures, backpropagating loss gradients across L layers involves repeated matrix products backwards:

$$\boldsymbol{\delta}^{(1)} = \left(\prod_{\ell=2}^{L} W^{(\ell)\top} \operatorname{diag}\Big(\phi'\Big(\mathbf{z}^{(\ell)}\Big)\Big) \right) \boldsymbol{\delta}^{(L)}. \tag{66}$$

The long-term behavior of this product is governed by the **spectral radius** of the effective Jacobian at each layer. Define the per-layer Jacobian $J^{(\ell)} \equiv W^{(\ell)\top} \operatorname{diag}\big(\phi'(\mathbf{z}^{(\ell)})\big)$, so that equation (66) becomes $\boldsymbol{\delta}^{(1)} = J^{(2)} J^{(3)} \cdots J^{(L)}\, \boldsymbol{\delta}^{(L)}$. The spectral radius $\rho(A) \equiv \max_i |\lambda_i(A)|$ of a square matrix A is the magnitude of its largest eigenvalue, and by the Gelfand formula it determines the asymptotic growth rate of iterated products:

$$\left\|A^n\right\|^{1/n} \xrightarrow{n\to\infty} \rho(A). \tag{67}$$

Applying this to the product of Jacobians, when all $J^{(\ell)}$ are similar and $\rho(J^{(\ell)}) < 1$ uniformly, the gradient norm decays exponentially in the depth L:

$$\left\|\boldsymbol{\delta}^{(1)}\right\| \lesssim \rho^{L-1} \left\|\boldsymbol{\delta}^{(L)}\right\| \xrightarrow{L\to\infty} 0 \qquad (\textit{vanishing gradients}). \tag{68}$$

Conversely, when $\rho(J^{(\ell)}) > 1$ the norm grows exponentially, yielding numerical overflow or pathological parameter updates (*exploding gradients*). The critical regime $\rho(J^{(\ell)}) \approx 1$—the **edge of stability**—is where information can propagate across many layers without attenuation.

Activation saturation and the sigmoid/tanh problem. The spectral radius analysis makes the failure of sigmoidal activations transparent. For $\phi(z) = \sigma(z) = (1 + e^{-z})^{-1}$, the derivative $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ saturates to zero for large $|z|$, with a maximum of 0.25 at $z = 0$. Since the effective Jacobian factors in $\mathrm{diag}(\phi')$, saturated neurons multiplicatively suppress the gradient by up to 75% per layer. With $L = 20$ layers and mean slope 0.25, the gradient magnitude scales as $0.25^{20} \approx 10^{-12}$—numerically zero. The **Rectified Linear Unit** (ReLU, $\phi(z) = \max(0, z)$) eliminates saturation for positive pre-activations, since $\phi'(z) = 1$ for $z > 0$, keeping the corresponding diagonal entries at unity and preventing activation-driven spectral collapse (Glorot and Bengio 2010).

Vanishing gradients in recurrent networks. The problem is particularly acute in **Recurrent Neural Networks (RNNs)**, where the same weight matrix W_h is shared across all time steps. Unrolling a sequence of T steps through time (Backpropagation Through Time, BPTT) yields a product of T copies of $W_h^\top \mathrm{diag}(\phi'(\mathbf{h}_t))$, so the spectral radius condition directly governs the maximum useful sequence length (Bengio et al. 1994). If $\rho(W_h) < 1$, the network cannot propagate credit over more than $\mathcal{O}(1/(1-\rho))$ steps; if $\rho(W_h) > 1$, training diverges. LSTMs and GRUs circumvent this by introducing *additive* state updates through gated paths, maintaining a near-identity Jacobian along the gradient highway.

Spectral radius and orthogonal initialization. The most principled remedy at initialization is to set each weight matrix $W^{(\ell)}$ to be **orthogonal**, satisfying $W^{(\ell)\top} W^{(\ell)} = I$. Orthogonal matrices have all singular values equal to 1, so $\rho(W^{(\ell)}) = 1$ exactly, and the isometric property $\|W^{(\ell)}\mathbf{v}\| = \|\mathbf{v}\|$ for all $\mathbf{v}$ ensures that gradient norms are preserved perfectly as they pass through linear layers:

$$\left\|W^{(\ell)\top}\boldsymbol{\delta}^{(\ell)}\right\| = \left\|\boldsymbol{\delta}^{(\ell)}\right\|. \tag{69}$$

Saxe et al. (2014) demonstrated that deep linear networks initialized with random orthogonal matrices learn in a qualitatively different regime than Gaussian-initialized networks: orthogonal initialization places the system precisely at the boundary of gradient stability, allowing gradient signals to propagate across arbitrary depth at the start of training. In practice, orthogonal initialization is generated via the QR decomposition of a random Gaussian matrix: if $G \sim \mathcal{N}(0,1)^{n\times m}$ and $G = QR$ is its QR decomposition, then Q is orthogonal and is used as the initial weight matrix.

For nonlinear networks, the activation derivative ϕ' perturbs the singular values of the effective Jacobian away from 1. With ReLU activations approximately half the units are active, so the effective gain is ≈ 0.5; to compensate, He et al. (2015) derived the **He (Kaiming) initialization** variance $\sigma^2 = 2/n_{\text{in}}$, which keeps the pre-activation variances stable across layers. Glorot (Xavier) initialization (Glorot and Bengio 2010) targets the geometric mean $\sigma^2 = 2/(n_{\text{in}} + n_{\text{out}})$ and is standard for sigmoid/tanh activations.

Spectral Radius and Gradient Flow Stability

The spectral radius $\rho(J^{(\ell)})$ of the per-layer Jacobian is the fundamental quantity controlling gradient flow. When $\rho < 1$ uniformly, gradients vanish exponentially with depth; when $\rho > 1$, they explode. Orthogonal initialization targets $\rho = 1$ exactly at the start of training, preserving gradient norms across all layers. In modern deep architectures, four complementary strategies together ensure stable gradient propagation: (1) variance-preserving initialization (He or Glorot); (2) non-saturating activations (ReLU and its variants); (3) residual (skip) connections that introduce an unattenuated identity gradient path; and (4) normalization layers (Batch Norm or Layer Norm) that re-center and rescale activations to

prevent spectral drift during training. Exploding gradients are additionally controlled by **gradient clipping**: rescaling $\boldsymbol{g} \leftarrow \boldsymbol{g}\,\tau / \max(\tau, \|\boldsymbol{g}\|)$ whenever the global gradient norm exceeds threshold τ.

Regularization and normalization in deep networks. Beyond basic weight decay, deep network training relies on specialized techniques to prevent overfitting and stabilize activation distributions across layers:

Technique	Mechanism & Scope	Primary Benefit & Use Case
Dropout	Randomly zeroes activations with probability p during training	Prevents co-adaptation; acts as an ensemble over sub-networks
Batch Norm	Normalizes activations across the mini-batch dimension	Reduces internal covariate shift in CNNs; sensitive to batch size
Layer Norm	Normalizes activations across feature channels per sample	Essential for Transformers and RNNs; independent of batch size
Weight Decay vs ℓ_2	Decouples coefficient penalty from adaptive moments (AdamW)	Prevents gradient scaling from eroding regularisation in LLMs

Table 5. Comparison of regularization and normalization mechanisms in modern deep architectures.

Two-Layer MLP with Manual Backpropagation on Non-Linear Data

```python
import numpy as np

# Non-linear XOR / concentric circle dataset
rng = np.random.default_rng(42)
X_b = rng.uniform(-2, 2, size=(200, 2))
y_b = ((X_b[:, 0]**2 + X_b[:, 1]**2) < 1.2).astype(int).reshape(-1, 1)

# Initialize weights with He Normalization (scaling by sqrt(2/fan_in))
W1 = rng.normal(0, np.sqrt(2/2), size=(2, 8)); b1 = np.zeros((1, 8))
W2 = rng.normal(0, np.sqrt(2/8), size=(8, 1)); b2 = np.zeros((1, 1))

for epoch in range(500):
    # Forward pass: Input -> Hidden (ReLU) -> Output (Sigmoid)
    z1 = X_b @ W1 + b1; a1 = np.maximum(0, z1)   # ReLU activation
    z2 = a1 @ W2 + b2;  hat_y = 1 / (1 + np.exp(-np.clip(z2, -500, 500)))

    # Backward pass (Gradient computation via chain rule)
    dz2 = (hat_y - y_b) / len(X_b)               # dL/dz2 for BCE loss
    dW2 = a1.T @ dz2; db2 = dz2.sum(axis=0, keepdims=True)
    da1 = dz2 @ W2.T;  dz1 = da1 * (z1 > 0)      # ReLU derivative
    dW1 = X_b.T @ dz1; db1 = dz1.sum(axis=0, keepdims=True)

    # Gradient descent parameter updates
    W1 -= 0.1 * dW1; b1 -= 0.1 * db1
    W2 -= 0.1 * dW2; b2 -= 0.1 * db2

bce_loss = -np.mean(y_b*np.log(hat_y+1e-7) + (1-y_b)*np.log(1-hat_y+1e-7))
acc = np.mean((hat_y > 0.5) == y_b)
print(f"Epoch 500 | BCE Loss: {bce_loss:.4f} | Accuracy: {acc:.3f}")
```

10.1 Representation learning: from manual engineering to latent spaces

In classical machine learning (Module 3), algorithm accuracy hinges on manual feature engineering, crafting input matrix columns $\mathbf{x} \in \mathbb{R}^n$ using domain heuristics (e.g., SIFT descriptors in vision, TF-IDF in NLP, polynomial features). Manual feature engineering is laborious, task-specific, fragile, and fails to scale to high-dimensional unstructured inputs like pixels or continuous audio waveforms. **Representation learning** (Bengio et al. 2013) eliminates manual feature engineering: neural networks discover parameterized continuous transformations $\mathbf{z} = f_{\boldsymbol{\theta}}(\mathbf{x}) \in \mathbb{R}^d$ that automatically extract latent semantic structures and data-generating invariants.

The Manifold Hypothesis: linear vs. non-linear projections. High-dimensional real-world data vectors $\mathbf{x} \in \mathbb{R}^n$ (such as $256 \times 256 \times 3$ RGB images in $\mathbb{R}^{196,608}$) occupy an infinitesimal fraction of ambient space. The **Manifold Hypothesis** posits that real-world data concentrates near a smooth, low-dimensional non-linear manifold $\mathcal{M} \subset \mathbb{R}^n$ of intrinsic dimension $d \ll n$. Linear dimensionality reduction (PCA, Module 2) computes orthogonal projections $\mathbf{z} = W^\top \mathbf{x}$, which capture global variance but fail to unfold non-linear manifolds (e.g., Swiss Roll geometry). Interleaving non-linear activation functions across hidden layers allows deep networks to learn non-linear coordinate charts, flattening complex curved manifolds in input space into linearly separable representations in latent space $\mathcal{Z}$.

> **The Core Principle of Representation Learning**
>
> Instead of hand-crafting task-specific features, representation learning trains neural network backbones to transform raw inputs $\mathbf{x}$ into continuous vector representations $\mathbf{z} \in \mathbb{R}^d$. Good representations disentangle underlying variation factors, compress noise, and make downstream tasks (classification, search, generation) trivial to solve using linear layers or distance metrics.

Autoencoders and Latent Space Geometry. An **Autoencoder** is a neural network trained to reconstruct its input through an architectural bottleneck:

$$\text{Encoder:} \quad \mathbf{z} = g_{\boldsymbol{\phi}}(\mathbf{x}) \in \mathbb{R}^d, \qquad \text{Decoder:} \quad \hat{\mathbf{x}} = f_{\boldsymbol{\theta}}(\mathbf{z}) \in \mathbb{R}^n. \tag{70}$$

Minimizing reconstruction loss $\mathcal{L}_{\text{rec}}(\mathbf{x}, \hat{\mathbf{x}}) = \|\mathbf{x} - \hat{\mathbf{x}}\|_2^2$ (or Binary Cross-Entropy) forces the bottleneck $\mathbf{z}$ to discard uninformative noise while preserving dominant structural variations:

- **Undercomplete Autoencoders:** Enforcing bottleneck dimension $d < n$ prevents identity memorization. For linear encoders/decoders under MSE loss, the learned bottleneck subspace is equivalent to PCA.
- **Denoising Autoencoders (DAEs):** To prevent memorization when $d \geq n$, DAEs (Vincent et al. 2008) corrupt inputs $\tilde{\mathbf{x}} \sim q(\tilde{\mathbf{x}} \mid \mathbf{x})$ via noise injection or random masking, training the decoder to reconstruct clean $\mathbf{x}$. DAEs learn a score-matching vector field $\nabla_{\mathbf{x}} \ln p(\mathbf{x})$ that projects off-manifold points back onto data manifold $\mathcal{M}$.
- **Variational Autoencoders (VAEs):** A probabilistic formulation (Kingma and Welling 2014) where the encoder outputs parameters $\boldsymbol{\mu}(\mathbf{x})$ and $\boldsymbol{\sigma}^2(\mathbf{x})$ of a latent Gaussian distribution $q_{\boldsymbol{\phi}}(\mathbf{z} \mid \mathbf{x}) = \mathcal{N}(\boldsymbol{\mu}, \text{diag}(\boldsymbol{\sigma}^2))$. Training maximizes the **Evidence Lower Bound (ELBO)**:

$$\mathcal{L}_{\text{ELBO}}(\boldsymbol{\phi}, \boldsymbol{\theta}; \mathbf{x}) = \mathbb{E}_{q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})}\big[\ln p_{\boldsymbol{\theta}}(\mathbf{x} \mid \mathbf{z})\big] \; - \; D_{\text{KL}}(q_{\boldsymbol{\phi}}(\mathbf{z} \mid \mathbf{x}) \,||\, p(\mathbf{z}))\,, \tag{71}$$

  where $p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, I)$ and D_{KL} penalizes latent distributions that deviate from standard normal priors. Reparameterization $\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}$ ($\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, I)$) enables gradient backpropagation through stochastic sampling.

Distributed Representations & Word Embeddings. Symbolic NLP represents words as orthogonal one-hot vectors $\mathbf{e}_i \in \{0,1\}^{|V|}$, where dot products $\mathbf{e}_i^\top \mathbf{e}_j = 0$ for all $i \neq j$, failing to capture

semantic similarity. **Distributed representations** (Mikolov et al. 2013) embed discrete tokens into dense continuous spaces $\mathbb{R}^d$ ($d \ll |V|$), mapping semantic relationships directly to vector angles and distances.

The **Word2Vec Skip-Gram** architecture predicts context tokens w_{t+j} given target token w_t within window size c. To avoid computing full-vocabulary softmax denominators over $|V|$, **Negative Sampling (SGNS)** reformulates training as binary logistic regressions distinguishing true context tokens from K noise words sampled from unigram distribution $P_n(w)$:

$$\mathcal{L}_{\text{SGNS}} = -\sum_{t=1}^{T} \sum_{-c \le j \le c, j \neq 0} \left[\ln \sigma\left(\mathbf{v}_{w_{t+j}}^{\prime\top} \mathbf{v}_{w_t} \right) + \sum_{k=1}^{K} \mathbb{E}_{w_n \sim P_n(w)} \left[\ln \sigma\left(-\mathbf{v}_{w_n}^{\prime\top} \mathbf{v}_{w_t} \right) \right] \right], \tag{72}$$

where $\mathbf{v}_w$ and $\mathbf{v}'_w$ denote target and context vectors. An emergent property of SGNS spaces is **linear vector compositionality**: semantic analogies correspond to parallel displacement vectors ($\mathbf{v}_{\text{king}} - \mathbf{v}_{\text{man}} + \mathbf{v}_{\text{woman}} \approx \mathbf{v}_{\text{queen}}$).

Metric Learning & Triplet Loss. Representation learning can also optimize explicit distance metrics in embedding space. **Triplet Loss** trains encoders on anchor $\mathbf{a}$, positive $\mathbf{p}$ (same class), and negative $\mathbf{n}$ (different class) samples, enforcing that positive pairs remain closer than negative pairs by margin $\alpha > 0$:

$$\mathcal{L}_{\text{triplet}} = \max\Big(0,\ \|\mathbf{z}_a - \mathbf{z}_p\|_2^2 - \|\mathbf{z}_a - \mathbf{z}_n\|_2^2 + \alpha\Big). \tag{73}$$

Deep Autoencoder Feature Extraction & Latent Space Analysis

```
import numpy as np
from sklearn.linear_model import LogisticRegression

# 1. Generate non-linear 10D synthetic dataset with 2D intrinsic manifold
rng = np.random.default_rng(42)
t = rng.uniform(-np.pi, np.pi, size=600)
# Non-linear 2D manifold embedded in 10D ambient space
X_raw = np.column_stack([np.sin(t), np.cos(t), t**2, np.sin(2*t), np.cos(2*t),
                         t, np.exp(t/5), np.sin(3*t), np.cos(3*t), t**3/10])
y_labels = (np.sin(t) + np.cos(2*t) > 0).astype(int)

# 2. Define 10D -> 4D -> 2D -> 4D -> 10D Deep Autoencoder in NumPy
W_e1 = rng.normal(0, np.sqrt(2/10), size=(10, 4)); b_e1 = np.zeros((1, 4))
W_e2 = rng.normal(0, np.sqrt(2/4), size=(4, 2));   b_e2 = np.zeros((1, 2))  # Bottleneck z
    in R^2
W_d1 = rng.normal(0, np.sqrt(2/2), size=(2, 4));   b_d1 = np.zeros((1, 4))
W_d2 = rng.normal(0, np.sqrt(2/4), size=(4, 10));  b_d2 = np.zeros((1, 10))

# 3. Train Autoencoder via SGD with MSE Reconstruction Loss
lr = 0.01
for epoch in range(400):
    # Forward Pass
    h1 = np.maximum(0, X_raw @ W_e1 + b_e1)           # Encoder L1 (ReLU)
    z  = h1 @ W_e2 + b_e2                             # Bottleneck representation z in R^2
    h2 = np.maximum(0, z @ W_d1 + b_d1)               # Decoder L1 (ReLU)
    X_hat = h2 @ W_d2 + b_d2                          # Reconstructed output in R^10

    # Backward Pass (Chain rule gradients)
    dX_hat = 2 * (X_hat - X_raw) / len(X_raw)
    dW_d2 = h2.T @ dX_hat;         db_d2 = dX_hat.sum(axis=0, keepdims=True)
    dh2   = (dX_hat @ W_d2.T) * (h2 > 0)
    dW_d1 = z.T @ dh2;             db_d1 = dh2.sum(axis=0, keepdims=True)
    dz    = dh2 @ W_d1.T
    dW_e2 = h1.T @ dz;             db_e2 = dz.sum(axis=0, keepdims=True)
    dh1   = (dz @ W_e2.T) * (h1 > 0)
    dW_e1 = X_raw.T @ dh1;         db_e1 = dh1.sum(axis=0, keepdims=True)

    # Parameter Updates
    W_e1 -= lr * dW_e1; b_e1 -= lr * db_e1; W_e2 -= lr * dW_e2; b_e2 -= lr * db_e2
    W_d1 -= lr * dW_d1; b_d1 -= lr * db_d1; W_d2 -= lr * dW_d2; b_d2 -= lr * db_d2

# 4. Extract 2D latent representation z and evaluate linear separability
z_latent = np.maximum(0, X_raw[:450] @ W_e1 + b_e1) @ W_e2 + b_e2
z_test   = np.maximum(0, X_raw[450:] @ W_e1 + b_e1) @ W_e2 + b_e2

clf_raw = LogisticRegression().fit(X_raw[:450], y_labels[:450])
clf_z   = LogisticRegression().fit(z_latent, y_labels[:450])

print(f"Epoch 400 Reconstruction MSE: {np.mean((X_hat - X_raw)**2):.4f}")
print(f"Linear Classifier Acc (Raw 10D Features):   {clf_raw.score(X_raw[450:],
    y_labels[450:]):.3f}")
print(f"Linear Classifier Acc (Learned 2D Latent z): {clf_z.score(z_test,
    y_labels[450:]):.3f}")
```

10.2 Architectures: CNNs, RNNs, and transformers

Convolutional networks (CNNs) exploit spatial structure in grid data (images, signals) via local receptive fields and weight sharing. A convolution slides a small learnable filter K over input I:

$$(I * K)(i,j) = \sum_m \sum_n I(i-m,\, j-n)\, K(m,n). \tag{74}$$

Sliding the filter confers built-in **translation equivariance** ($(I*K)(x-\Delta) = ((I \text{ shifted}) * K)(x)$): shifting an input shifts its feature map identically. Spatial pooling and hierarchical aggregation then build approximate **translation invariance** at higher layers. LeNet established document recognition (LeCun et al. 1998); AlexNet's GPU depth ignited modern computer vision (Krizhevsky et al. 2012).

Recurrent networks (RNNs) handle sequences by carrying a hidden state:

$$\mathbf{h}_t = \tanh\big(W_h\, \mathbf{h}_{t-1} + W_x\, \mathbf{x}_t + \mathbf{b}\big). \tag{75}$$

Training across long sequences suffers from vanishing gradients; the **LSTM**'s gated memory cells provided the decisive fix (Hochreiter and Schmidhuber 1997), dominating sequential modeling for two decades.

Transformers replaced sequential recurrent state updates with parallel *self-attention*, positional encodings, and feed-forward transformations (Vaswani et al. 2017). Each token emits a query Q, a key K, and a value V; the output for each position is a weighted sum of all values, with weights given by query–key compatibility:

$$\text{Attention}(Q,K,V) = \text{softmax}\left(\frac{Q\,K^\top}{\sqrt{d_k}}\right) V. \tag{76}$$

Because every token attends to all others in a single parallel step, transformers train vastly faster than RNNs on modern parallel hardware. However, this global connectivity introduces two distinct quadratic bottlenecks. **Training memory:** the full $n \times n$ attention score matrix $A = \text{softmax}(QK^\top/\sqrt{d_k})$ must be materialised and retained for backpropagation, incurring $O(n^2)$ memory per layer per head. **Inference memory (KV-cache):** to avoid recomputing keys and values at every decoding step, the key and value projections $K, V \in \mathbb{R}^{n \times d_k}$ are cached across all layers; with L layers and h heads, the cache occupies $O(L \cdot h \cdot n \cdot d_k) = O(n \cdot d_{\text{model}})$ memory per token in the growing context, which becomes the dominant memory cost during long-sequence generation. Together these two bottlenecks explain why the effective context window of deployed LLMs is constrained by hardware VRAM, and have motivated approximations such as FlashAttention (which avoids materialising the full A matrix) and sliding-window or sparse attention patterns.

Scaled Dot-Product Attention from Scratch in NumPy

```python
import numpy as np

def scaled_dot_product_attention(Q, K, V, mask=None):
    d_k = Q.shape[-1]
    scores = (Q @ K.T) / np.sqrt(d_k)
    if mask is not None:
        scores = np.where(mask == 0, -1e9, scores)
    weights = np.exp(scores - np.max(scores, axis=-1, keepdims=True))
    weights /= weights.sum(axis=-1, keepdims=True)  # Softmax
    return weights @ V, weights

# Demonstrate with 4 tokens of dimension 8
seq_len, d_k = 4, 8
Q = K = V = np.random.default_rng(0).standard_normal((seq_len, d_k))
causal_mask = np.tril(np.ones((seq_len, seq_len)))  # Lower-triangular mask
out, attn = scaled_dot_product_attention(Q, K, V, mask=causal_mask)
print("Attention Output Shape:", out.shape)
print("Causal Attention Weights Matrix:\n", np.round(attn, 3))
```

Multi-Head Attention (MHA). Single-head attention averages relationships across the full representation space. To allow the network to jointly attend to information from different representation subspaces at different positions, **Multi-Head Attention** (Vaswani et al. 2017) projects queries, keys, and values h times with distinct learnable linear projections:

$$\mathrm{MultiHead}(Q, K, V) = \mathrm{Concat}(\mathrm{head}_1, \ldots, \mathrm{head}_h) W^O, \tag{77}$$

$$\text{where} \quad \mathrm{head}_i = \mathrm{Attention}\big(QW_i^Q,\ KW_i^K,\ VW_i^V\big), \tag{78}$$

with projection parameter matrices $W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$, and output projection $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$. In practice, $d_k = d_v = d_{\text{model}}/h$, keeping total computational cost comparable to single-head attention with full dimensionality.

Multi-Head Attention from Scratch in NumPy

```python
import numpy as np

def multi_head_attention(X, num_heads=4, d_model=64):
    B, N, _ = X.shape
    d_k = d_model // num_heads
    rng = np.random.default_rng(42)
    W_Q, W_K, W_V = [rng.normal(0, 0.1, size=(d_model, d_model)) for _ in range(3)]
    W_O = rng.normal(0, 0.1, size=(d_model, d_model))

    # Linear projection & reshape into (B, num_heads, N, d_k)
    Q = (X @ W_Q).reshape(B, N, num_heads, d_k).transpose(0, 2, 1, 3)
    K = (X @ W_K).reshape(B, N, num_heads, d_k).transpose(0, 2, 1, 3)
    V = (X @ W_V).reshape(B, N, num_heads, d_k).transpose(0, 2, 1, 3)

    # Parallel dot-product attention per head
    scores = (Q @ K.transpose(0, 1, 3, 2)) / np.sqrt(d_k)
    weights = np.exp(scores - np.max(scores, axis=-1, keepdims=True))
    weights /= weights.sum(axis=-1, keepdims=True)

    # Concatenate heads & final linear projection -> (B, N, d_model)
    heads_out = (weights @ V).transpose(0, 2, 1, 3).reshape(B, N, d_model)
    return heads_out @ W_O

X_in = np.random.default_rng(42).normal(size=(2, 8, 64))  # Batch 2, Seq 8, Dim 64
mha_out = multi_head_attention(X_in, num_heads=4, d_model=64)
print("Multi-Head Attention Output Shape:", mha_out.shape)  # (2, 8, 64)
```

Positional encodings. Plain self-attention is permutation-invariant. To inject sequence order, **positional encodings** are added to input embeddings. The original Transformer used sinusoidal frequencies (Vaswani et al. 2017). Modern LLMs (e.g., Llama 3) predominantly use **Rotary Position Embeddings (RoPE)** (Su et al. 2024), rotating query and key vectors in complex 2D planes to encode relative distance directly into $QK^\top$.

Residual connections, normalization, and optimization. Deep Transformer stacks survive vanishing gradients by wrapping every sub-layer with a **residual (skip) connection** (He et al. 2016) and **Layer Normalization**. While original Transformers used **Post-Layer Normalization** ($\mathbf{y} = \mathrm{LayerNorm}(\mathbf{x} + \mathcal{F}(\mathbf{x}))$), modern foundation models standardly use **Pre-Layer Normalization** (Pre-LN):

$$\mathbf{y} = \mathbf{x} + \mathcal{F}\big(\mathrm{LayerNorm}(\mathbf{x})\big). \tag{79}$$

Pre-LN preserves an unattenuated identity gradient highway ($\mathbf{x}$ passed directly to addition), enabling stable training across hundreds of layers without delicate warm-up schedules. Model optimization relies on **AdamW** (Loshchilov and Hutter 2019) with learning rate warmup and cosine decay schedules.

10.3 Transformer architecture variants: encoder, decoder, and encoder–decoder

The original Transformer (Vaswani et al. 2017) was designed for sequence-to-sequence tasks and contains two stacks: an **encoder** that maps a source sequence to contextual representations, and a **decoder** that generates a target sequence one token at a time. Modern architectures specialize one or both stacks for their primary task.

Encoder-only models. The encoder stack applies *bidirectional* (non-causal) self-attention: every token attends to every other token. Because there is no autoregressive constraint, the model can exploit both past and future context, making encoder representations particularly rich for

Variant	Attention type	Typical use	Examples
Encoder-only	Bidirectional self-attention	Classification, NER, embeddings	BERT, RoBERTa
Decoder-only	Causal (masked) self-attention	Autoregressive generation	GPT, Llama, Mistral
Encoder–decoder	Enc: bidirectional; Dec: causal + cross-attention	Seq2seq: translation, summarisation	T5, BART, Whisper

Table 6. The three principal Transformer variants and their defining attention patterns.

classification and retrieval tasks. BERT (Devlin et al. 2019) is pre-trained with Masked Language Modeling (MLM), randomly masking 15% of tokens and training the model to recover them from full bidirectional context.

Decoder-only models. The decoder stack applies *causal* (autoregressive) self-attention by adding a lower-triangular mask M to the attention logits before softmax:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top + M}{\sqrt{d_k}}\right) V, \quad M_{ij} = \begin{cases} 0 & j \le i \\ -\infty & j > i. \end{cases} \tag{80}$$

This ensures that the prediction of token t depends only on tokens $1, \ldots, t-1$, preserving the autoregressive factorisation $p(\mathbf{x}) = \prod_t p(x_t \mid x_{<t})$. The GPT family (Radford et al. 2018; Brown et al. 2020) and modern LLMs (Llama (Dubey et al. 2024), Mistral) follow this design exclusively.

Encoder–decoder models and cross-attention. The original Transformer (Vaswani et al. 2017) separates *understanding* from *generation* into two distinct stacks connected by **cross-attention**. Each decoder layer contains three sub-layers:

1. **Masked (causal) self-attention** over the target sequence generated so far.
2. **Cross-attention:** queries Q come from the decoder's hidden state, while keys K and values V come from the encoder's output $\mathbf{H}^{\text{enc}} \in \mathbb{R}^{n_{\text{src}} \times d_{\text{model}}}$:

$$\text{CrossAttention}(Q_{\text{dec}}, K_{\text{enc}}, V_{\text{enc}}) = \text{softmax}\left(\frac{Q_{\text{dec}} K_{\text{enc}}^\top}{\sqrt{d_k}}\right) V_{\text{enc}}. \tag{81}$$

   Cross-attention lets each generated token selectively "look up" any position in the source sequence, providing a dynamic, content-based alignment between source and target.
3. **Position-wise feed-forward network** (two linear layers with a ReLU or GELU nonlinearity).

When to use which Transformer variant

Choose the architecture by the information access pattern required at inference:

- **Need full bidirectional context over a fixed input** (classification, semantic search, NER) $\Rightarrow$ **encoder-only**.
- **Need to generate an open-ended continuation** (chat, code completion, story generation) $\Rightarrow$ **decoder-only**.
- **Need to transform one sequence into another** where source length $\neq$ target length and alignment is non-trivial (translation, abstractive summarisation, speech recognition) $\Rightarrow$ **encoder–decoder**.

Decoder-only models have largely displaced encoder–decoder designs for generation tasks because a single unified stack enables more efficient pre-training at scale; however encoder–decoder remains the preferred choice when a separate high-quality source representation is beneficial.

Check your understanding

1. Remove the nonlinearity from an MLP: what functions can the resulting network represent, and why?
2. In an undercomplete Autoencoder with linear layers and MSE loss, what classical linear algebra technique does the bottleneck space replicate?
3. Explain how Skip-Gram Negative Sampling (SGNS) overcomes the computational bottleneck of full-vocabulary softmax in Word2Vec.
4. Why is Multi-Head Attention (MHA) preferred over single-head attention in modern Transformers?

Sources & further reading

- F. Rosenblatt (1958). "The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain". In: *Psychological Review* 65.6, pp. 386–408
- D. E. Rumelhart et al. (1986). "Learning Representations by Back-Propagating Errors". In: *Nature* 323.6088, pp. 533–536
- K. Hornik et al. (1989). "Multilayer Feedforward Networks Are Universal Approximators". In: *Neural Networks* 2.5, pp. 359–366
- V. Nair and G. E. Hinton (2010). "Rectified Linear Units Improve Restricted Boltzmann Machines". In: *Proceedings of the 27th International Conference on Machine Learning (ICML)*, pp. 807–814
- Y. Bengio et al. (2013). "Representation Learning: A Review and New Perspectives". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 35.8, pp. 1798–1828
- P. Vincent et al. (2008). "Extracting and Composing Robust Features with Denoising Autoencoders". In: *Proceedings of the 25th International Conference on Machine Learning (ICML)*, pp. 1096–1103
- Y. LeCun et al. (1998). "Gradient-Based Learning Applied to Document Recognition". In: *Proceedings of the IEEE* 86.11, pp. 2278–2324
- S. Hochreiter and J. Schmidhuber (1997). "Long Short-Term Memory". In: *Neural Computation* 9.8, pp. 1735–1780
- K. He et al. (2016). "Deep Residual Learning for Image Recognition". In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778
- A. Vaswani et al. (2017). "Attention Is All You Need". In: *Advances in Neural Information Processing Systems 30*, pp. 5998–6008
- X. Glorot and Y. Bengio (2010). "Understanding the Difficulty of Training Deep Feedforward Neural Networks". In: *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS)*, pp. 249–256
- K. He et al. (2015). "Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification". In: *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pp. 1026–1034
- Y. Bengio et al. (1994). "Learning Long-Term Dependencies with Gradient Descent is Difficult". In: *IEEE Transactions on Neural Networks* 5.2, pp. 157–166
- A. M. Saxe et al. (2014). "Exact Solutions to the Nonlinear Dynamics of Learning in Deep Linear Neural Networks". In: *International Conference on Learning Representations (ICLR)*

11 Foundation Models and Generative AI

Module 5, at a glance (Estimated Study: 8 hours • Level: Advanced)

Focus. The self-supervised foundation model paradigm: subword tokenization, SSL pretext objectives, scaling laws, generative models (diffusion, LLMs), fine-tuning alignment (LoRA, RLHF, DPO), high-performance inference (KV-caching, RAG), and responsible AI revisited.

Key concepts. Tokenization (BPE), self-supervised learning, InfoNCE loss, Causal LM, Masked LM, scaling laws, diffusion models, SFT, LoRA/QLoRA, RLHF, DPO, KV-caching, RAG, and alignment failures.

After this module you will be able to explain how raw text is converted into tokens, derive contrastive InfoNCE and DPO loss formulations, explain compute-optimal scaling laws, implement parameter-efficient fine-tuning via LoRA, trace LLM next-token generation step-by-step, and audit foundation models against ethical safety dimensions.

11.1 Self-supervised learning and subword tokenization

The transition from classical machine learning to modern foundation models rests fundamentally on **Self-Supervised Learning (SSL)**. Classical supervised learning achieves high accuracy but is strictly bottlenecked by expensive, non-scalable human annotations $(\mathbf{x}_i, y_i)$. Traditional unsupervised learning (K-Means, PCA) scales to unlabelled data but struggles to discover task-relevant high-level semantics without targets. Self-supervised learning bridges this divide by turning unlabelled raw data into its own supervisor: the model automatically constructs artificial target signals (*pretext tasks*) directly from intrinsic structural patterns within raw inputs $\mathbf{x}$.

Tokenization: Bridging Raw Text and Model Inputs. Foundation models do not process raw string characters or whole dictionary words directly. Whole-word vocabularies suffer from out-of-vocabulary (OOV) failures and massive matrix dimensions, while character-level models yield long sequence lengths. Modern LLMs use **Subword Tokenization**, primarily **Byte-Pair Encoding (BPE)**, which iteratively merges the most frequent adjacent character or byte pairs into subwords.

Byte-Pair Encoding (BPE) Tokenization Step from Scratch

```python
from collections import defaultdict

def get_stats(vocab):
    pairs = defaultdict(int)
    for word, freq in vocab.items():
        symbols = word.split()
        for i in range(len(symbols) - 1):
            pairs[symbols[i], symbols[i+1]] += freq
    return pairs

def merge_vocab(pair, v_in):
    v_out = {}
    bigram = ' '.join(pair)
    replacement = ''.join(pair)
    for word in v_in:
        w_out = word.replace(bigram, replacement)
        v_out[w_out] = v_in[word]
    return v_out

# Synthetic toy vocabulary with word frequencies
vocab = {'l o w </w>': 5, 'l o w e r </w>': 2, 'n e w e s t </w>': 6, 'w i d e s t </w>': 3}
print("Initial Subword Vocab:", vocab)
for step in range(3):
    pairs = get_stats(vocab)
    best_pair = max(pairs, key=pairs.get)
    vocab = merge_vocab(best_pair, vocab)
    print(f"Step {step+1} Best Pair {best_pair} -> Merged Vocab: {vocab}")
```

Self-Supervised Learning: The Foundation Model Engine

Self-Supervised Learning converts unlabelled raw data (text, images, audio) into supervised learning targets by hiding, masking, or augmenting parts of the input and training a network to predict the missing signal. SSL provides an unlimited training signal, turning web-scale raw datasets into universal pre-trained representations.

Taxonomy of SSL Pretext Objectives. SSL transforms raw signals into target supervision through three primary mathematical formulations:

1. **Causal (Autoregressive) Language Modeling (CLM):** The model predicts token x_t conditioned on preceding historical context $\mathbf{x}_{1:t-1}$, minimizing negative log-likelihood over sequence length T:

$$\mathcal{L}_{\text{CLM}}(\boldsymbol{\theta}) = -\sum_{t=1}^{T} \ln P_{\boldsymbol{\theta}}(x_t \mid x_1, x_2, \ldots, x_{t-1}). \tag{82}$$

   Pioneered at scale by the GPT family (Radford et al. 2018; Brown et al. 2020), CLM creates generative autoregressive models capable of next-token prediction and zero/few-shot in-context learning.

2. **Masked Reconstruction (MLM & MAE):** A subset of positions $\mathcal{M}$ (typically 15% in text, 75% in vision) is corrupted or masked out. The network predicts original values x_i using bidirectional context $\mathbf{x}_{\setminus\mathcal{M}}$:

$$\mathcal{L}_{\text{MLM}}(\boldsymbol{\theta}) = -\sum_{i\in\mathcal{M}} \ln P_{\boldsymbol{\theta}}(x_i \mid \mathbf{x}_{\setminus\mathcal{M}}). \tag{83}$$

   Pioneered by BERT (Devlin et al. 2019) in NLP and Masked Autoencoders (MAE) (He et al.

2022) in computer vision, masked reconstruction forces bidirectional attention layers to model deep contextual dependencies.

3. **Contrastive Learning & InfoNCE Loss:** Beyond text, SSL for vision and multimodal data generates two stochastic augmented views $(\tilde{\mathbf{x}}_i, \tilde{\mathbf{x}}'_i)$ of the same image (positive pair) via cropping, color jittering, and blurring. Encoders project views into normalized latent vectors $\mathbf{z}_i = g(f(\tilde{\mathbf{x}}_i))$. The **InfoNCE Loss** (Oord et al. 2018; Chen et al. 2020) pulls positive pairs together while pushing negative views $\mathbf{z}_k^-$ (other images in the batch) apart:

$$\mathcal{L}_{\text{InfoNCE}} = -\sum_{i=1}^{N} \ln \frac{\exp\left(\text{sim}(\mathbf{z}_i, \mathbf{z}_i^+)/\tau\right)}{\exp\left(\text{sim}(\mathbf{z}_i, \mathbf{z}_i^+)/\tau\right) + \sum_{k \neq i} \exp\left(\text{sim}(\mathbf{z}_i, \mathbf{z}_k^-)/\tau\right)}, \tag{84}$$

where $\text{sim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u}^\top \mathbf{v}}{\|\mathbf{u}\|_2 \|\mathbf{v}\|_2}$ is cosine similarity and temperature hyperparameter $\tau > 0$ controls the penalty hardness assigned to difficult negative samples.

Non-contrastive SSL & Collapse Prevention. Contrastive learning requires large batch sizes (e.g., 4096 in SimCLR (Chen et al. 2020)) to supply sufficient negative samples. Non-contrastive methods (BYOL, SimSiam) eliminate negative pairs entirely, training an online network to predict target network representations. Representation collapse (where the encoder outputs a constant vector $\mathbf{z}_i = \mathbf{c}$) is prevented using asymmetrical predictor heads, moving-average target networks, and *stop-gradient* operations ($\text{sg}[\cdot]$).

The Symbiotic Relationship between SSL and Transformers. The simultaneous rise of Transformers (Vaswani et al. 2017) and SSL represents a profound technological synergy. SSL provided an infinite supply of training supervision from web text, while the Transformer provided the exact architectural mechanisms required to ingest that scale:

- **Elimination of Recurrent Bottlenecks:** RNNs process input sequences sequentially ($\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t)$), requiring $O(N)$ sequential operations during training. Backpropagation Through Time (BPTT) creates severe memory bottlenecks. Transformers compute all pairwise self-attention interactions in parallel ($O(1)$ sequential depth), unlocking full GPU throughput across trillions of SSL tokens.
- **Unrestricted Inductive Bias & Scaling Laws:** CNNs enforce local spatial kernels, and RNNs enforce strict temporal decay. While helpful on small datasets, rigid inductive biases cap model capacity on massive datasets. Transformers replace rigid priors with dynamic soft self-attention. When combined with SSL, Transformer capacity scales predictably according to empirical power laws without early performance saturation (Kaplan et al. 2020; Hoffmann et al. 2022).
- **The Pretrain–Finetune Paradigm Shift:** SSL converts raw unlabelled data into universal cognitive representations. The modern AI paradigm uses SSL to pretrain massive base Transformer models, which transfer efficiently to downstream tasks via Supervised Fine-Tuning (SFT), Parameter-Efficient Adaptation (LoRA (Hu et al. 2022)), or zero/few-shot in-context learning (Brown et al. 2020).

SimCLR Contrastive Learning with InfoNCE Loss from Scratch

```python
import numpy as np

# 1. Define vectorized InfoNCE Loss with Temperature Scaling (SimCLR style)
def infonce_loss(z1, z2, temperature=0.1):
    """
    z1, z2: (N, d) L2-normalized embedding projections for 2 augmented views
    """
    N = len(z1)
    # Concatenate both views -> 2N x d matrix
    z = np.vstack([z1, z2])

    # Compute 2N x 2N cosine similarity matrix (since vectors are L2-normalized)
    sim_matrix = (z @ z.T) / temperature

    # Mask out self-similarity along main diagonal
    np.fill_diagonal(sim_matrix, -1e9)

    # Positive pair target indices: row i pairs with (i + N) % (2N)
    labels = np.arange(2 * N)
    labels[:N] += N
    labels[N:] -= N

    # Numerically stable Log-Softmax Cross-Entropy Loss
    max_sim = np.max(sim_matrix, axis=1, keepdims=True)
    exp_sim = np.exp(sim_matrix - max_sim)
    log_prob = sim_matrix - max_sim - np.log(np.sum(exp_sim, axis=1, keepdims=True))

    loss = -np.mean(log_prob[np.arange(2 * N), labels])
    return loss

# 2. Simulate 2 augmented views of 4 samples in 8-dimensional projection space
rng = np.random.default_rng(42)
N, d = 4, 8
# Base representations
z_base = rng.normal(size=(N, d))
z_base /= np.linalg.norm(z_base, axis=1, keepdims=True)

# Positive views (small random augmentation perturbation)
z1 = z_base + 0.05 * rng.normal(size=(N, d))
z2 = z_base + 0.05 * rng.normal(size=(N, d))

# L2 normalize projection vectors
z1 /= np.linalg.norm(z1, axis=1, keepdims=True)
z2 /= np.linalg.norm(z2, axis=1, keepdims=True)

loss_aligned = infonce_loss(z1, z2, temperature=0.1)

# Unaligned random representations baseline
z_rand = rng.normal(size=(N, d))
z_rand /= np.linalg.norm(z_rand, axis=1, keepdims=True)
loss_unaligned = infonce_loss(z1, z_rand, temperature=0.1)

print(f"InfoNCE Loss (Aligned Positive Pairs):    {loss_aligned:.4f}")
print(f"InfoNCE Loss (Random Unaligned Pairs):   {loss_unaligned:.4f}")
```

11.2 Scaling laws and generative architectures

Latent Space & Diffusion Models. Generative models compress data into lower-dimensional latent representations. Variational Autoencoders (Kingma and Welling 2014) and GANs (Goodfellow et al. 2014) pioneered latent generation. **Diffusion Models** (Ho et al. 2020) dominate image/audio synthesis by training U-Nets or Vision Transformers to reverse a forward Gaussian noise process $q(\mathbf{x}_t \mid \mathbf{x}_{t-1})$.

Large Language Models & Causal Masking. Autoregressive LLMs predict the next token via cross-entropy loss over vocabulary V:

$$p(y = j \mid \mathbf{x}) = \frac{\exp(z_j)}{\sum_{k=1}^{V} \exp(z_k)}, \qquad \mathcal{L}_{\text{CE}} = -\log p(y_{\text{true}} \mid \mathbf{x}). \tag{85}$$

Causal attention masking ($M_{ij} = -\infty$ for $j > i$) prevents tokens from viewing future positions.

Empirical Scaling Laws. Kaplan et al. (2020) showed that language model loss $\mathcal{L}$ follows a power law in each of three independent variables when the others are held fixed:

$$\mathcal{L}(N) \approx \left(\frac{N_c}{N}\right)^{\alpha_N}, \quad \mathcal{L}(D) \approx \left(\frac{D_c}{D}\right)^{\alpha_D}, \quad \mathcal{L}(C) \approx \left(\frac{C_c}{C}\right)^{\alpha_C}, \tag{86}$$

where N is the number of non-embedding parameters, D is the training token count, C the total compute (FLOPs), and $N_c, D_c, C_c, \alpha_N, \alpha_D, \alpha_C$ are empirically fitted constants ($\alpha_N \approx 0.076$, $\alpha_D \approx 0.095$ in the original work). Crucially, the three scaling directions are *not* independent in practice: training always consumes compute $C \approx 6ND$ FLOPs (6 FLOPs per parameter per token: 2 for the forward pass and 4 for backward).

Chinchilla Compute-Optimal Scaling (Hoffmann et al. 2022)

Given a fixed compute budget C (in FLOPs), the cross-entropy loss is minimised by the parameter count N^* and token count D^* satisfying:

$$N^*(C) = G\left(\frac{C}{6}\right)^{a}, \qquad D^*(C) = G^{-1}\left(\frac{C}{6}\right)^{b}, \tag{87}$$

where $a \approx b \approx 0.50$ (Hoffmann et al. fit a=0.46, b=0.54) and G is a scale constant. To a good approximation both exponents are equal, meaning N **and** D **should scale at the same rate as** C **grows**. The concrete rule of thumb emerging from the paper is:

$$D^* \approx 20N^*. \tag{88}$$

This is a statement about the *ratio* under a fixed budget, not about absolute magnitudes. Example: a compute-optimal 70B-parameter model requires ≈1.4 trillion tokens, not 70 billion.

Inference-Optimal ("Overtrained") Scaling. The Chinchilla result assumes each training FLOP costs the same as each inference FLOP. In production, a model is trained once but served billions of times. **Inference-optimal** strategies deliberately *overtrain* a smaller model beyond the Chinchilla-optimal token budget: a smaller N lowers per-query cost, and additional tokens keep improving loss even past $D^* = 20N$ (Dubey et al. 2024). The Llama family exemplifies this—Llama 3 8B was trained on ∼15 T tokens, roughly $10\times$ the Chinchilla optimum—achieving loss competitive with much larger models while remaining cheap to deploy.

11.3 Fine-tuning, parameter efficiency, and preference alignment

Following pretraining, alignment transforms raw base models into helpful assistants. The standard pipeline proceeds in three stages:

Stage 1 — Supervised Fine-Tuning (SFT). The pretrained model is fine-tuned on a curated dataset of $(\text{prompt}, \text{demonstration})$ pairs written by human labellers. SFT teaches the model the *format* of a helpful response; it does not yet optimise human preference directly.

Stage 2a — Reinforcement Learning from Human Feedback (RLHF). (Christiano et al. 2017; Ouyang et al. 2022) RLHF introduces a separate **reward model** r_ϕ trained on human preference comparisons. Given two model outputs y_w (preferred) and y_l (dispreferred) for the same prompt x, human raters select the winner. The reward model is fitted via the **Bradley–Terry** pairwise preference model:

$$\mathcal{L}_{\text{RM}}(\phi) = -\mathbb{E}_{(x,\, y_w,\, y_l)\sim\mathcal{D}}\big[\ln \sigma\big(r_\phi(x, y_w) - r_\phi(x, y_l)\big)\big]\,, \tag{89}$$

where σ is the sigmoid function. Once r_ϕ is trained, the SFT policy π_θ is updated via **Proximal Policy Optimization (PPO)** to maximise expected reward while staying close to the SFT reference policy π_{ref} via a KL-divergence penalty:

$$\max_{\pi_\theta}\ \mathbb{E}_{x\sim\mathcal{D},\, y\sim\pi_\theta(\cdot|x)}\big[r_\phi(x,y)\big] - \beta\, D_{\text{KL}}\big(\pi_\theta(\cdot\mid x)\,\|\,\pi_{\text{ref}}(\cdot\mid x)\big), \tag{90}$$

where $\beta > 0$ controls the strength of the regularisation. The KL penalty prevents *reward hacking*: without it the policy exploits weaknesses in r_ϕ to produce outputs that score highly but are not genuinely helpful.

Stage 2b — Direct Preference Optimization (DPO). (Rafailov et al. 2023) DPO eliminates the explicit reward model by observing that the optimal policy π^* that solves the RLHF objective (90) can be written in closed form:

$$\pi^*(y\mid x) \propto \pi_{\text{ref}}(y\mid x)\,\exp\!\left(\frac{r^*(x,y)}{\beta}\right). \tag{91}$$

Inverting this relationship expresses the implicit reward as $r^*(x,y) = \beta\ln\frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)} + \beta\ln Z(x)$. Substituting into the Bradley–Terry objective and cancelling the partition function $Z(x)$ yields the **DPO loss**, which can be optimised directly on preference pairs without a reward model:

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x,\, y_w,\, y_l)}\left[\ln\sigma\left(\beta\ln\frac{\pi_\theta(y_w\mid x)}{\pi_{\text{ref}}(y_w\mid x)} - \beta\ln\frac{\pi_\theta(y_l\mid x)}{\pi_{\text{ref}}(y_l\mid x)}\right)\right]. \tag{92}$$

RLHF vs. DPO: key trade-offs

Property	RLHF (PPO)	DPO
Reward model	Explicit r_ϕ, trained separately	Implicit; absorbed into policy loss
Optimizer	Online RL (PPO); requires a rollout	Offline supervised; no rollout needed
Stability	Sensitive to r_ϕ quality; reward hacking risk	More stable; simpler implementation
Flexibility	Can optimise non-differentiable reward signals	Requires pre-collected preference pairs
Practice	InstructGPT, Claude (early), Gemini	Zephyr, Llama-3-Instruct, Phi-3

Stage 3 — Parameter-Efficient Fine-Tuning (PEFT / LoRA). Full fine-tuning of a billion-parameter model is prohibitively expensive. **LoRA** (Hu et al. 2022) freezes pretrained weight matrices $W_0 \in \mathbb{R}^{d\times k}$ and injects trainable low-rank decomposition matrices $\Delta W = BA$ ($B \in \mathbb{R}^{d\times r}$, $A \in$

$\mathbb{R}^{r\times k}$, $r \ll \min(d,k)$), leveraging classical low-rank matrix factorisation (Strang 2012) to reduce trainable parameters by $>99\%$. **QLoRA** (Dettmers et al. 2024) extends this by quantising base weights to 4-bit NormalFloat while preserving full-precision low-rank adapters, enabling fine-tuning of 70B+ models on a single GPU.

11.4 Foundation model inference, KV-caching, and RAG

At inference, token sampling controls decoding:

- **Temperature Scaling & Greedy Decoding:** Softmax $\mathrm{softmax}(\mathbf{z}/T)$ controls distribution entropy. As $T \to 0^+$, the distribution converges to a one-hot vector at $\arg\max \mathbf{z}$, which code libraries alias as `temperature=0` (greedy decoding).
- **Sampling Strategies:** Top-k truncates sampling to the k most probable tokens; Top-p (nucleus) samples from the smallest candidate set comprising cumulative probability p.
- **Inference Acceleration:** High-throughput inference relies on **KV Caching**[2], continuous batching, quantization (INT8/INT4), and speculative decoding.
- **Retrieval-Augmented Generation (RAG):** RAG (Lewis et al. 2020) grounds generation by retrieving relevant document snippets from vector databases into prompt context.

How an LLM writes one token

1. Tokenise the prompt into sub-word tokens.
2. Embed each token into a vector (plus RoPE positional encoding).
3. Pass the sequence through stacked Pre-LN transformer blocks (causal self-attention + FFN).
4. Map final position hidden state through linear head and softmax over vocabulary V.
5. Sample next token via Temperature / Top-p or select argmax greedy token ($T \to 0^+$).
6. Append token to sequence and update KV-cache for the next step.

11.5 Ethics, alignment, and safety revisited: the generative AI frontier

Module 1 introduced six baseline dimensions of AI ethics: algorithmic bias, privacy, alignment, copyright/provenance, environmental footprint, and automation bias. Generative AI and foundation models exacerbate these challenges while introducing distinct novel failure modes:

Ethics & Safety Spotlight: Foundation Model Risk Checklist

- **Hallucinations & Grounding Failure:** Fluent syntax masks ungrounded factual inaccuracies. RLHF can exacerbate "sycophancy" where models echo user biases rather than truth.
- **Data Provenance & Copyright:** Web-scale pretraining relies on copyrighted literary and visual art without attribution or compensation, driving legal and policy battles.
- **Watermarking & Synthetic Content Identification:** As AI-generated text, image, and deepfake video proliferate, cryptographic provenance and statistical watermarking become vital for info-ecosystem integrity.
- **Environmental & Compute Externalities:** Training modern foundation models consumes gigawatt-hours of electricity and significant cooling water (Strubell et al. 2019; Patterson et al. 2021), highlighting the need for efficient architectures and compute-optimal

[2] KV Caching stores past K and V tensors in memory, making incremental attention computation per generated token linear in context length $O(N)$ instead of $O(N^2)$.

training.

Check your understanding

1. Why does Self-Supervised Learning (SSL) depend so critically on the parallel self-attention mechanism of Transformers compared to sequential RNNs?
2. Mathematically, what role does temperature τ play in the InfoNCE loss for contrastive learning?
3. What key trade-off does subword tokenization (e.g. Byte-Pair Encoding) balance compared to whole-word or character-level tokenization?
4. According to Chinchilla scaling laws, what is the compute-optimal ratio between model parameters and training tokens, and why might a practitioner deliberately deviate from it?
5. Explain how LoRA achieves $> 99\%$ parameter reduction during fine-tuning via low-rank matrix decomposition.

Sources & further reading

- A. Vaswani et al. (2017). "Attention Is All You Need". In: *Advances in Neural Information Processing Systems 30*, pp. 5998–6008
- A. van den Oord et al. (2018). "Representation Learning with Contrastive Predictive Coding". In: *arXiv preprint arXiv:1807.03748*
- T. Chen et al. (2020). "A Simple Framework for Contrastive Learning of Visual Representations". In: *Proceedings of the 37th International Conference on Machine Learning (ICML)*, pp. 1597–1607
- J. Devlin et al. (2019). "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding". In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, pp. 4171–4186
- A. Radford et al. (2018). "Improving Language Understanding by Generative Pre-Training". In: *OpenAI Blog*
- A. Dosovitskiy et al. (2021). "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale". In: *International Conference on Learning Representations (ICLR)*
- K. He et al. (2022). "Masked Autoencoders Are Scalable Vision Learners". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 16000–16009
- J. Su et al. (2024). "RoFormer: Enhanced Transformer with Rotary Position Embedding". In: *Neurocomputing* 568, p. 127063
- T. Mikolov et al. (2013). "Distributed Representations of Words and Phrases and Their Compositionality". In: *Advances in Neural Information Processing Systems 26*, pp. 3111–3119
- D. P. Kingma and M. Welling (2014). "Auto-Encoding Variational Bayes". In: *International Conference on Learning Representations (ICLR)*, pp. 1–9
- I. Goodfellow et al. (2014). "Generative Adversarial Nets". In: *Advances in Neural Information Processing Systems 27*, pp. 2672–2680
- J. Ho et al. (2020). "Denoising Diffusion Probabilistic Models". In: *Advances in Neural Information Processing Systems 33*, pp. 6840–6851
- T. B. Brown et al. (2020). "Language Models Are Few-Shot Learners". In: *Advances in Neural Information Processing Systems 33*, pp. 1877–1901
- J. Hoffmann et al. (2022). "Training Compute-Optimal Large Language Models". In: *arXiv preprint arXiv:2203.15556*
- L. Ouyang et al. (2022). "Training Language Models to Follow Instructions with Human Feedback". In: *Advances in Neural Information Processing Systems 35*, pp. 27730–27744
- R. Rafailov et al. (2023). "Direct Preference Optimization: Your Language Model is Secretly a Reward Model". In: *Advances in Neural Information Processing Systems 36*
- E. J. Hu et al. (2022). "LoRA: Low-Rank Adaptation of Large Language Models". In: *International Conference on Learning Representations (ICLR)*

- T. Dettmers et al. (2024). “QLoRA: Efficient Finetuning of Quantized LLMs”. In: *Advances in Neural Information Processing Systems 36*
- P. Lewis et al. (2020). “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Advances in Neural Information Processing Systems 33*, pp. 9459–9474
- E. Strubell et al. (2019). “Energy and Policy Considerations for Deep Learning in NLP”. in: *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 3645–3650
- D. Patterson et al. (2021). “Carbon Emissions and Large Neural Network Training”. In: *Advances in Neural Information Processing Systems 34*, pp. 13470–13481
- E. M. Bender et al. (2021). “On the Dangers of Stochastic Parrots: Can Language Models Be Too Big?” In: *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency (FAccT)*, pp. 610–623

12 Modern LLM Architectures

Module 6, at a glance (Estimated Study: 5 hours • Level: Advanced)

Focus. How leading open-weight LLMs (Llama, Mistral, Mixtral) modernise the vanilla Transformer decoder: replacing Layer Norm with RMSNorm, GELU with SwiGLU, absolute positional encodings with RoPE, Multi-Head Attention (MHA) with Grouped-Query Attention (GQA), and dense FFNs with Sparse Mixture-of-Experts (MoE) routing.

Key concepts. RMSNorm, SwiGLU, Grouped-Query Attention, Mixture of Experts, Top-k gating, load-balancing loss, sliding-window attention, decoder-only architecture.

After this module you will be able to identify the architectural components of Llama 3, Mistral 7B, and Mixtral 8×7B, explain the motivation for each design choice in terms of efficiency and quality, and derive the MoE gating computation.

12.1 Core modifications in modern decoder-only LLMs

The vanilla Transformer decoder (Vaswani et al. 2017) uses Post-Layer Normalization, learned sinusoidal positional encodings, Multi-Head Attention, and feed-forward networks with GELU activations. Every major open-weight LLM released since 2023 makes systematic improvements to each of these components.

RMSNorm. Standard Layer Normalization (Vaswani et al. 2017) centres and rescales activations:

$$\mathrm{LayerNorm}(\mathbf{x}) = \frac{\mathbf{x} - \mu}{\sigma + \epsilon}\boldsymbol{\gamma} + \boldsymbol{\beta}, \quad \mu = \frac{1}{d}\sum_i x_i, \quad \sigma^2 = \frac{1}{d}\sum_i (x_i - \mu)^2. \tag{93}$$

Root Mean Square Layer Normalization (RMSNorm) (Zhang and Sennrich 2019) drops mean subtraction, computing only the re-scaling:

$$\mathrm{RMSNorm}(\mathbf{x}) = \frac{\mathbf{x}}{\mathrm{RMS}(\mathbf{x})}\boldsymbol{\gamma}, \quad \mathrm{RMS}(\mathbf{x}) = \sqrt{\frac{1}{d}\sum_i x_i^2 + \epsilon}. \tag{94}$$

Removing the mean-subtraction eliminates the implicit centering invariance, which is empirically redundant, saving ≈ 7–8% of normalization compute. Llama 3 (Dubey et al. 2024) and Mistral (Jiang et al. 2023) both adopt RMSNorm.

SwiGLU feed-forward networks. The original Transformer FFN applies two linear layers with a GELU nonlinearity between them. Shazeer (2020) introduced *Gated Linear Units (GLU)* variants. **SwiGLU**, used in Llama and PaLM, replaces the standard FFN:

$$\mathrm{FFN}_{\mathrm{SwiGLU}}(\mathbf{x}) = \big(\mathrm{Swish}(W_1\mathbf{x}) \odot W_3\mathbf{x}\big)\, W_2, \tag{95}$$

where $\mathrm{Swish}(z) = z \cdot \sigma(z)$ is a smooth, non-monotonic activation and $\odot$ is element-wise multiplication. The gating mechanism ($W_3\mathbf{x}$) allows the network to selectively suppress or amplify intermediate activations, improving representational capacity without increasing depth. To keep FLOPs constant vs. the vanilla FFN, the hidden dimension is scaled to $\frac{2}{3} \cdot 4d_{\mathrm{model}}$.

12.2 Grouped-Query Attention (GQA)

Multi-Head Attention (MHA) with h heads maintains h independent Key and Value matrices, incurring a KV-cache of size $O(h \cdot n \cdot d_k)$ per layer. **Multi-Query Attention (MQA)** reduces this

to a single shared KV head, dramatically shrinking the cache but sometimes hurting quality. **Grouped-Query Attention (GQA)** (Ainslie et al. 2023) interpolates: G groups share a single KV head, while each group contains h/G query heads:

$$\mathrm{GQA}(Q,K,V) = \mathrm{Concat}\Big(\mathrm{head}_1,\ldots,\mathrm{head}_h\Big)\,W^O, \quad \mathrm{head}_{g,i} = \mathrm{Attention}\big(Q_{g,i}\,W^Q_{g,i},\; K_g\,W^K_g,\; V_g\,W^V_g\big). \tag{96}$$

With $G = h$ we recover MHA; with $G = 1$ we recover MQA. Llama 3 uses $G = 8$ (8 KV heads for 32 query heads), reducing KV-cache memory by $4\times$ with minimal quality degradation.

12.3 Mixture-of-Experts (MoE)

Sparse Mixture-of-Experts (MoE) (Shazeer et al. 2017) replaces each dense FFN with E independent *expert* networks and a learned *router* that activates only a top-k subset of experts per token. For input token $\mathbf{x}$:

$$g(\mathbf{x}) = \mathrm{Softmax}\big(\mathbf{x}\,W_r\big) \in \mathbb{R}^E, \tag{97}$$

$$\mathcal{S}(\mathbf{x}) = \text{Top-}k\big\{(i, g_i(\mathbf{x}))\big\}, \tag{98}$$

$$\mathrm{MoE}(\mathbf{x}) = \sum_{(i,g_i)\in\mathcal{S}(\mathbf{x})} g_i(\mathbf{x})\,\mathrm{FFN}_i(\mathbf{x}). \tag{99}$$

Mixtral 8×7B (Jiang et al. 2024) uses $E = 8$ experts and $k = 2$: each token is processed by 2 of 8 FFN experts, giving $\sim$13B active parameters out of $\sim$47B total. A *load-balancing auxiliary loss* penalises unequal expert utilisation:

$$\mathcal{L}_{\text{aux}} = \alpha \cdot E \sum_{i=1}^{E} f_i \cdot P_i, \tag{100}$$

where f_i is the fraction of tokens routed to expert i and P_i is the average routing probability, encouraging uniform load across experts.

Modern LLM Architecture Signature

Every modern open-weight LLM decoder combines: **Pre-RMSNorm**, **RoPE** positional encoding, **GQA** for memory-efficient attention, and **SwiGLU** feed-forward blocks. Larger models additionally use **Sparse MoE** FFN layers to scale parameter count without scaling compute proportionally.

Model	Params	Norm	Activation	Pos. Enc.	Attention
GPT-2	1.5B	LayerNorm	GELU	Learned abs.	MHA
Llama 2	7–70B	RMSNorm	SwiGLU	RoPE	GQA (34B+)
Llama 3	8–70B	RMSNorm	SwiGLU	RoPE	GQA
Mistral 7B	7B	RMSNorm	SwiGLU	RoPE	GQA + SWA
Mixtral 8×7B	47B total	RMSNorm	SwiGLU	RoPE	GQA + MoE
Gemma 2	2–27B	RMSNorm	GEGLU	RoPE	GQA

Table 7. Architectural components across representative modern decoder-only LLMs. SWA = Sliding-Window Attention; GQA = Grouped-Query Attention; MoE = Mixture-of-Experts.

Check your understanding

1. Why does RMSNorm eliminate mean subtraction, and what property of representations makes this acceptable?
2. Derive the FLOPs of GQA with G groups versus standard MHA, and show the KV-cache reduction factor.
3. In Mixtral's MoE architecture, why is a load-balancing auxiliary loss necessary, and what pathological behaviour does it prevent?

Sources & further reading

- A. Vaswani et al. (2017). "Attention Is All You Need". In: *Advances in Neural Information Processing Systems 30*, pp. 5998–6008
- A. Dubey et al. (2024). "The Llama 3 Herd of Models". In: *arXiv preprint arXiv:2407.21783*
- H. Touvron et al. (2023). "Llama 2: Open Foundation and Fine-Tuned Chat Models". In: *arXiv preprint arXiv:2307.09288*
- A. Q. Jiang et al. (2023). "Mistral 7B". in: *arXiv preprint arXiv:2310.06825*
- A. Q. Jiang et al. (2024). "Mixtral of Experts". In: *arXiv preprint arXiv:2401.04088*
- N. Shazeer (2020). "GLU Variants Improve Transformer". In: *arXiv preprint arXiv:2002.05202*
- J. Ainslie et al. (2023). "GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints". In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 4895–4901

13 Efficient Attention Mechanisms

Module 7, at a glance (Estimated Study: 4 hours • Level: Advanced)

Focus. Why standard self-attention is a memory bottleneck, and how FlashAttention, sliding-window attention, and ALiBi overcome it without sacrificing output quality.

Key concepts. IO-awareness, HBM vs. SRAM, tiling, online softmax, FlashAttention, sliding-window attention, ALiBi, sparse attention, linear attention.

After this module you will be able to explain the IO complexity of standard vs. FlashAttention, derive the tiling recurrence for online softmax, and compare length-extrapolation strategies.

13.1 The memory bottleneck: IO complexity of self-attention

Standard self-attention materialises the full $n \times n$ attention score matrix $A = QK^\top/\sqrt{d_k}$ in High-Bandwidth Memory (HBM), before applying softmax and multiplying by V. The number of HBM reads and writes (measured in bytes, not FLOPs) is:

$$\mathrm{IO}_{\text{standard}} = \Theta\big(nd_k + n^2\big), \tag{101}$$

where the n^2 term corresponds to reading and writing the full attention matrix. For long contexts ($n = 128$k tokens), this dominates compute time because the GPU's arithmetic units are *memory-bandwidth bound*, not *compute bound*.

13.2 FlashAttention: IO-aware tiled attention

Dao et al. (2022) observed that self-attention can be computed *without ever materialising* the full $n \times n$ matrix, provided keys, queries, and values are processed in tiles that fit in on-chip SRAM ($\ll$HBM). The key insight is the **online softmax recurrence**: given partial sums $m^{(\text{old})}$ (running max) and $\ell^{(\text{old})}$ (running normalizer), updating the softmax denominator when a new tile of scores $\mathbf{s}^{(\text{new})}$ arrives requires only:

$$m^{(\text{new})} = \max\big(m^{(\text{old})},\, \max(\mathbf{s}^{(\text{new})})\big), \tag{102}$$

$$\ell^{(\text{new})} = e^{m^{(\text{old})} - m^{(\text{new})}}\, \ell^{(\text{old})} + \sum_j e^{s_j^{(\text{new})} - m^{(\text{new})}}. \tag{103}$$

The accumulated output O is rescaled at each tile step, yielding the exact softmax output without storing intermediate $n \times n$ matrices. FlashAttention achieves:

$$\mathrm{IO}_{\text{Flash}} = \Theta\big(n^2 d_k / M\big), \tag{104}$$

where M is the SRAM size. For practical values ($M \approx 100$ KB, $d_k = 64$), this is 4–20$\times$ fewer HBM accesses, translating directly to wall-clock speedup and $O(n)$ HBM memory footprint (vs. $O(n^2)$). **FlashAttention-2** (Dao 2024) further improves work partitioning across warps to achieve $\approx 70\%$ of theoretical A100 GPU FLOPs utilisation.

13.3 Sliding-Window Attention (SWA)

For long documents, full attention is often unnecessary: most token interactions decay rapidly with distance. **Sliding-Window Attention** restricts each token to attend only to its w nearest

tokens:

$$A_{ij} = \begin{cases} \text{Attention}(q_i, k_j, v_j) & |i-j| \le w/2 \\ 0 & \text{otherwise.} \end{cases} \tag{105}$$

With L layers, the *effective receptive field* grows to $L \cdot w$ positions — sufficient for most practical document lengths. SWA reduces attention complexity from $O(n^2)$ to $O(n \cdot w)$ and is deployed in Mistral 7B (Jiang et al. 2023) for its 32k-token context.

13.4 Positional encoding and length extrapolation

ALiBi (Attention with Linear Biases) (Press et al. 2022) eliminates learned or sinusoidal positional embeddings entirely, instead adding a fixed negative linear bias to attention logits proportional to distance:

$$\text{Attention}(Q, K, V) = \text{softmax}\Big(\frac{QK^\top}{\sqrt{d_k}} - m \cdot |i-j|\Big) V, \tag{106}$$

where scalar m is head-specific. This design enables *length extrapolation*: models trained on sequences of length L generalise to $L' \gg L$ at inference, a property not shared by RoPE or sinusoidal encodings without finetuning.

Efficient Attention Trade-offs

- **FlashAttention**: exact attention, $O(n^2 d/M)$ IO (vs. $O(n^2)$), best for moderate context.
- **Sliding-Window**: $O(nw)$ compute, limited receptive field unless layered.
- **Sparse/Block Sparse**: $O(n\sqrt{n})$ or $O(n \log n)$ patterns; approximate.
- **Linear Attention**: $O(nd^2)$ via kernel trick; approximate, no softmax guarantee.

FlashAttention is the near-universal choice for dense attention today; SWA provides additional savings for very long contexts.

Check your understanding

1. Why is the $n \times n$ attention matrix a *memory-bandwidth* bottleneck rather than a *FLOPs* bottleneck on modern GPUs?
2. Explain the online softmax recurrence used by FlashAttention: what two running statistics does it maintain, and why is their update numerically stable?
3. Compare the length-extrapolation properties of RoPE, ALiBi, and learned absolute positional embeddings.

Sources & further reading

- A. Vaswani et al. (2017). "Attention Is All You Need". In: *Advances in Neural Information Processing Systems 30*, pp. 5998–6008
- T. Dao et al. (2022). "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness". In: *Advances in Neural Information Processing Systems 35*, pp. 16344–16359
- T. Dao (2024). "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning". In: *International Conference on Learning Representations (ICLR)*
- O. Press et al. (2022). "Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation". In: *International Conference on Learning Representations (ICLR)*
- R. Child et al. (2019). "Generating Long Sequences with Sparse Transformers". In
- A. Q. Jiang et al. (2023). "Mistral 7B". in: *arXiv preprint arXiv:2310.06825*
- J. Su et al. (2024). "RoFormer: Enhanced Transformer with Rotary Position Embedding". In: *Neurocomputing* 568, p. 127063

14 Model Quantization

Module 8, at a glance (Estimated Study: 4 hours • Level: Advanced)

Focus. Reducing the memory and compute footprint of LLMs through weight and activation quantization: from first principles of uniform quantization through GPTQ, AWQ, and bitsandbytes.

Key concepts. Symmetric/asymmetric quantization, calibration, Post-Training Quantization (PTQ), Quantization-Aware Training (QAT), GPTQ, AWQ, NF4, LLM.int8().

After this module you will be able to derive the quantization error bound, explain why second-order information is used in GPTQ, describe how AWQ selects salient weights, and compare deployment formats (GGUF, bitsandbytes, TensorRT-LLM).

14.1 Quantization fundamentals

A floating-point weight tensor $W \in \mathbb{R}^{m \times n}$ is quantized to b bits by mapping the range $[w_{\min}, w_{\max}]$ to 2^b integers. **Uniform symmetric quantization** uses:

$$W_q = \text{clip}\left(\text{round}\left(\frac{W}{\Delta}\right),\ -2^{b-1},\ 2^{b-1}-1\right), \quad \Delta = \frac{\max(|W|)}{2^{b-1}-1}, \tag{107}$$

and the dequantized approximation $\hat{W} = W_q \cdot \Delta$ introduces a worst-case absolute error bounded by $|\Delta|/2$ per element. The **quantization SNR** (signal-to-noise ratio) decreases as b decreases: moving from FP32 to INT8 loses roughly $\log_2(2^{32}/2^8) = 24$ bits of effective precision.

Format	Bits / Param	Memory (7B)	Notes
FP32	32	~28 GB	Training reference
BF16	16	~14 GB	Standard inference
FP8	8	~7 GB	H100 native; NVIDIA TensorRT-LLM
INT8	8	~7 GB	LLM.int8(), bitsandbytes
GPTQ-4	4	~3.5 GB	Group-wise, second-order PTQ
NF4	4	~3.5 GB	Normally-distributed float; QLoRA
GGUF Q4	4	~3.5 GB	llama.cpp mixed-precision blocks
GGUF Q2	2	~1.8 GB	Severe quality degradation

Table 8. Common quantization formats, their memory footprint for a 7B-parameter model, and typical deployment context.

14.2 GPTQ: second-order post-training quantization

GPTQ (Frantar et al. 2023) applies the Optimal Brain Quantization (OBQ) framework to quantize an entire LLM layer by layer. For each weight matrix $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, the quantization error on the layer output is:

$$\mathcal{L}_{\text{quant}}(W, \hat{W}) = \left\| (W - \hat{W})\, X \right\|_F^2, \tag{108}$$

where $X \in \mathbb{R}^{d_{\text{in}} \times T}$ is the layer's input activations over a calibration dataset ($T \approx 128$ samples). GPTQ greedily quantizes columns one by one, at each step computing the optimal compensating update to the remaining unquantized weights via the inverse Hessian $H^{-1} = (2XX^\top)^{-1}$:

$$\delta W_{:,j'} = -\frac{w_{:,j} - \hat{w}_{:,j}}{[H^{-1}]_{jj}} \cdot [H^{-1}]_{j,j'>j}, \tag{109}$$

which propagates quantization error to remaining columns. Using Cholesky decomposition of H^{-1}, GPTQ quantizes a 175B-parameter model in ≈4 GPU-hours with less than 1% perplexity increase at 4 bits.

14.3 AWQ: activation-aware weight quantization

AWQ (Lin et al. 2024) observes that only $\approx 1\%$ of weight channels correspond to *salient* input activations with large magnitudes, and that quantizing these salient channels introduces disproportionately large output error. AWQ protects salient channels by scaling them up before quantization (and inverting the scale after), effectively assigning more quantization levels to high-activation-magnitude channels:

$$\hat{W} = \text{Quantize}(W \cdot \text{diag}(\mathbf{s})) \cdot \text{diag}(\mathbf{s})^{-1}, \tag{110}$$

where saliency scale vector $\mathbf{s}$ is chosen to minimize per-channel output error on a calibration set. AWQ achieves comparable quality to GPTQ with lower calibration cost and better hardware compatibility (no sequential column-by-column dependency).

14.4 bitsandbytes and NF4

LLM.int8() (Dettmers et al. 2022) performs mixed-precision decomposition: extreme-magnitude *outlier* columns (identified at runtime) are kept in FP16, while the remaining $\approx 99.9\%$ of weights are quantized to INT8. Matrix multiplication is split across the two precisions and the results are summed. This preserves model quality at INT8 with no calibration dataset needed.

NF4 (NormalFloat-4) (Dettmers et al. 2024) is a 4-bit data type that minimizes quantization error under the assumption that pre-trained weights follow a zero-centred normal distribution $\mathcal{N}(0, \sigma^2)$. The 16 quantization levels are placed at the quantiles of the normal distribution, not uniformly: $\{q_i\}_{i=1}^{16}$ where $q_i = \Phi^{-1}(i/17)\,\sigma$ (re-normalised to $[-1, 1]$). NF4 is the default quantization format for QLoRA fine-tuning.

When to Use Which Quantization Scheme

- **BF16**: default for inference on H100/A100 with sufficient VRAM.
- **GPTQ-4 / AWQ-4**: best quality at 4-bit for GPU deployment; requires calibration.
- **LLM.int8()**: drop-in 8-bit for any model without calibration (bitsandbytes).
- **QLoRA + NF4**: 4-bit base model + 16-bit LoRA adapters for fine-tuning on consumer GPUs.
- **GGUF**: CPU/mixed inference via llama.cpp, supports Q2–Q8 per-block quantization.

Check your understanding

1. Derive the maximum absolute quantization error for symmetric b-bit uniform quantization with scale Δ.
2. Why does GPTQ use the inverse Hessian H^{-1} when propagating quantization error?

What quantity does the Hessian measure?

3. Explain the key insight behind AWQ's saliency-weighted scaling, and why it achieves lower output error than naive per-tensor or per-channel quantization.

Sources & further reading

- E. Frantar et al. (2023). "GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers". In: *International Conference on Learning Representations (ICLR)*
- J. Lin et al. (2024). "AWQ: Activation-Aware Weight Quantization for LLM Compression and Acceleration". In: *Machine Learning and Systems (MLSys)*
- T. Dettmers et al. (2022). "LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale". In: *Advances in Neural Information Processing Systems 35*, pp. 30318–30332
- T. Dettmers et al. (2024). "QLoRA: Efficient Finetuning of Quantized LLMs". In: *Advances in Neural Information Processing Systems 36*
- E. J. Hu et al. (2022). "LoRA: Low-Rank Adaptation of Large Language Models". In: *International Conference on Learning Representations (ICLR)*

15 Distributed Training

Module 9, at a glance (Estimated Study: 5 hours • Level: Advanced)

Focus. Training large models across hundreds or thousands of GPUs using Data Parallelism (DDP), the ZeRO memory optimisation stages (PyTorch FSDP), Tensor Parallelism, and Pipeline Parallelism as used in Megatron-LM.

Key concepts. All-Reduce, gradient averaging, ZeRO stages 1–3, Fully Sharded Data Parallelism (FSDP), tensor parallelism, pipeline parallelism, micro-batching, gradient checkpointing, BF16 mixed-precision.

After this module you will be able to derive the memory savings of ZeRO stage 3, explain the All-Reduce collective and its bandwidth cost, and understand how 3D parallelism composes all three parallelism dimensions.

15.1 Data parallelism and the All-Reduce bottleneck

Data Parallelism (DP) replicates the full model on every GPU and partitions the training mini-batch across D workers. After each forward/backward pass, workers must synchronise gradients via an **All-Reduce** collective:

$$\bar{\mathbf{g}} = \frac{1}{D}\sum_{d=1}^{D} \mathbf{g}_d, \tag{111}$$

which reduces the per-GPU gradient $\mathbf{g}_d$ to the global mean and broadcasts it back. The communication volume per All-Reduce is $2P$ parameters (reduce + broadcast), where P is parameter count. **PyTorch DistributedDataParallel (DDP)** overlaps this gradient communication with the backward pass via *gradient bucketing*, hiding latency behind compute.

15.2 ZeRO: sharding optimizer state, gradients, and parameters

Standard DP replicates not just model parameters ($16P$ bytes in BF16 + FP32 master copy) but also optimizer states ($8P$ bytes for Adam's first and second moments) on every GPU. For a 7B-parameter model this totals ≈112 GB — far beyond a single 80 GB A100. **ZeRO (Zero Redundancy Optimizer)** (Rajbhandari et al. 2020) shards memory across D workers in three stages:

ZeRO Stage	What is sharded	Memory per GPU
Stage 1	Optimizer states (P_{os})	$\frac{K\Psi}{D} + 2\Psi$
Stage 2	Optimizer states + gradients	$\frac{(K+2)\Psi}{D} + 2\Psi$
Stage 3	Optimizer states + gradients + parameters	$\frac{(K+2+2)\Psi}{D}$

Table 9. ZeRO memory breakdown. Ψ = parameter count in BF16; K = optimizer state multiplier (12 for Adam: 4 master + 4 m + 4 v bytes); D = number of GPUs.

ZeRO Stage 3 achieves linear memory scaling: a 7B model that requires 112 GB per GPU with DDP requires only $112/D$ GB per GPU with Stage 3, enabling training on clusters of commodity 80 GB GPUs. **PyTorch Fully Sharded Data Parallel (FSDP)** is the official ZeRO Stage 3 implementation, with All-Gather and Reduce-Scatter collectives reconstructing parameter shards on-the-fly during forward and backward passes.

15.3 Tensor and pipeline parallelism

For models that do not fit on a single GPU even at Stage 3 (e.g. 70B+ dense models), the model itself must be split. **Megatron-LM** (Shoeybi et al. 2019; Narayanan et al. 2021) introduces two complementary strategies:

Tensor Parallelism (TP) splits each weight matrix column- or row-wise across T GPUs. For the MHA projection $Y = XA$ with $A \in \mathbb{R}^{d\times 4d}$:

$$Y = X[A_1 \mid A_2 \mid \cdots \mid A_T], \quad Y_t = XA_t \quad \text{(each GPU } t \text{ holds } A_t\text{)}. \tag{112}$$

A single All-Reduce over T GPUs synchronises the output after each column-partitioned GeMM, incurring $O(Bnd/T)$ communication per layer where B is batch size and n is sequence length.

Pipeline Parallelism (PP) partitions the L Transformer layers into P sequential stages, each on a different set of GPUs. A micro-batching schedule (e.g., *1F1B*: one forward, one backward interleaved) keeps all pipeline stages busy, reducing the idle *pipeline bubble* from $O(P)$ to $O(1)$ micro-batches.

3D Parallelism combines DP, TP, and PP: for a cluster of $D \cdot T \cdot P$ GPUs, DP operates across DP groups, TP within nodes (fast NVLink), and PP across nodes (slower InfiniBand). This is the configuration used to train GPT-3, Megatron-Turing NLG, and Llama 3 at scale.

15.4 Gradient checkpointing and mixed precision

Gradient checkpointing (also called *activation recomputation*) reduces activation memory from $O(L \cdot n \cdot d)$ to $O(\sqrt{L \cdot n \cdot d})$ by storing only a sparse subset of activations and recomputing intermediate ones during the backward pass, at the cost of ≈33% extra FLOPs.

BF16 mixed-precision training maintains a master FP32 copy of parameters for the optimizer step, while forward and backward passes use BF16 ($5\times$ more bandwidth than FP32). Dynamic **loss scaling** prevents BF16 gradient underflow: the loss is multiplied by a scale factor s before the backward pass, and gradients are unscaled before the optimizer step, with s halved on overflow and doubled on clean steps.

Parallelism Strategy Selection

- $\leq$ 7B parameters: DDP (single node, 8×A100).
- 7–70B: FSDP (ZeRO-3) across multiple nodes.
- 70B+: FSDP + Tensor Parallelism (TP within nodes via NVLink).
- ≥200B: 3D Parallelism (DP + TP + PP) as in Megatron-LM.

Gradient checkpointing and BF16 mixed precision are universally applied to maximise batch size and stabilise training.

Check your understanding

1. Compute the total memory per GPU for Adam optimizer states, gradients, and BF16 parameters for a 13B-parameter model on a 64-GPU ZeRO Stage 3 cluster.
2. Why does Tensor Parallelism require fast intra-node interconnect (NVLink) while Pipeline Parallelism is more tolerant of slower inter-node links (InfiniBand)?
3. Explain the 1F1B micro-batch schedule in pipeline parallelism: what is the pipeline bubble fraction, and how does increasing the number of micro-batches reduce it?

Sources & further reading

- S. Rajbhandari et al. (2020). "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models". In: *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC20)*, pp. 1–16
- M. Shoeybi et al. (2019). "Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism". In
- D. Narayanan et al. (2021). "Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM". in: *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC21)*, pp. 1–15
- I. Loshchilov and F. Hutter (2019). "Decoupled Weight Decay Regularization". In: *International Conference on Learning Representations (ICLR)*
- J. Kaplan et al. (2020). "Scaling Laws for Neural Language Models". In: *arXiv preprint arXiv:2001.08361*

16 Evaluation of Language Models

Module 10, at a glance (Estimated Study: 4 hours • Level: Intermediate–Advanced)

Focus. Rigorous evaluation of LLMs across knowledge, reasoning, code, and truthfulness using standardised benchmarks: perplexity, MMLU, HellaSwag, HumanEval, and the HELM holistic evaluation framework.

Key concepts. Perplexity, bits-per-character, MMLU, HellaSwag, TruthfulQA, HumanEval pass@k, HELM, benchmark contamination, model cards.

After this module you will be able to compute perplexity from log-likelihood, interpret accuracy on standard benchmarks, use the `lm-evaluation-harness` framework, and identify benchmark contamination risks.

16.1 Perplexity and bits-per-character

Perplexity (PPL) is the primary intrinsic metric for language models. For a test corpus $\mathbf{x} = (x_1, \ldots, x_T)$:

$$\mathrm{PPL}(\mathbf{x}) = \exp\left(-\frac{1}{T}\sum_{t=1}^{T}\log_e P_\theta(x_t \mid x_1, \ldots, x_{t-1})\right). \tag{113}$$

A lower perplexity indicates better predictive performance. A PPL of k means the model is, on average, as uncertain as choosing uniformly among k equally likely next tokens. Perplexity is highly sensitive to tokenizer vocabulary: a model with a 100k-vocabulary tokenizer will have lower perplexity on the same text than one with a 32k vocabulary, simply because it produces fewer tokens. **Bits-per-character (BPC)** normalises by character count and is tokenizer-independent:

$$\mathrm{BPC}(\mathbf{x}) = -\frac{1}{C}\sum_{t=1}^{T}\log_2 P_\theta(x_t \mid x_{<t}), \tag{114}$$

where C is the total number of Unicode characters. Lower BPC is better; state-of-the-art LLMs achieve $\lesssim 1.0$ BPC on English Wikipedia.

16.2 Standard benchmarks

MMLU (Massive Multitask Language Understanding) (Hendrycks et al. 2021) measures breadth of world knowledge via 14,079 four-choice multiple-choice questions spanning 57 subjects (STEM, humanities, professional law, medicine). Performance is reported as accuracy; GPT-4 achieves ≈86%, human experts ≈89%.

HellaSwag (Zellers et al. 2019) tests commonsense reasoning via adversarially-filtered sentence-completion tasks; models must select the most plausible continuation from 4 options. GPT-2 scored 40%; GPT-4 scores ≈95%.

TruthfulQA measures the tendency to reproduce common misconceptions rather than accurate statements. Models optimised only for likelihood can score *below* humans because fluent falsehoods are common in pre-training corpora.

HumanEval (Chen et al. 2021) assesses functional code generation: 164 Python programming problems, each with a docstring, reference tests, and a canonical solution. The metric is *pass@k*,

the probability that at least one of k generated samples passes all unit tests:

$$\text{pass@}k = \mathbb{E}_{\text{problem}}\left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}\right], \tag{115}$$

where n samples are drawn and c pass all tests. This unbiased estimator avoids the variability of evaluating exactly k samples.

16.3 HELM and holistic evaluation

HELM (Holistic Evaluation of Language Models) (Liang et al. 2023) evaluates models across seven *metrics* (accuracy, calibration, robustness, fairness, bias, toxicity, efficiency) on 42 scenarios, revealing that no single model dominates across all dimensions. Key findings: models optimised for NLP benchmarks often exhibit high toxicity; smaller models can outperform larger ones on specific tasks; benchmark contamination — test data appearing in pre-training corpora — is widespread and difficult to detect.

16.4 Benchmark contamination and the evaluation stack

Contamination occurs when evaluation examples appear verbatim or paraphrased in pre-training data. Standard mitigations include: n-gram deduplication between training and test sets, canonical held-out test splits, and perplexity-based contamination detection (unusually low PPL on "unseen" examples signals contamination). The **lm-evaluation-harness** (Gao et al. 2020) is the standard open-source framework for reproducible zero-shot and few-shot evaluation across 60+ benchmarks, used by EleutherAI, Hugging Face Open LLM Leaderboard, and most academic comparisons.

Evaluation Best Practices

Always report: (1) exact prompt format and number of few-shot examples; (2) model size, quantization level, and decoding strategy; (3) benchmark version and evaluation framework; (4) evidence of contamination checks. A single-number accuracy on a single benchmark is rarely sufficient to characterise a model's capabilities.

Benchmark	Task type	Metric	What it tests
Perplexity (Wiki)	Language modelling	PPL/BPC	Fluency and compression
MMLU	4-choice QA	Accuracy (%)	World knowledge, 57 subjects
HellaSwag	4-choice completion	Accuracy (%)	Commonsense reasoning
TruthfulQA	4-choice QA	Accuracy (%)	Truthfulness vs. popular myths
HumanEval	Code generation	pass@k	Functional correctness
HELM	Multi-task	7 metrics	Holistic behaviour

Table 10. Key LLM evaluation benchmarks, their formats, and primary metrics.

Check your understanding

1. A model achieves perplexity 15 on a test set tokenized at 1.3 tokens per character. Compute the equivalent BPC.
2. Why is pass@k defined using an unbiased combinatorial estimator rather than simply generating exactly k samples?
3. Describe two mechanisms by which benchmark contamination inflates reported accuracy, and one method to detect it.

Sources & further reading

- D. Hendrycks et al. (2021). “Measuring Massive Multitask Language Understanding”. In
- P. Liang et al. (2023). “Holistic Evaluation of Language Models”. In: *Transactions on Machine Learning Research*
- M. Chen et al. (2021). “Evaluating Large Language Models Trained on Code”. In
- L. Gao et al. (2020). *The Pile: An 800GB Dataset of Diverse Text for Language Modeling*
- R. Zellers et al. (2019). “HellaSwag: Can a Machine Really Finish Your Sentence?” In: *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 4791–4800
- T. B. Brown et al. (2020). “Language Models Are Few-Shot Learners”. In: *Advances in Neural Information Processing Systems* 33, pp. 1877–1901

17 LLM Deployment and Serving

Module 11, at a glance (Estimated Study: 4 hours • Level: Advanced)

Focus. Serving large language models at production scale: memory management with PagedAttention (vLLM), continuous batching, structured outputs, speculative decoding, and an overview of TGI and TensorRT-LLM.

Key concepts. KV-cache fragmentation, PagedAttention, continuous batching, speculative decoding, TTFT, TPOT, structured generation, tensor compilation, FP8 inference.

After this module you will be able to explain why KV-cache fragmentation wastes GPU memory and how PagedAttention resolves it, compute serving throughput from TTFT and TPOT, and compare vLLM, TGI, and TensorRT-LLM for different deployment scenarios.

17.1 The KV-cache management problem

During autoregressive generation, each decoder layer caches the Key and Value projections for all preceding tokens (KV-cache). For a model with L layers, h heads, head dimension d_k, and maximum context n, the KV-cache per sequence occupies:

$$\text{KV_mem} = 2 \cdot L \cdot h \cdot d_k \cdot n \cdot \text{sizeof}(\text{dtype}) \text{ bytes.} \tag{116}$$

For Llama 3 70B (BF16, $L = 80$, $h = 8$, $d_k = 128$, $n = 8192$): $2 \times 80 \times 8 \times 128 \times 8192 \times 2 \approx 26$ GB per sequence. Serving multiple sequences simultaneously requires careful GPU memory management.

Naive implementations pre-allocate a contiguous memory block of size $n_{\max}$ per sequence at request arrival, resulting in **internal fragmentation** (unused space in over-allocated blocks) and **external fragmentation** (unusable gaps between blocks). Kwon et al. (2023) measure that up to 60–80% of GPU memory is wasted by fragmentation in naive KV-cache management.

17.2 PagedAttention and vLLM

PagedAttention (Kwon et al. 2023) adapts the OS virtual memory paradigm to KV caches. The KV-cache is divided into fixed-size **blocks** (analogous to memory pages), each holding B_k tokens of key/value tensors. A per-sequence **block table** maps logical KV positions to physical block locations:

$$\text{Attention}(q_t, K_{1:t}, V_{1:t}) = \sum_b \text{Attention}\big(q_t, K_b, V_b\big), \tag{117}$$

where blocks K_b, V_b are gathered by the block table indirection. This allows:

- **On-demand allocation**: blocks are allocated as new tokens are generated, eliminating internal fragmentation.
- **Block sharing**: multiple requests sharing a common prefix (system prompt) can share physical blocks via copy-on-write, reducing memory by up to 55% for high-prefix-reuse workloads.
- **Continuous batching**: the scheduler can swap blocks to/from CPU RAM, preempt sequences, and merge variable-length requests into a single GPU kernel call.

vLLM achieves 2–$24\times$ higher throughput than naive HuggingFace generation at equivalent quality.

17.3 Serving metrics

LLM serving is characterized by two latency metrics:

- **Time to First Token (TTFT)**: latency from request submission to the first generated token. Determined by prefill (processing the input prompt) compute and batch queuing time. Target: < 500 ms for interactive applications.
- **Time Per Output Token (TPOT)**: latency per subsequently generated token after the first. Determined by the decode step and KV-cache memory bandwidth. Target: < 50 ms/token (> 20 tokens/s) for perceptible streaming.

Throughput (tokens/s/GPU) is the primary cost metric for batch serving. The throughput–latency trade-off is controlled by *maximum batch size*: larger batches amortize model weight loading across more sequences, improving throughput, but increase queuing latency (higher TTFT).

17.4 Speculative decoding

Speculative decoding exploits the observation that a small *draft model* p_{draft} can propose γ tokens simultaneously, which the large *target model* p_{target} verifies in parallel in a single forward pass. Tokens are accepted if $u \leq p_{\text{target}}(x)/p_{\text{draft}}(x)$ for a uniform random variable $u \sim U[0,1]$; rejected tokens are resampled from the corrected distribution. The expected number of accepted tokens per target model call is:

$$\mathbb{E}[\text{accepted}] = \gamma - \sum_{t=1}^{\gamma} \max\bigl(0, p_{\text{draft}}(x_t) - p_{\text{target}}(x_t)\bigr), \tag{118}$$

yielding 2–$3\times$ wall-clock speedup with identical output distribution.

17.5 TGI and TensorRT-LLM

TGI (Text Generation Inference) by Hugging Face is an open-source serving framework implementing continuous batching, flash attention, and quantization (GPTQ, AWQ, bitsandbytes) with a REST/gRPC API, designed for straightforward deployment of HuggingFace-format models.

TensorRT-LLM by NVIDIA compiles LLMs into highly optimised GPU kernels using TensorRT graph compilation, fused attention, and INT8/FP8 inference targeting NVIDIA H100/A100 GPUs. It achieves the highest raw throughput on NVIDIA hardware by exploiting hardware-specific features (Tensor Cores, Transformer Engine), at the cost of a more complex build pipeline.

Deployment Framework Selection

- **vLLM**: best throughput-efficiency for NVIDIA and AMD GPUs; PagedAttention + continuous batching; active open-source community; easy to extend.
- **TGI**: tight HuggingFace ecosystem integration; straightforward API; good for rapid deployment and experimentation.
- **TensorRT-LLM**: maximum throughput on NVIDIA H100; requires FP8/INT8 calibration; ideal for production at scale on NVIDIA infrastructure.
- **llama.cpp + GGUF**: CPU and Apple Silicon deployment; no CUDA required; best for edge and laptop inference.

Check your understanding

1. Compute the KV-cache memory for Llama 3 8B (BF16, $L = 32$, 8 KV heads, $d_k = 128$) for a single sequence of length 8192 tokens. Then compute the maximum number of concurrent sequences on an 80 GB A100, assuming model weights require 16 GB.
2. Explain why speculative decoding produces *identical* output distribution to the target model, not merely an approximation.
3. A serving system must minimise TTFT for interactive chat while maximising throughput for batch summarisation on the same GPU. Describe the scheduling trade-off and a simple strategy to balance both objectives.

Sources & further reading

- W. Kwon et al. (2023). "Efficient Memory Management for Large Language Model Serving with PagedAttention". In: *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, pp. 611–626
- T. Dao et al. (2022). "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness". In: *Advances in Neural Information Processing Systems 35*, pp. 16344–16359
- T. Dao (2024). "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning". In: *International Conference on Learning Representations (ICLR)*
- E. Frantar et al. (2023). "GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers". In: *International Conference on Learning Representations (ICLR)*
- J. Lin et al. (2024). "AWQ: Activation-Aware Weight Quantization for LLM Compression and Acceleration". In: *Machine Learning and Systems (MLSys)*
- T. Dettmers et al. (2024). "QLoRA: Efficient Finetuning of Quantized LLMs". In: *Advances in Neural Information Processing Systems 36*
- T. Wolf et al. (2020). "Transformers: State-of-the-Art Natural Language Processing". In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pp. 38–45

Where to go next

The guide ends; the field does not. A few well-marked paths, matched to what you now know:

- **Strengthen the mathematics:** (Strang 2019; Wasserman 2004; Boyd and Vandenberghe 2004).
- **Go deep on classical ML:** (Hastie et al. 2009; Bishop 2006; Pedregosa et al. 2011).
- **Deep learning proper:** (Goodfellow et al. 2016; Bishop and Bishop 2024; Zhang et al. 2023).
- **Build with LLMs, LoRA & RAG:** Hugging Face ecosystem (Wolf et al. 2020), LoRA (Hu et al. 2022), and RAG (Lewis et al. 2020), read alongside (Vaswani et al. 2017; Brown et al. 2020; Ouyang et al. 2022; Rafailov et al. 2023).
- **Modern open-weight LLMs:** Llama 3 (Dubey et al. 2024), Mistral (Jiang et al. 2023), and Mixtral (Jiang et al. 2024); architectural deep-dives via FlashAttention (Dao et al. 2022; Dao 2024).
- **Efficient deployment and quantization:** GPTQ (Frantar et al. 2023), AWQ (Lin et al. 2024), vLLM (Kwon et al. 2023), and TensorRT-LLM for production serving.
- **Scale training:** ZeRO / FSDP (Rajbhandari et al. 2020) and 3D parallelism (Narayanan et al. 2021) for multi-GPU and multi-node setups.
- **Rigorous evaluation:** MMLU (Hendrycks et al. 2021), HELM (Liang et al. 2023), HumanEval (Chen et al. 2021), and the `lm-evaluation-harness` (Gao et al. 2020).
- **Stay grounded:** (Christian 2020; Bender et al. 2021).

Practical Coding Projects Series

To accompany the theoretical foundation developed across the eleven modules, a complete series of six substantial hands-on coding projects is provided in the repository's `projects/` directory (https://github.com/DanielBerns/AI-course). Each project bridges theoretical concepts to runnable Python implementation using industry-standard frameworks (`NumPy`, `Pandas`, `Scikit-Learn`, `XGBoost`, `PyTorch`, `Transformers`, and `vLLM`), with explicit emphasis on data cleaning, exploratory visualization, model diagnostics, resumable training, quantization, and inference pipelines.

Project 1: Data Pipeline, Exploratory Analysis & Algorithmic Bias Audit

Curriculum Mapping: Module 1 (The AI Landscape, Compute/Cloud, Data Pipelines, Ethics & Safety).

Overview & Learning Objectives: Hands-on data engineering and ethical auditing pipeline preceding model training on clinical dataset.

Project 2: Math Engine from Scratch (Linear Algebra, Stats & Optimization)

Curriculum Mapping: Module 2 (Mathematics and Data Foundations). Pure NumPy matrix calculus, PCA, Adam/AdamW optimizers.

Project 3: Classical Machine Learning Pipeline & Diagnostics

Curriculum Mapping: Module 3 (Classical Machine Learning). Supervised classification, DBSCAN/K-Means clustering, IsoForest anomaly detection, learning curves.

Project 4: Deep Learning Foundations & Resumable Training Pipeline

Curriculum Mapping: Module 4 (Neural Networks, Backpropagation, PyTorch/Keras). Manual matrix backprop engine, PyTorch MLP, checkpoint manager.

Project 5: Computer Vision & Sequential Architectures (CNNs & LSTMs)

Curriculum Mapping: Module 4 (CNNs, RNNs/LSTMs, Multi-Head Attention). PyTorch 2D CNN with Pre-LN Residual Skip Connections and PyTorch LSTM.

Project 6: Transformer Architectures & LLM Fine-Tuning Pipeline

Curriculum Mapping: Module 5 (Foundation Models, Generative AI & LLMs). Causal self-attention from scratch, RoPE, Hugging Face fine-tuning (SFT/LoRA), next-token sampling engine.

Appendix: Answer Key for Check Your Understanding

This appendix provides concise model answers for all self-check exercises across the eleven modules.

Module 1: Introduction and the AI Landscape

1. **Classification:** Chess engine, spam filter, and autonomous reasoning demos are all Narrow AI.
2. **GPU vs CPU throughput:** GPUs possess thousands of concurrent lightweight arithmetic cores optimized for parallel matrix tensor operations ($A\mathbf{x} + \mathbf{b}$).
3. **Job screening harms & mitigations:** (1) Historical bias propagation $\rightarrow$ audit for Equalized Odds / proxy features; (2) Representation gaps $\rightarrow$ human oversight & threshold calibration.

Module 2: Mathematics and Data Foundations

1. **Eigenvector of Covariance Matrix:** Specifies principal direction of maximum variance projection without linear distortion.
2. **Bayes' Theorem with 10% Prevalence:** $P(\text{disease} \mid +) = \frac{0.99 \times 0.10}{(0.99 \times 0.10) + (0.05 \times 0.90)} \approx 0.6875$ (68.8%).
3. **Divergence with $\eta = 1.1$ for $J(w) = w^2$:** Step $w_{t+1} = -1.2 w_t$ flips sign and expands magnitude by $1.2\times$, diverging to $\pm\infty$.

Module 3: Classical Machine Learning

1. **Cancer Screening Metric:** Prioritize Recall to minimize False Negatives; precision drops (more false alarms requiring follow-up tests).
2. **K-Means on Concentric Rings:** K-Means assumes convex spherical clusters; concentric rings require density-based clustering (DBSCAN) or spectral methods.
3. **K-Armed Bandit ϵ Trade-off:** Higher ϵ accelerates early exploration but caps asymptotic long-term return due to persistent suboptimal random actions.
4. **Cross-Validation Score Variance (0.62 to 0.91):** Indicates model instability or fold sample imbalance; use stratified k-fold, stronger regularization, or more data.
5. **Self-Supervised Learning & Self-Annotation:** SSL automatically splits or transforms unlabelled raw inputs to create pseudo-labels y_{target} without human effort (e.g., Masked Language Modeling self-annotates hidden words from surrounding text context).

Module 4: Neural Networks and Representation Learning

1. **MLP without Nonlinearity:** Collapses mathematically into a single linear map $\mathbf{a}^{(L)} = W_{\text{effective}}\mathbf{x}$, restricted to linear hyperplanes.
2. **Linear Undercomplete Autoencoder Subspace:** Under MSE loss, a linear undercomplete autoencoder bottleneck learns the exact principal subspace as Principal Component Analysis (PCA).
3. **Skip-Gram Negative Sampling (SGNS):** Converts expensive full-vocabulary softmax over $|V|$ into $K + 1$ binary logistic regressions per step, replacing $O(|V|)$ denominator summations with $O(K)$ binary updates.

4. **Multi-Head Attention Advantage:** Projects inputs into h distinct subspace heads in parallel, enabling the model to simultaneously attend to different aspects (e.g. syntactic agreement vs semantic reference) at different positions.

Module 5: Foundation Models and Generative AI

1. **SSL & Transformer Synergy:** SSL relies on pseudo-tasks (CLM/MLM) over unlabelled web-scale corpora containing trillions of tokens. Sequential RNNs process inputs sequentially ($O(N)$ steps) with memory bottlenecks in BPTT, making web-scale training computationally intractable. Transformers compute all pairwise token interactions in parallel ($O(1)$ sequential operations) and lack rigid local/temporal inductive biases, enabling parameter capacity to absorb massive scale without early performance saturation.
2. **InfoNCE Temperature τ:** Scales cosine similarities before log-softmax, controlling the relative gradient penalty weight assigned to hard negative samples vs. easy negatives.
3. **Subword Tokenization Trade-off:** Balances vocabulary size and sequence length by keeping frequent words intact while breaking rare words into subword pieces, avoiding out-of-vocabulary (OOV) tokens.
4. **Chinchilla Scaling Ratio:** Under a fixed compute budget $C \approx 6ND$, the compute-optimal strategy scales parameters N and training tokens D *equally*—approximately 20 tokens per parameter ($D^* \approx 20N$). This ratio minimises loss for a given C; it does not imply $N = D$ in absolute terms. When inference cost dominates (queries $\gg$ 1), practitioners overtrain smaller models beyond this optimum (*inference-optimal* regime, e.g. Llama 3), trading additional training compute for cheaper per-query deployment.
5. **LoRA Parameter Reduction:** Replaces full weight matrix update $\Delta W \in \mathbb{R}^{d\times k}$ with low-rank decomposition BA ($B \in \mathbb{R}^{d\times r}, A \in \mathbb{R}^{r\times k}$ with $r \ll \min(d,k)$), drastically reducing trainable parameters from $d \cdot k$ to $r(d+k)$.

Module 6: Modern LLM Architectures

1. **RMSNorm Variance vs. Mean Subtraction:** RMSNorm drops mean subtraction because empirical studies (Zhang and Sennrich 2019) demonstrate that activation scaling invariance is the primary factor stabilizing deep network dynamics. Eliminating mean centering reduces memory bandwidth and computation by $\approx$ 7–8% without degrading convergence or model quality.
2. **GQA FLOPs & KV-Cache Reduction:** Standard MHA uses h Query and h Key/Value heads. Grouped-Query Attention (GQA) uses h Query heads grouped into G KV heads ($1 \le G \le h$). Key/Value projection FLOPs scale down by a factor of h/G. Crucially, the inference KV-cache memory decreases from $O(h \cdot n \cdot d_k)$ to $O(G \cdot n \cdot d_k)$, achieving an exact h/G memory reduction factor ($4\times$ reduction in Llama 3 where $h = 32, G = 8$).
3. **MoE Auxiliary Load-Balancing Loss:** Without load balancing, router networks suffer from a "winner-take-all" collapse where a few experts handle almost all tokens while remaining experts stay underutilized and undertrained. The auxiliary loss $\mathcal{L}_{\text{aux}} = \alpha \cdot E \sum_{i=1}^{E} f_i \cdot P_i$ penalizes imbalance between actual routing frequencies f_i and routing probabilities P_i, forcing uniform token distribution across all E experts.

Module 7: Efficient Attention Mechanisms

1. **Attention Memory-Bandwidth Bottleneck:** Standard self-attention materializes the $n \times n$ attention matrix A in High-Bandwidth Memory (HBM), incurring $\Theta(nd_k + n^2)$ IO accesses per layer. Because modern GPUs feature massive arithmetic compute capacity relative to

HBM transfer rates, execution is bottlenecked by waiting for memory reads/writes (memory-bandwidth bound) rather than FLOP execution.

2. **FlashAttention Online Softmax Recurrence:** FlashAttention processes inputs in SRAM-sized tiles and maintains two running row statistics: running maximum $m^{(\text{old})}$ and running exponential sum $\ell^{(\text{old})}$. When a new score block arrives, $m^{(\text{new})} = \max(m^{(\text{old})}, \max(\mathbf{s}^{(\text{new})}))$ is computed, and previous sums are rescaled by $e^{m^{(\text{old})} - m^{(\text{new})}}$. This keeps exponent arguments non-positive (≤ 0), ensuring complete numerical stability against overflow while eliminating $O(n^2)$ HBM allocation.
3. **Positional Extrapolation (RoPE, ALiBi, Absolute):** Learned absolute embeddings cannot extrapolate beyond L_{train} because unseen indices lack weights. RoPE embeds relative angles via complex rotations; direct extrapolation past L_{train} degrades performance unless frequency scaling (NTK/YaRN) is applied. ALiBi adds a static distance penalty $-m \cdot |i - j|$ directly to attention logits, naturally extrapolating to sequence lengths $L' \gg L_{\text{train}}$ zero-shot without fine-tuning.

Module 8: Model Quantization

1. **Quantization Error Bound Derivation:** Symmetric b-bit uniform quantization maps $w \in [-w_{\max}, w_{\max}]$ to $w_q = \text{clip}(\text{round}(w/\Delta), -2^{b-1}, 2^{b-1} - 1)$ with scale $\Delta = \frac{\max(|W|)}{2^{b-1}-1}$. Because $\text{round}(x)$ rounds to the nearest integer, the rounding error is bounded by $|\text{round}(x) - x| \leq 0.5$. Re-scaling by Δ gives maximum absolute reconstruction error $|w - \hat{w}| \leq \frac{\Delta}{2}$ per element.
2. **GPTQ Inverse Hessian Compensation:** The Hessian $H = 2XX^\top$ measures the second-order curvature of output reconstruction loss with respect to weight perturbations. When column j is quantized, GPTQ uses the inverse Hessian row $[H^{-1}]_{j,:}$ to compute the optimal compensating weight update δW for unquantized columns, minimizing output distortion across calibration inputs X.
3. **AWQ Activation-Saliency Scaling:** AWQ observes that output error is dominated by the top $\approx 1\%$ of weight channels corresponding to large input activation magnitudes. By scaling up these salient weight channels by factor $s > 1$ before quantization (and dividing after), AWQ allocates finer quantization resolution to high-impact weights without incurring mixed-precision runtime overhead.

Module 9: Distributed Training

1. **ZeRO-3 Memory Calculation (13B Model):** For $P = 13 \times 10^9$ parameters, unsharded DDP requires $2P$ (BF16 params) + $2P$ (BF16 grads) + $12P$ (FP32 Adam states) $= 16P = 208$ GB per GPU. Under ZeRO Stage 3 across $D = 64$ GPUs, all parameters, gradients, and optimizer states are sharded equally: $\text{Memory}_{\text{per GPU}} = \frac{16P}{D} = \frac{208 \text{ GB}}{64} = 3.25$ GB per GPU.
2. **Interconnect Demands (TP vs. PP):** Tensor Parallelism (TP) splits weight matrices within every layer, requiring two high-frequency All-Reduce operations per layer for each forward and backward pass. This requires high-bandwidth, sub-microsecond NVLink interconnect (~ 900 GB/s). Pipeline Parallelism (PP) only passes activation tensors across stage boundary layers, communicating far less frequently and easily fitting within inter-node InfiniBand bandwidth (~ 50–100 GB/s).
3. **1F1B Pipeline Schedule & Bubble Reduction:** The 1F1B schedule alternates one forward and one backward micro-batch step on each GPU after a warmup period. The idle bubble fraction is $\frac{P-1}{M+P-1}$, where P is pipeline stages and M is micro-batches. Increasing $M \gg P$ reduces the bubble fraction toward zero, maximizing cluster utilization.

Module 10: Evaluation of Language Models

1. **Perplexity to BPC Conversion:** Perplexity $\mathrm{PPL} = 15$ yields cross-entropy $H = \log_2(15) \approx 3.907$ bits/token. Given $\mu = 1.3$ tokens/character, Bits-Per-Character is $\mathrm{BPC} = H \times \mu = 3.907 \times 1.3 \approx 5.079$ bits/character.
2. **Unbiased pass@k Estimator:** Evaluating exactly k generated samples introduces high variance across runs. The combinatorial estimator $\mathrm{pass@}k = \mathbb{E}\left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}\right]$ draws a larger sample pool $n \geq k$ with c correct solutions and computes the exact expected pass probability across all possible k-subsets, eliminating sampling variance.
3. **Contamination Inflation & Detection:** Contamination inflates accuracy via (1) verbatim memorization of test QA pairs and (2) task/format exposure in pre-training corpora. Detection is performed via Perplexity Drop Analysis: an anomalous drop in model perplexity on benchmark test sets relative to clean reference text indicates prior training exposure.

Module 11: LLM Deployment and Serving

1. **KV-Cache Footprint & GPU Concurrency:** For Llama 3 8B ($L = 32, h = 8, d_k = 128, n = 8192$ in BF16): $\mathrm{KV_mem} = 2 \times 32 \times 8 \times 128 \times 8192 \times 2 = 1.073$ GB per sequence (≈ 1.0 GB). On an 80 GB A100 with 16 GB reserved for weights, remaining VRAM is 64 GB, supporting $\frac{64\ \text{GB}}{1.0\ \text{GB/seq}} = 64$ concurrent sequences.
2. **Exactness of Speculative Decoding:** Speculative decoding uses rejection sampling with draft acceptance probability $\min\left(1, \frac{p_{\text{target}}(x)}{p_{\text{draft}}(x)}\right)$ and resamples rejected tokens from $p_{\text{adj}}(x) = \frac{\max(0, p_{\text{target}}(x) - p_{\text{draft}}(x))}{\sum_y \max(0, p_{\text{target}}(y) - p_{\text{draft}}(y))}$. Summing probabilities across accepted and rejected branches recovers $p_{\text{target}}(x)$ exactly.
3. **TTFT vs. Throughput Trade-off & Scheduling:** Large batch sizes maximize throughput during decode but increase prefill queuing delay (TTFT). Small batch sizes minimize TTFT but underutilize memory bandwidth. The trade-off is resolved using Chunked Prefills and Priority Queuing: prefill requests are chunked (e.g. 512 tokens) and piggybacked with decode steps, while interactive chat requests are prioritized over background batch tasks.

Glossary

Table 11. Quick reference for the vocabulary used in this guide.

Term	Meaning
Artificial intelligence (AI)	Broad field: systems that perform tasks associated with intelligent behaviour.
Machine learning (ML)	Sub-field: algorithms whose performance on a task improves from data.
Deep learning (DL)	ML with multi-layer neural networks that learn hierarchical representations.
Feature / label	Input measurement / target answer supplied during supervised training.
Model	Parameterised function fitted to data.
Loss function	Scalar score of single-example error; cost function averages loss over data.
Gradient	Vector of partial derivatives indicating the direction of steepest ascent.
Backpropagation	Efficient chain-rule algorithm computing loss gradients for all weights layer by layer.
Epoch	One complete pass over the entire training dataset during model optimization.
Hyperparameter	Configuration variable set before training rather than learned from data.
Regularization	Techniques (ℓ_1/ℓ_2, dropout, weight decay) constraining model complexity to prevent overfitting.
Vanishing gradient	Exponential decay of backpropagated error gradients toward zero in deep networks.
Attention	Mechanism dynamically weighting sequence representations based on query relevance.
Multi-Head Attention	Parallel attention mechanisms operating over distinct linear projection subspaces.
Inductive bias	Set of assumptions an algorithm uses to predict outputs for unseen inputs.
Overfitting	Memorising training data; high training performance, poor generalization.
Embedding	Dense vector representation encoding semantic relationships in continuous space.
Tokenization	Segmenting raw text into discrete integer vocabulary tokens (e.g. via BPE).
Token	Sub-word unit that language models process and generate.
Parameter	Learnable weight/bias inside a model; LLMs contain billions of parameters.
Fine-tuning	Adapting a pretrained model to a specific task via SFT, LoRA, or DPO.
Hallucination	Model output that is fluent and syntactically convincing but factually false.

References

Ainslie, J., J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai (2023). "GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints". In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 4895–4901.

Amodei, D., C. Olah, J. Steinhardt, et al. (2016). "Concrete Problems in AI Safety". In: *arXiv preprint arXiv:1606.06565*.

Bender, E. M., T. Gebru, A. McMillan-Major, and S. Shmitchell (2021). "On the Dangers of Stochastic Parrots: Can Language Models Be Too Big?" In: *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency (FAccT)*, pp. 610–623.

Bengio, Y., A. Courville, and P. Vincent (2013). "Representation Learning: A Review and New Perspectives". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 35.8, pp. 1798–1828.

Bengio, Y., P. Simard, and P. Frasconi (1994). "Learning Long-Term Dependencies with Gradient Descent is Difficult". In: *IEEE Transactions on Neural Networks* 5.2, pp. 157–166.

Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. New York: Springer.

Bishop, C. M. and H. Bishop (2024). *Deep Learning: Foundations and Concepts*. Cham: Springer.

Boyd, S. and L. Vandenberghe (2004). *Convex Optimization*. Cambridge: Cambridge University Press.

Breiman, L. (2001). "Random Forests". In: *Machine Learning* 45.1, pp. 5–32.

Brown, T. B., B. Mann, N. Ryder, et al. (2020). "Language Models Are Few-Shot Learners". In: *Advances in Neural Information Processing Systems 33*, pp. 1877–1901.

Buolamwini, J. and T. Gebru (2018). "Gender Shades: Intersectional Accuracy Disparities in Commercial Gender Classification". In: *Proceedings of the 1st Conference on Fairness, Accountability and Transparency*, pp. 77–91.

Chen, M. et al. (2021). "Evaluating Large Language Models Trained on Code". In.

Chen, T. and C. Guestrin (2016). "XGBoost: A Scalable Tree Boosting System". In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794.

Chen, T., S. Kornblith, M. Norouzi, and G. Hinton (2020). "A Simple Framework for Contrastive Learning of Visual Representations". In: *Proceedings of the 37th International Conference on Machine Learning (ICML)*, pp. 1597–1607.

Child, R., S. Gray, A. Radford, and I. Sutskever (2019). "Generating Long Sequences with Sparse Transformers". In.

Christian, B. (2020). *The Alignment Problem: Machine Learning and Human Values*. New York: W. W. Norton & Company.

Christiano, P. F., J. Leike, T. B. Brown, M. Martic, S. Legg, and D. Amodei (2017). "Deep Reinforcement Learning from Human Preferences". In: *Advances in Neural Information Processing Systems 30*, pp. 4299–4307.

Cortes, C. and V. Vapnik (1995). "Support-Vector Networks". In: *Machine Learning* 20.3, pp. 273–297.

Dao, T. (2024). "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning". In: *International Conference on Learning Representations (ICLR)*.

Dao, T., D. Y. Fu, S. Ermon, A. Rudra, and C. Ré (2022). "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness". In: *Advances in Neural Information Processing Systems 35*, pp. 16344–16359.

Deisenroth, M. P., A. A. Faisal, and C. S. Ong (2020). *Mathematics for Machine Learning*. Cambridge: Cambridge University Press.

Dettmers, T., M. Lewis, Y. Belkada, and L. Zettlemoyer (2022). "LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale". In: *Advances in Neural Information Processing Systems 35*, pp. 30318–30332.

Dettmers, T., A. Pagnoni, A. Holtzman, and L. Zettlemoyer (2024). "QLoRA: Efficient Finetuning of Quantized LLMs". In: *Advances in Neural Information Processing Systems 36*.

Devlin, J., M.-W. Chang, K. Lee, and K. Toutanova (2019). "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding". In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, pp. 4171–4186.

Dosovitskiy, A. et al. (2021). “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”. In: *International Conference on Learning Representations (ICLR)*.

Dubey, A., A. Jauhri, A. Pandey, et al. (2024). “The Llama 3 Herd of Models”. In: *arXiv preprint arXiv:2407.21783*.

Ester, M., H.-P. Kriegel, J. Sander, and X. Xu (1996). “A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise”. In: *Proceedings of the 2nd International Conference on Knowledge Discovery and Data Mining (KDD)*, pp. 226–231.

Fawcett, T. (2006). “An Introduction to ROC Analysis”. In: *Pattern Recognition Letters* 27.8, pp. 861–874.

Frantar, E., S. Ashkboos, T. Hoefler, and D. Alistarh (2023). “GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers”. In: *International Conference on Learning Representations (ICLR)*.

Gao, L. et al. (2020). *The Pile: An 800GB Dataset of Diverse Text for Language Modeling*.

Glorot, X. and Y. Bengio (2010). “Understanding the Difficulty of Training Deep Feedforward Neural Networks”. In: *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS)*, pp. 249–256.

Golub, G. H. and C. F. Van Loan (2013). *Matrix Computations*. 4th ed. Baltimore, MD: Johns Hopkins University Press.

Goodfellow, I., Y. Bengio, and A. Courville (2016). *Deep Learning*. Cambridge, MA: MIT Press.

Goodfellow, I. et al. (2014). “Generative Adversarial Nets”. In: *Advances in Neural Information Processing Systems 27*, pp. 2672–2680.

Harris, C. R., K. J. Millman, S. J. van der Walt, et al. (2020). “Array Programming with NumPy”. In: *Nature* 585, pp. 357–362.

Hastie, T., R. Tibshirani, and J. Friedman (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2nd ed. New York: Springer.

He, K., X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick (2022). “Masked Autoencoders Are Scalable Vision Learners”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 16000–16009.

He, K., X. Zhang, S. Ren, and J. Sun (2015). “Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification”. In: *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pp. 1026–1034.

He, K., X. Zhang, S. Ren, and J. Sun (2016). “Deep Residual Learning for Image Recognition”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778.

Hendrycks, D. et al. (2021). “Measuring Massive Multitask Language Understanding”. In.

Ho, J., A. Jain, and P. Abbeel (2020). “Denoising Diffusion Probabilistic Models”. In: *Advances in Neural Information Processing Systems 33*, pp. 6840–6851.

Hochreiter, S. and J. Schmidhuber (1997). “Long Short-Term Memory”. In: *Neural Computation* 9.8, pp. 1735–1780.

Hoffmann, J. et al. (2022). “Training Compute-Optimal Large Language Models”. In: *arXiv preprint arXiv:2203.15556*.

Hornik, K., M. Stinchcombe, and H. White (1989). “Multilayer Feedforward Networks Are Universal Approximators”. In: *Neural Networks* 2.5, pp. 359–366.

Hu, E. J. et al. (2022). “LoRA: Low-Rank Adaptation of Large Language Models”. In: *International Conference on Learning Representations (ICLR)*.

Jiang, A. Q. et al. (2023). “Mistral 7B”. In: *arXiv preprint arXiv:2310.06825*.

Jiang, A. Q. et al. (2024). “Mixtral of Experts”. In: *arXiv preprint arXiv:2401.04088*.

Jouppi, N. P., C. Young, N. Patil, et al. (2017). “In-Datacenter Performance Analysis of a Tensor Processing Unit”. In: *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA)*, pp. 1–12.

Kaplan, J., S. McCandlish, T. Henighan, et al. (2020). “Scaling Laws for Neural Language Models”. In: *arXiv preprint arXiv:2001.08361*.

Kingma, D. P. and J. Ba (2015). “Adam: A Method for Stochastic Optimization”. In: *International Conference on Learning Representations (ICLR)*, pp. 1–15.

Kingma, D. P. and M. Welling (2014). “Auto-Encoding Variational Bayes”. In: *International Conference on Learning Representations (ICLR)*, pp. 1–9.

Krizhevsky, A., I. Sutskever, and G. E. Hinton (2012). “ImageNet Classification with Deep Convolutional Neural Networks”. In: *Advances in Neural Information Processing Systems 25*, pp. 1097–1105.

Kwon, W. et al. (2023). “Efficient Memory Management for Large Language Model Serving with PagedAttention”. In: *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, pp. 611–626.

LeCun, Y., Y. Bengio, and G. Hinton (2015). “Deep Learning”. In: *Nature* 521.7553, pp. 436–444.

LeCun, Y., L. Bottou, Y. Bengio, and P. Haffner (1998). “Gradient-Based Learning Applied to Document Recognition”. In: *Proceedings of the IEEE* 86.11, pp. 2278–2324.

Lewis, P. et al. (2020). “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Advances in Neural Information Processing Systems 33*, pp. 9459–9474.

Liang, P. et al. (2023). “Holistic Evaluation of Language Models”. In: *Transactions on Machine Learning Research*.

Lin, J. et al. (2024). “AWQ: Activation-Aware Weight Quantization for LLM Compression and Acceleration”. In: *Machine Learning and Systems (MLSys)*.

Liu, F. T., K. M. Ting, and Z.-H. Zhou (2008). “Isolation Forest”. In: *Proceedings of the 8th IEEE International Conference on Data Mining (ICDM)*, pp. 413–422.

Loshchilov, I. and F. Hutter (2019). “Decoupled Weight Decay Regularization”. In: *International Conference on Learning Representations (ICLR)*.

Maaten, L. van der and G. Hinton (2008). “Visualizing Data using t-SNE”. In: *Journal of Machine Learning Research* 9, pp. 2579–2605.

MacQueen, J. (1967). “Some Methods for Classification and Analysis of Multivariate Observations”. In: *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*. Vol. 1, pp. 281–297.

McInnes, L., J. Healy, and J. Melville (2018). “UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction”. In: *arXiv preprint arXiv:1802.03426*.

McKinney, W. (2010). “Data Structures for Statistical Computing in Python”. In: *Proceedings of the 9th Python in Science Conference (SciPy)*, pp. 56–61.

Mikolov, T., I. Sutskever, K. Chen, G. S. Corrado, and J. Dean (2013). “Distributed Representations of Words and Phrases and Their Compositionality”. In: *Advances in Neural Information Processing Systems 26*, pp. 3111–3119.

Minsky, M. and S. Papert (1969). *Perceptrons: An Introduction to Computational Geometry*. Cambridge, MA: MIT Press.

Mitchell, T. M. (1997). *Machine Learning*. New York: McGraw-Hill.

Mnih, V., K. Kavukcuoglu, D. Silver, et al. (2015). “Human-level Control Through Deep Reinforcement Learning”. In: *Nature* 518.7540, pp. 529–533.

Nair, V. and G. E. Hinton (2010). “Rectified Linear Units Improve Restricted Boltzmann Machines”. In: *Proceedings of the 27th International Conference on Machine Learning (ICML)*, pp. 807–814.

Narayanan, D. et al. (2021). “Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM”. In: *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC21)*, pp. 1–15.

Oord, A. van den, Y. Li, and O. Vinyals (2018). “Representation Learning with Contrastive Predictive Coding”. In: *arXiv preprint arXiv:1807.03748*.

Ouyang, L., J. Wu, X. Jiang, et al. (2022). “Training Language Models to Follow Instructions with Human Feedback”. In: *Advances in Neural Information Processing Systems 35*, pp. 27730–27744.

Patterson, D., J. Gonzalez, Q. Le, et al. (2021). “Carbon Emissions and Large Neural Network Training”. In: *Advances in Neural Information Processing Systems 34*, pp. 13470–13481.

Pearson, K. (1901). “On Lines and Planes of Closest Fit to Systems of Points in Space”. In: *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science* 2.11, pp. 559–572.

Pedregosa, F., G. Varoquaux, A. Gramfort, et al. (2011). “Scikit-learn: Machine Learning in Python”. In: *Journal of Machine Learning Research* 12, pp. 2825–2830.

Press, O., N. A. Smith, and M. Lewis (2022). “Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation”. In: *International Conference on Learning Representations (ICLR)*.

Radford, A., K. Narasimhan, T. Salimans, and I. Sutskever (2018). “Improving Language Understanding by Generative Pre-Training”. In: *OpenAI Blog*.

Rafailov, R., A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn (2023). “Direct Preference Optimization: Your Language Model is Secretly a Reward Model”. In: *Advances in Neural Information Processing Systems 36*.

Rajbhandari, S., J. Rasley, O. Ruwase, and Y. He (2020). “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models”. In: *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC20)*, pp. 1–16.

Rosenblatt, F. (1958). “The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain”. In: *Psychological Review* 65.6, pp. 386–408.

Rousseeuw, P. J. (1987). “Silhouettes: A Graphical Aid to the Interpretation and Validation of Cluster Analysis”. In: *Journal of Computational and Applied Mathematics* 20, pp. 53–65.

Rumelhart, D. E., G. E. Hinton, and R. J. Williams (1986). “Learning Representations by Back-Propagating Errors”. In: *Nature* 323.6088, pp. 533–536.

Russell, S. J. and P. Norvig (2021). *Artificial Intelligence: A Modern Approach*. 4th ed. Hoboken, NJ: Pearson.

Saxe, A. M., J. L. McClelland, and S. Ganguli (2014). “Exact Solutions to the Nonlinear Dynamics of Learning in Deep Linear Neural Networks”. In: *International Conference on Learning Representations (ICLR)*.

Schulman, J., F. Wolski, P. Dhariwal, A. Radford, and O. Klimov (2017). “Proximal Policy Optimization Algorithms”. In.

Searle, J. R. (1980). “Minds, Brains, and Programs”. In: *Behavioral and Brain Sciences* 3.3, pp. 417–424.

Shazeer, N. (2020). “GLU Variants Improve Transformer”. In: *arXiv preprint arXiv:2002.05202*.

Shazeer, N. et al. (2017). “Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer”. In: *International Conference on Learning Representations (ICLR)*.

Shoeybi, M., M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro (2019). “Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism”. In.

Silver, D., A. Huang, C. J. Maddison, et al. (2016). “Mastering the Game of Go with Deep Neural Networks and Tree Search”. In: *Nature* 529.7587, pp. 484–489.

Strang, G. (2012). *Linear Algebra and Its Applications*. 4th ed. Cengage Learning.

Strang, G. (2019). *Linear Algebra and Learning from Data*. Wellesley, MA: Wellesley–Cambridge Press.

Strubell, E., A. Ganesh, and A. McCallum (2019). “Energy and Policy Considerations for Deep Learning in NLP”. In: *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 3645–3650.

Su, J., M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu (2024). “RoFormer: Enhanced Transformer with Rotary Position Embedding”. In: *Neurocomputing* 568, p. 127063.

Sutton, R. S. and A. G. Barto (2018). *Reinforcement Learning: An Introduction*. 2nd ed. Cambridge, MA: MIT Press.

Touvron, H. et al. (2023). “Llama 2: Open Foundation and Fine-Tuned Chat Models”. In: *arXiv preprint arXiv:2307.09288*.

Trefethen, L. N. and D. Bau (1997). *Numerical Linear Algebra*. Philadelphia, PA: SIAM.

Turing, A. M. (1950). “Computing Machinery and Intelligence”. In: *Mind* 59.236, pp. 433–460.

VanderPlas, J. (2016). *Python Data Science Handbook: Essential Tools for Working with Data*. Sebastopol, CA: O’Reilly Media.

Vaswani, A. et al. (2017). “Attention Is All You Need”. In: *Advances in Neural Information Processing Systems 30*, pp. 5998–6008.

Vincent, P., H. Larochelle, Y. Bengio, and P.-A. Manzagol (2008). “Extracting and Composing Robust Features with Denoising Autoencoders”. In: *Proceedings of the 25th International Conference on Machine Learning (ICML)*, pp. 1096–1103.

Wasserman, L. (2004). *All of Statistics: A Concise Course in Statistical Inference*. New York: Springer.

Watkins, C. J. C. H. and P. Dayan (1992). “Q-Learning”. In: *Machine Learning* 8.3-4, pp. 279–292.

Wolf, T. et al. (2020). “Transformers: State-of-the-Art Natural Language Processing”. In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pp. 38–45.

Zellers, R., A. Holtzman, Y. Bisk, A. Farhadi, and Y. Choi (2019). “HellaSwag: Can a Machine Really Finish Your Sentence?” In: *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 4791–4800.

Zhang, A., Z. C. Lipton, M. Li, and A. J. Smola (2023). *Dive into Deep Learning*. Cambridge: Cambridge University Press.

Zhang, B. and R. Sennrich (2019). “Root Mean Square Layer Normalization”. In: pp. 12360–12371.

End of document